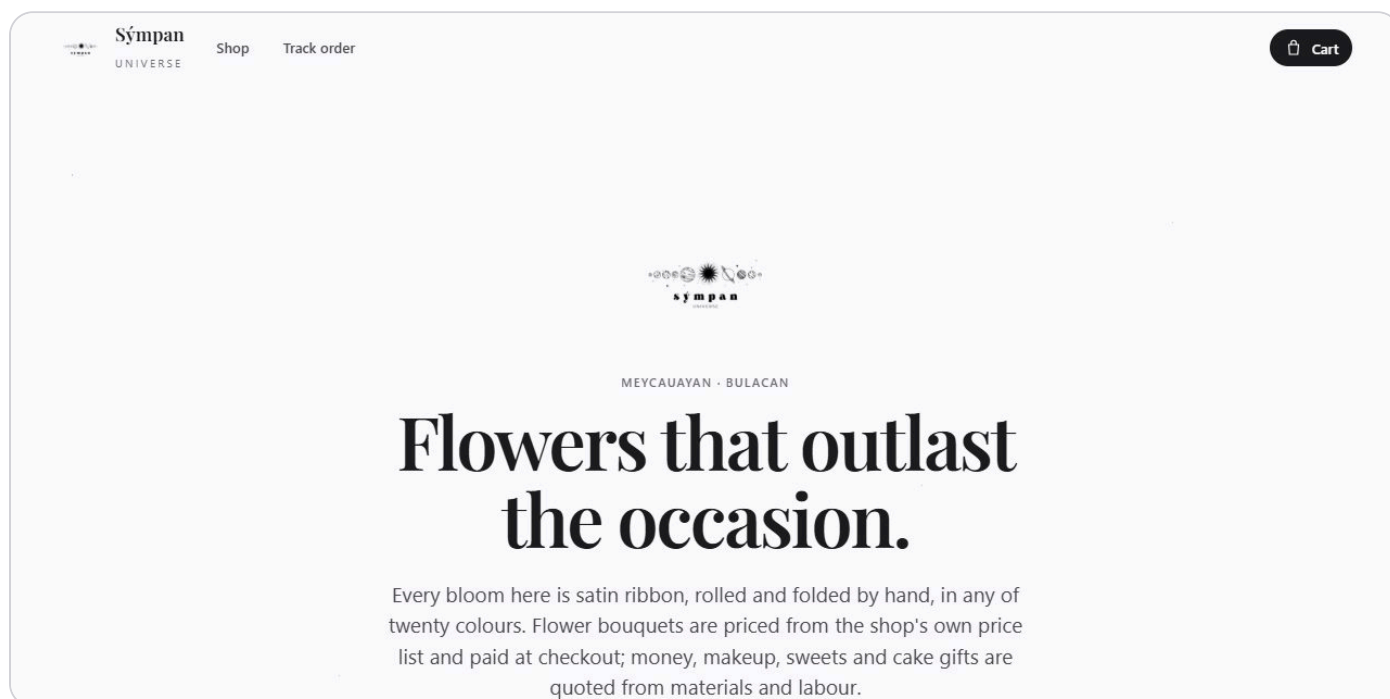


# Sýmpan Universe Defense Reviewer

Our module-by-module manual: the problem each part solves, its input, process and output, the data structures and algorithms inside, their time and space complexity, why we chose them, how to demonstrate them live — and a plain-language walk through the code.



## Part 1 · The shop window

1 Catalogue & Best Sellers · 2 Category Navigation · 3 Search & Sort

## Part 2 · Choosing a gift

4 Flower Customiser · 5 Gift Customiser · 6 Cart

## Part 3 · Checkout and payment

7 Checkout & Delivery · 8 Quotation Desk · 9 Payment, Receipts & Email

## Part 4 · Behind the counter

10 Production & Tracking · 11 Order Records · 12 Order Desk

Sýmpan Universe — handcrafted ribbon bouquets and gift cakes, Lawa, Meycauayan, Bulacan · Built with HTML, CSS, Bootstrap 5 and JavaScript, with every data structure and algorithm written by hand · October 2026

# How to use this reviewer

Every module has the same layout, in the same order the panel asks the questions:

## 1 • Business process

What task or problem of the client (Sýmpan Universe) the module solves.

## 2 • Input

The data the module needs — what the user types or picks, and the stored records it reads.

## 3 • Process

The data structure and algorithm used, step by step, with an Input → Process → Output picture.

## 4 • Output

What the module produces: records, screens, messages.

## 5 • Complexity analysis

Time and space complexity of each operation, and what  $n$  means.

## 6 • Justification

Why that data structure or algorithm, and why it fits the problem better than the alternatives.

## 7 • Demonstration

How to show it live, real screenshots, and a trace computed from the real code and data.

## Code walkthrough

The actual functions from our files (with line numbers), a basic explanation of each step, and their time and space complexity.

Each of the four parts also has a **one-page summary** before its modules: a short pitch, the structures to be able to explain, and likely panel questions with answers. All numbers in the traces were computed by running our real JavaScript on the demo data — they match the screenshots.

## Running the demo

1. Open `index.html` by double-clicking it (Edge or Chrome). No server is needed; the internet is used only for Bootstrap's stylesheet and for sending email.
2. The order desk is at `index.html#admin` — nothing on the shop links to it. Sign in with:

| Role  | E-mail                           | Password | Then                                                                                                     |
|-------|----------------------------------|----------|----------------------------------------------------------------------------------------------------------|
| Owner | kurlchester31feliciano@gmail.com | admin123 | the six-digit code sent to that address (shown on screen, clearly labelled, while EmailJS is not set up) |

3. Sample order for *Track order*: SU-201 with mobile 0917 555 0142. For a payment, any 13-digit GCash reference not used before, e.g. 1234567890123.
4. **Reloading the page resets all data** to the demo history (the arrays live in memory). Demo dates are relative to today, so there are always orders due this week, quotations waiting, and so on.

5. "Undo last" in the order desk reverses the last "Mark ready".

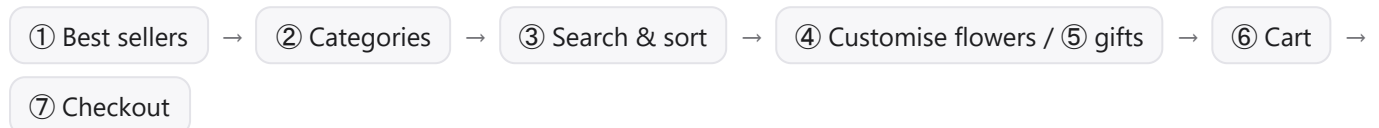
## Contents

1. System overview and the project rules
2. Data structures and algorithms used — a primer
3. Part 1 · The shop window — Module 1 Catalogue & Best Sellers · Module 2 Category Navigation · Module 3 Search & Sort
4. Part 2 · Choosing a gift — Module 4 Flower Customiser · Module 5 Gift Customiser · Module 6 Cart
5. Part 3 · Checkout and payment — Module 7 Checkout & Delivery · Module 8 Quotation Desk · Module 9 Payment, Receipts & Email
6. Part 4 · Behind the counter — Module 10 Production & Tracking · Module 11 Order Records · Module 12 Order Desk
7. Appendix — complexity summary, general panel questions, glossary, file map

# System overview

Sýmpan Universe makes handcrafted ribbon bouquets and gift cakes in Lawa, Meycauayan, Bulacan. Without a system, every order is a chat thread: the owner prices each request by hand, keeps payments and pickup dates in notes, and answers "is my order ready?" one message at a time. The website handles the customer's whole order — browsing, customising, checkout, payment, tracking — and the owner's work behind it.

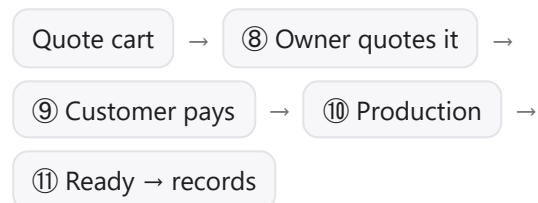
## The customer and owner flow



A cart of flowers or picture bouquets (priced from the price list)



A cart with a money, makeup, sweets, diaper or beer gift (quoted by the owner)



⑫ The owner signs in (password + emailed code), watches the Overview by day, month or year, and edits orders and products.

## How the program is built

### One page, many screens

Arrays live in the browser's memory, and memory is wiped whenever a new page opens — data cannot transfer between separate HTML files. So the whole site is one `index.html`: the shop and the order desk (`#admin`) are sections shown and hidden. The arrays survive every screen change.

### Layers

`js/core.js`, `structures.js`, `algorithms.js` — Our own array, text and date helpers; the linked list, circular queue, min-heap, stack, n-ary tree and hash table; the sorts, searches, recursion and greedy algorithm.

`js/data.js`, `store.js`, `orders.js`, `notify.js`, `editing.js`, `reports.js` — The "database" (arrays) and the business rules: the catalogue, cart, checkout, quotations, payments, production, records, emails and reports.

`js/ui.js`, `motion.js`, `mail.js`, `receipt.js` — Screens, sheets, animation, the email sender (EmailJS) and receipts.

`js/shop-*.js`, `admin-*.js`, `app.js` — One script per screen: the shop, and the order desk at `index.html#admin`.

## The data

| Array / structure                   | Key                                    | How it stays in order                                                         | Used by modules |
|-------------------------------------|----------------------------------------|-------------------------------------------------------------------------------|-----------------|
| orders                              | order number (201, 202 ...)            | numbers only grow → append, $O(1)$ ; the array stays sorted for binary search | 1, 7–12         |
| products                            | product id (101 ...)                   | the fixed catalogue, edited in place                                          | 1–5, 12         |
| receipts, outbox                    | receipt / email number                 | append, $O(1)$                                                                | 9, 12           |
| basket                              | position                               | singly linked list, in the order added                                        | 6               |
| quoteQueue, standardLane, mailQueue | order id / outbox position             | circular queues, first in first out                                           | 8, 9, 10        |
| rushLane                            | (due date, arrival)                    | min-heap                                                                      | 10              |
| completedStack                      | { id, position }                       | stack, last in first out                                                      | 11              |
| categoryTree                        | slug                                   | n-ary tree stored in one array                                                | 2               |
| referenceIndex, areaIndex           | "flower arrangement colour"; city name | hash tables with chaining                                                     | 4, 7            |

## The project rules and how we met them

| Rule from our instructor                                                                                                                                                                                                                                    | How the code follows it                                                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HTML, CSS, Bootstrap and JavaScript only; JS is the "backend"                                                                                                                                                                                               | No server, no framework, no database program. Bootstrap 5.3 gives the grid; everything else is our own code.                                                                                                                                              |
| Data cannot transfer between pages (arrays)                                                                                                                                                                                                                 | One page, index.html: the shop and the order desk (#admin) are shown and hidden, so the arrays survive every screen change. No localStorage, no cookies, no server. A reload starts again from the demo data.                                             |
| Procedural, not OOP                                                                                                                                                                                                                                         | Only functions and plain records: no class, this, prototype or new (except new Date once, to read the clock). A queue is the record { items, head, tail, count, capacity } plus functions like cqEnqueue(queue, value).                                   |
| No built-in helpers: push, pop, shift, unshift, splice, slice, concat, sort, reverse, indexOf, includes, find, filter, map, forEach, reduce, some, every, join; split, replace, trim, toLowerCase, toUpperCase, padStart, substring; no regular expressions | All replaced by our own functions in js/core.js, js/structures.js and js/algorithms.js — listAdd, copyRange, keepWhere, firstWhere, cutText, strip, toLower, insertionSort, binarySearch ... tests/rubric.js scans every script and fails on any of them. |
| State the time and space complexity, using only $O(1)$ , $O(\log n)$ , $O(n)$ and $O(n^2)$                                                                                                                                                                  | Written above every function in js/. The rubric fails on a missing note or on any other notation — which is why there is no merge sort: it is $O(n \log n)$ .                                                                                             |
| Use Date only to read the clock                                                                                                                                                                                                                             | readClock() holds the only new Date. Adding days, weekdays, month lengths and leap years are our own day-number arithmetic (dayNumber, dateOfDayNumber in js/core.js).                                                                                    |

Built-ins that are used are only for input and output: the page itself (the DOM), Date (to read the clock), Math, Number, String, charAt and charCodeAt, the length of an array or text, timers, crypto.getRandomValues (the six-digit sign-in code), and fetch with JSON.stringify to hand an email to EmailJS.

# Data structures & algorithms primer

$n$  = the size of whatever an operation goes through (records, items in a queue, or the characters of a text). Only the four notations from class are used:  $O(1)$ ,  $O(\log n)$ ,  $O(n)$ ,  $O(n^2)$ . All of these live in `js/structures.js` and `js/algorithms.js`.

## Array (our "table")

|            |            |            |                |
|------------|------------|------------|----------------|
| 201<br>[0] | 202<br>[1] | 203<br>[2] | 204<br>[3] new |
|------------|------------|------------|----------------|

`listAdd` writes at index `length`:  $O(1)$ . Order numbers only grow, so appending keeps the orders array sorted.  
Reading `a[i]`:  $O(1)$ .

## Linear search — $O(n)$

|             |                 |                 |           |
|-------------|-----------------|-----------------|-----------|
| Rose<br>1st | Plumeria<br>2nd | Dahlia<br>match | Sunflower |
| Money       |                 |                 |           |

Check the items one by one until one matches. Works on any array, sorted or not. Best case  $O(1)$ , worst  $O(n)$ . The search box and many look-ups use it.

## Binary search — $O(\log n)$

|     |     |     |               |     |     |
|-----|-----|-----|---------------|-----|-----|
| 203 | 204 | 205 | 206<br>middle | 207 | 208 |
|-----|-----|-----|---------------|-----|-----|

Needs a sorted array. Look at the middle; if the target is bigger keep the right half, else the left. Each step halves the range: 22 orders need at most 5 steps. Finds an order by number.

## Insertion sort — best $O(n)$ , worst $O(n^2)$ , stable

|      |        |      |               |
|------|--------|------|---------------|
| Beer | Dahlia | Rose | Money<br>held |
|------|--------|------|---------------|

Take the next item and slide the larger items before it one place right until its place is found. Very fast when the list is already almost in order — ours usually are — so it is our general sort.

## Selection sort — always $O(n^2)$

|                   |          |         |          |
|-------------------|----------|---------|----------|
| Rose 7<br>largest | Dahlia 3 | Money 5 | Diaper 2 |
|-------------------|----------|---------|----------|

For each position, scan the rest for the item that belongs there and move it in. Always  $n(n-1)/2$  comparisons. Ranks the best sellers; ours shifts instead of swapping, so it is stable.

## Naive string matching — $O(n^2)$

|   |   |   |   |   |   |   |             |
|---|---|---|---|---|---|---|-------------|
| m | a | t | c | h | a | . | r<br>start? |
|---|---|---|---|---|---|---|-------------|

Try the search word at every starting position of the text and compare letter by letter. "matcha" finds the four flower bouquets through their colour names.

## Singly linked list — $O(1)$ append

|              |   |         |   |               |
|--------------|---|---------|---|---------------|
| Rose<br>head | → | Picture | → | Money<br>tail |
|--------------|---|---------|---|---------------|

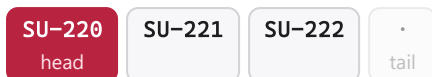
Nodes in one array, each with the index of the next. Adding at the tail is  $O(1)$ ; removing re-points one link. The cart.

## Stack — LIFO, push / pop $O(1)$

|        |        |               |
|--------|--------|---------------|
| SU-214 | SU-215 | SU-216<br>top |
|--------|--------|---------------|

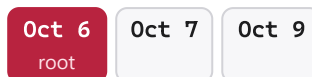
Record { items, top }. "Mark ready" pushes { order, its place }; Undo pops the newest. Also the depth-first walks of the category tree.

### Circular queue — FIFO, $O(1)$



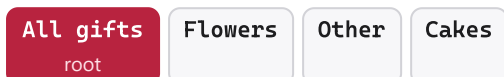
Record { items, head, tail, count, capacity }; the tail moves to  $(tail + 1) \% capacity$ , so nothing is shifted. Quote requests, standard production and outgoing email.

### Min-heap — $O(\log n)$



An array where every parent comes before its children; the earliest due date is always at index 0. Insert sifts up, extract sifts down:  $O(\log n)$ . The rush lane.

### N-ary tree — $O(n)$ walk



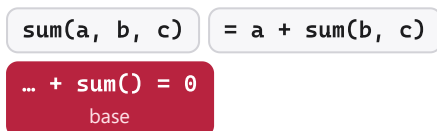
Each node keeps its parent and its children's indices. The shop's three columns and their ten lines; "everything under a branch" is its leaves.

### Hash table — $O(1)$ average



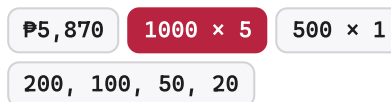
FNV-1a turns a key into a bucket number; keys that share a bucket (a collision) form a short chain. The 160 photographs and the delivery areas.

### Recursion — $O(n)$



A function that calls itself on the rest of the list, ending at the empty list. Adds up the add-ons and the quote lines.

### Greedy — $O(n)$



Take the largest note that fits, then the next. For the peso this always gives the fewest bills. The money bouquet.

### Why procedural?

Every structure is a plain record plus functions that receive it — `stackPush(stack, value)`, `cqEnqueue(queue, value)`, `hashPut(table, key, value)` — instead of objects with methods. This follows the project rule and makes each step of the algorithm visible.

## PART 1

# The shop window

Module 1 Catalogue & Best Sellers · Module 2 Category Navigation · Module 3 Search & Sort

## 60-second pitch

These modules are what a customer sees first. Catalogue & Best Sellers ranks the products by the pieces really sold — one pass over the paid orders with a hash table, then a selection sort. Category Navigation stores the shop's three columns as an n-ary tree, so "everything under Flower Bouquets" is a depth-first walk with a stack. Search & Sort finds words anywhere in a product with a linear search and naive string matching, and orders the results with a stable insertion sort.

## Structures and algorithms to be able to explain

| Where                 | Structure / algorithm                                    | Time                                 | Space       |
|-----------------------|----------------------------------------------------------|--------------------------------------|-------------|
| Count pieces sold     | Hash table (product id → tally row) over the order lines | $O(n^2)$ – every line of every order | $O(n)$      |
| Rank the best sellers | Selection sort (stable version)                          | $O(n^2)$ always                      | $O(n)$ copy |
| Branch → its lines    | N-ary tree, depth-first search with a stack              | $O(n)$                               | $O(n)$      |
| Search box            | Linear search + naive string matching                    | $O(n^2)$                             | $O(n)$      |
| Sort menu             | Insertion sort (stable)                                  | $O(n)$ best, $O(n^2)$ worst          | $O(n)$ copy |

## Likely panel questions

### Why selection sort for the best sellers?

For ten products any of the sorts from class is instant. Selection sort always makes  $n(n-1)/2 = 45$  comparisons — a fixed cost that is easy to show — and each pass answers the business question "which product is the biggest seller left?". Ours shifts instead of swapping, so it is stable: equal sellers keep their catalogue order.

### Why a hash table to count sales?

Without it, the count went through every order once for each product. With a row per product found by `hashGet` in  $O(1)$  on average, one pass over the orders and their lines does it.

### Why a tree for the categories instead of a list?

The categories are a hierarchy. "All lines under this branch" becomes "all leaves under this node", with no list of which line belongs where written into the page. Adding a line is adding one leaf.

### How does the tree search work without recursion?

`treeFind` and `treeLeaves` use our own stack: push the root, pop a node, push its children — depth-first, each node visited once,  $O(n)$ .

**Why not binary search for the search box?**

Binary search needs an exact key in a sorted array. The search box asks "does this word appear anywhere in the name, materials or colours?", which only a scan of each record can answer.

**Why is the text search  $O(n^2)$ ?**

textHas tries the word at every starting position of the text and compares letter by letter — a loop inside a loop. Run over a whole table, that is the  $O(n^2)$  naive string matching from class.

# Catalogue & Best Sellers

|         |                                                                                                             |
|---------|-------------------------------------------------------------------------------------------------------------|
| Screens | Home page — the "Best sellers" strip and every product card; the Overview's best sellers (order desk)       |
| Files   | js/store.js (salesTally, salesRank, topSellers) · js/algorithms.js (selectionSortDesc) · js/shop-catalog.js |
| Data    | orders — array sorted by order number; products — 10 records; a hash table product id → tally row           |
| DSA     | <b>Hash table · Selection sort</b>                                                                          |

## 1 Business process

*What task or problem from the client are you solving?*

The shop needs a front window: every bouquet and cake the customer can order, with its photo, its price label and how many have sold. Customers trust what other customers buy, so the products that sell the most are lifted into a "Best sellers" strip at the top.

The owner used to decide by hand what to call a best seller. Now the ranking is worked out from the real paid orders: buy ten roses and the rose moves up at once. Cancelled (voided) orders and requests that were never paid do not count.

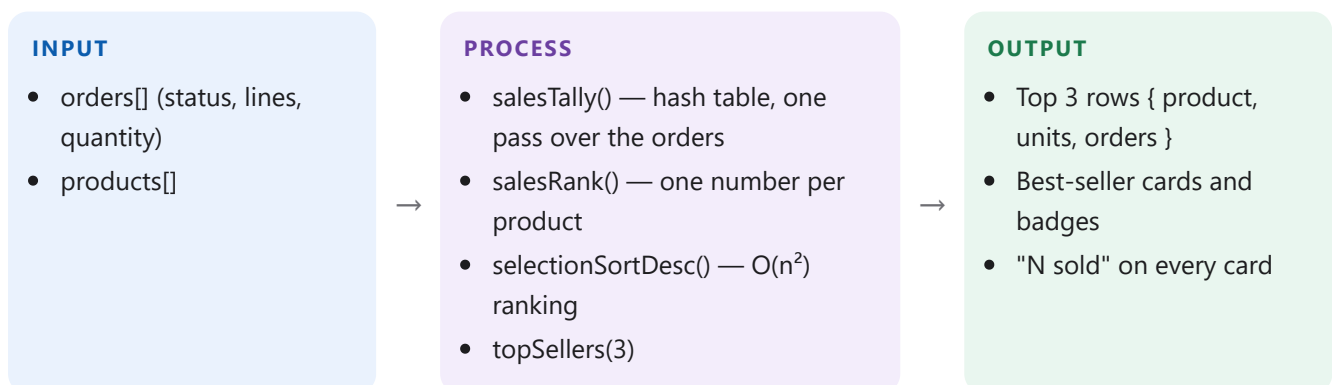
## 2 Input

*What data is needed by the module?*

| Input            | What it is                                                                                                                   |
|------------------|------------------------------------------------------------------------------------------------------------------------------|
| Stored: orders   | Every order record — its status (only paid and completed orders count) and its lines, each with a product id and a quantity. |
| Stored: products | The catalogue — name, price list, photos, and whether the product is active.                                                 |
| Customer picks   | Nothing is typed: the strip is built when the page opens and after every sale.                                               |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Hash table · Selection sort):

- 1 Tally: make one row per product and put each row in a hash table keyed by product id. Then walk every paid or completed order once; each of its lines finds its product's row with one hash look-up and adds its pieces (units). An order is counted once per product.
- 2 Give each product a rank key:  $\text{units} \times 100000 + \text{orders}$ , so pieces sold decide first and the number of orders only breaks a tie.
- 3 Selection sort, descending: for position 0, scan the rest of the list for the largest key and move it to the front; repeat for position 1, 2, ...
- 4 Keep the first three that are active and have sold at least one piece; draw them as cards with a "Best seller" badge.

## 4 Output

*What result does the module produce?*

- The "Best sellers" strip on the home page (three cards).
- A "Best seller" badge on those products in the catalogue and on the product sheet.
- The "N sold" count on every card, and the "Best selling" order of the catalogue sort.
- The Best sellers panel in the order desk Overview (top five, for the chosen day, month or year).

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                        | Time                               | Space  | Note                                                                         |
|----------------------------------|------------------------------------|--------|------------------------------------------------------------------------------|
| <code>salesTally()</code>        | $O(n^2)$ — each line of each order | $O(n)$ | Counts, for every product, how many pieces were sold and in how many orders. |
| <code>salesRank()</code>         | $O(1)$                             | $O(1)$ | Turns a row into one number so the sort can compare rows.                    |
| <code>selectionSortDesc()</code> | $O(n^2)$                           | $O(n)$ | The selection sort itself — largest key first, and stable.                   |
| <code>topSellers()</code>        | $O(n^2)$                           | $O(n)$ | Ranks a tally and keeps the first k that are active and have sold.           |
| <code>renderBestSellers()</code> | $O(n^2)$                           | $O(n)$ | Draws the strip.                                                             |

**What n means here:** For the sort,  $n$  = number of products (10). For the tally,  $n$  = the order lines it reads — every line of every paid order.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Selection sort: the list being ranked is tiny — ten products — so its  $O(n^2)$  costs at most 45 comparisons. It is the clearest sort to explain and trace: each pass picks "the biggest one left". It is written to be stable (it shifts instead of swapping), so two products with the same sales keep their catalogue order instead of trading places on every refresh.

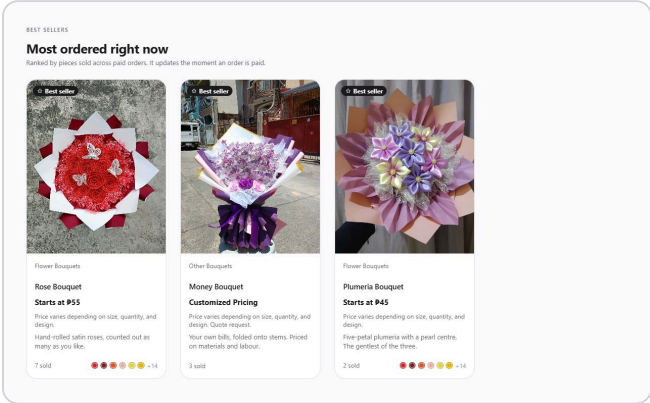
Hash table for the tally: counting product by product meant reading every order once per product. A row per product, found by its id in  $O(1)$  on average, lets one pass over the orders do the whole count.

A hand-kept "best seller" flag would go stale. Counting from the paid orders means it is always right, and a voided order drops out by itself.

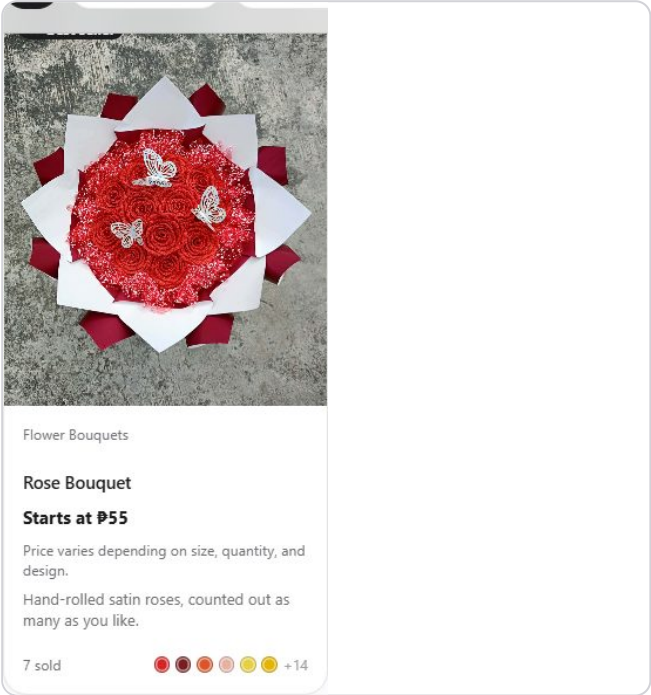
7 **Demonstration**

Show that the module works correctly — the input, the process and the output.

- 1 Open index.html. Point at the "Best sellers" strip and the "N sold" labels.
- 2 Add 10 Plumeria bouquets to the cart and check out (pay in full, any 13-digit reference such as 1234567890123).
- 3 Scroll back up: Plumeria has jumped into first place, because 10 pieces beat the rose's 7.
- 4 In the order desk, void that order: Plumeria drops back. Cancelled orders do not count.



The best-seller strip, ranked from paid orders



A product card with its "sold" count

## Trace — the best-seller ranking (computed by our code)

### Input — salesTally() over the 22 demo orders (catalogue order)

One pass over the paid and completed orders and their lines; each line finds its product's row with one hash-table look-up (hashGet by product id), so the orders are read once, not once per product.

| # | Product                 | Orders | Units | Rank key |
|---|-------------------------|--------|-------|----------|
| 0 | Rose Bouquet            | 6      | 7     | 700006   |
| 1 | Plumeria Bouquet        | 2      | 2     | 200002   |
| 2 | Dahlia Bouquet          | 1      | 1     | 100001   |
| 3 | Sunflower Bouquet       | 0      | 0     | 0        |
| 4 | Money Bouquet           | 3      | 3     | 300003   |
| 5 | Makeup Bouquet          | 1      | 1     | 100001   |
| 6 | Picture Bouquet         | 1      | 1     | 100001   |
| 7 | Sweets & Snacks Bouquet | 0      | 0     | 0        |
| 8 | Diaper Cake             | 1      | 1     | 100001   |
| 9 | Beer-in-Can Cake        | 1      | 1     | 100001   |

### Process — selectionSortDesc(tally, salesRank)

The rank function was called 90 times: two reads per comparison, so 45 comparisons for  $n = 10$  — exactly  $n(n-1)/2 = 45$ , the  $O(n^2)$  of selection sort.

| Rank | Product                 | Units | Orders |
|------|-------------------------|-------|--------|
| 1    | Rose Bouquet            | 7     | 6      |
| 2    | Money Bouquet           | 3     | 3      |
| 3    | Plumeria Bouquet        | 2     | 2      |
| 4    | Dahlia Bouquet          | 1     | 1      |
| 5    | Makeup Bouquet          | 1     | 1      |
| 6    | Picture Bouquet         | 1     | 1      |
| 7    | Diaper Cake             | 1     | 1      |
| 8    | Beer-in-Can Cake        | 1     | 1      |
| 9    | Sunflower Bouquet       | 0     | 0      |
| 10   | Sweets & Snacks Bouquet | 0     | 0      |

### Output — bestSellers(3)

1. Rose Bouquet (7 sold) 2. Money Bouquet (3 sold) 3. Plumeria Bouquet (2 sold)

## Then — placeOrder() for 10 Plumeria bouquets, paid in full

bestSellers(3) is now: 1. Plumeria Bouquet (12 sold) 2. Rose Bouquet (7 sold) 3. Money Bouquet (3 sold). Ten pieces moved Plumeria to first place at once.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

**js/store.js** · lines 572–596 · salesTally() — Counts, for every product, how many pieces were sold and in how many orders.

```
572  /* Orders and units per product, optionally only for orders a test
573     accepts (a date range). One row per product, in catalogue order; a
574     hash table keyed by product id finds a line's row in O(1) average, so
575     every order is read once instead of once per product.
576                                     Time O(n²) – each line of each order · Space O(n) */
577  function salesTally(include) {
578  ❶    var out = [], rowFor = hashCreate(31);
579      for (var p = 0; p < products.length; p++) {
580          var row = { product: products[p], orders: 0, units: 0, lastOrder: -1 };
581          listAdd(out, row);
582  ❷    hashPut(rowFor, String(products[p].id), row);
583      }
584      for (var o = 0; o < orders.length; o++) {
585  ❸    if (!countsAsSale(orders[o]) || (include && !include(orders[o]))) continue;
586          var items = orders[o].items;
587          for (var i = 0; i < items.length; i++) {
588  ❹    var hit = hashGet(rowFor, String(items[i].productId));
589          if (hit === null) continue;
590  ❺    hit.units += items[i].quantity;
591          // Count the order once, however many of its lines are this product.
592  ❻    if (hit.lastOrder !== o) { hit.orders++; hit.lastOrder = o; }
593      }
594  }
595  return out;
596  }
```

- ❶ Line 578 A list of rows, and a hash table to find a row by product id.
- ❷ Line 582 Make one row per product and index it by the product id.
- ❸ Line 585 Skip orders that are not paid or completed — voided and unpaid ones never count.
- ❹ Line 588 Each line finds its product's row with one hash look-up.
- ❺ Line 590 Add the line's pieces to that product.
- ❻ Line 592 Count the order once per product, however many lines it has.

Time O(n²) – each line of each order · Space O(n)

**js/store.js** · lines 610–615 · salesRank() — Turns a row into one number so the sort can compare rows.

```
610  /* The rank key: total units sold first, then the number of orders to
611     break a tie. Buying more of a product counts every piece, so a large
612     order moves it up the ranking at once. Time O(1) · Space O(1) */
613  function salesRank(row) {
614  ❶    return row.units * 100000 + row.orders;
615  }
```

- ❶ Line 614 Units dominate; the order count only matters when units tie.

Time O(1) · Space O(1)

js/algorithms.js · lines 62–78 · selectionSortDesc() — The selection sort itself — largest key first, and stable.

```
62  function selectionSortDesc(records, countOf) {
63  ❶    var work = copyArray(records);
64  ❷    for (var i = 0; i < work.length - 1; i++) {
65        var best = i;
66        for (var j = i + 1; j < work.length; j++) {
67            // Strictly greater, so ties keep their catalogue order.
68  ❸    if (countOf(work[j]) > countOf(work[best])) best = j;
69        }
70        if (best !== i) {
71            var held = work[best];
72            // Shift rather than swap, so the sort stays stable.
73  ❹    for (var k = best; k > i; k--) work[k] = work[k - 1];
74  ❺    work[i] = held;
75        }
76    }
77    return work;
78 }
```

- ❶ line 63 Sort a copy, so the original tally is untouched.
- ❷ line 64 Pass i fills position i.
- ❸ line 68 Scan everything after i for the largest key. Strictly greater keeps ties in their old order.
- ❹ line 73 Shift the items between i and best one place right...
- ❺ line 74 ...and drop the largest into position i. Shifting (not swapping) is what makes it stable.

Time  $O(n^2)$  · Space  $O(n)$

js/store.js · lines 624–633 · topSellers() — Ranks a tally and keeps the first k that are active and have sold.

```
624  /* Rank any tally – the whole history, or one day, month or year.
625                                     Time  $O(n^2)$  · Space  $O(n)$  */
626  function topSellers(tally, k) {
627  ❶    var ranked = selectionSortDesc(tally, salesRank);
628        var out = [];
629        for (var i = 0; i < ranked.length && out.length < k; i++) {
630  ❷    if (ranked[i].units > 0 && ranked[i].product.active) listAdd(out, ranked[i]);
631        }
632        return out;
633 }
```

- ❶ line 627 Sort the rows by the rank key.
- ❷ line 630 Leave out disabled products and products nobody has bought.

Time  $O(n^2)$  · Space  $O(n)$

js/shop-catalog.js · lines 71–83 · renderBestSellers() — Draws the strip.

```
71  /*                                     Time  $O(n^2)$  · Space  $O(n)$  */
72  function renderBestSellers() {
73  ❶    var top = bestSellers(3), ids = [];
74        for (var i = 0; i < top.length; i++) listAdd(ids, top[i].product.id);
75        var grid = $('#bestSellers');
76        if (top.length === 0) {
77            grid.innerHTML = '<p class="t-callout dim">No paid orders yet – the first ones will appear here.
78            </p>';
79            return;
80        }
81        var rows = [];
82        for (var k = 0; k < top.length; k++) listAdd(rows, { product: top[k].product, orders: top[k].orders,
83  ❷    units: top[k].units });
84        grid.innerHTML = renderEach(rows, function (row) { return productCard(row, ids); });
```

- ❶ line 73 Ask for the top three.
- ❷ line 82 Render each one as a product card with the badge.

Time  $O(n^2)$  · Space  $O(n)$

# Category Navigation

|         |                                                                                                                               |
|---------|-------------------------------------------------------------------------------------------------------------------------------|
| Screens | Catalogue — the branch chips (All, Flower Bouquets, Other Bouquets, Gift Cakes) and their line chips                          |
| Files   | js/structures.js (treeAdd, treeFromOutline, treeFind, treeLeaves, stack) · js/data.js (CATEGORY_OUTLINE) · js/shop-catalog.js |
| Data    | categoryTree — 14 nodes in one array { label, slug, parent, children, depth }; CATEGORY_OUTLINE                               |
| DSA     | <b>N-ary tree · Stack</b>                                                                                                     |

## 1 Business process

*What task or problem from the client are you solving?*

The shop's own order sheet groups everything in three columns: Flower Bouquets, Other Bouquets and Gift Cakes, each with its own lines (Rose, Plumeria, Money, Diaper Cake...). Customers browse the same way.

Picking a branch such as "Flower Bouquets" must show every line under it, and picking a line such as "Rose" must show only that line — without the page hard-coding which lines belong where.

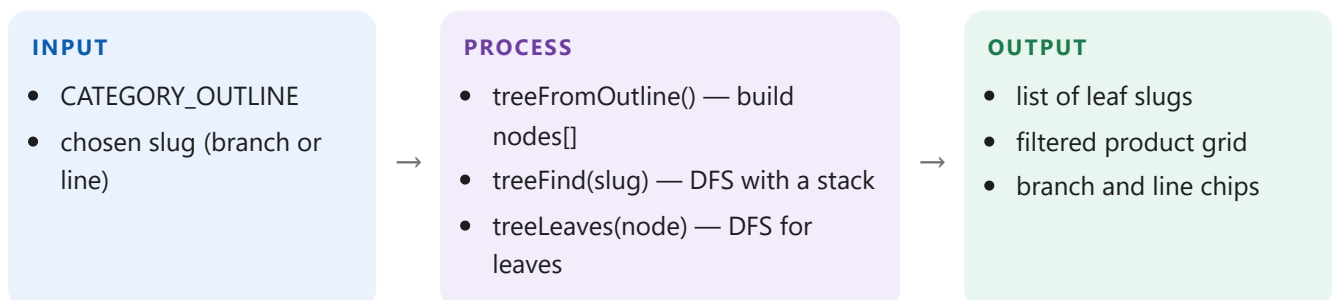
## 2 Input

*What data is needed by the module?*

| Input                    | What it is                                                                           |
|--------------------------|--------------------------------------------------------------------------------------|
| Stored: CATEGORY_OUTLINE | The three branches and their lines, from the order sheet (js/data.js).               |
| Customer picks           | A branch chip (e.g. "Gift Cakes"), then optionally a line chip (e.g. "Diaper Cake"). |
| Stored: products         | Each product's line (its leaf in the tree).                                          |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (N-ary tree · Stack):

- 1 Build the tree once: a root, three branch nodes, ten leaf nodes. Each node stores its parent's index and its children's indices in one array.
- 2 When a chip is pressed, find that node by its slug with a depth-first search that uses our stack.
- 3 Collect every leaf under it, again depth-first with a stack.
- 4 Show only the products whose line is one of those leaves.

## 4 Output

*What result does the module produce?*

- The branch chips and, after a branch is picked, its line chips.
- The catalogue filtered to the chosen branch or line, with the "N of 10 items match" count.
- The branch name shown on every card ("Flower Bouquets").

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                      | Time     | Space  | Note                                                       |
|--------------------------------|----------|--------|------------------------------------------------------------|
| <code>treeAdd()</code>         | $O(1)$   | $O(1)$ | Adds one node under a parent.                              |
| <code>treeFromOutline()</code> | $O(n)$   | $O(n)$ | Builds the whole tree from the order sheet outline.        |
| <code>treeFind()</code>        | $O(n)$   | $O(n)$ | Finds a node by slug with our stack (depth-first).         |
| <code>treeLeaves()</code>      | $O(n)$   | $O(n)$ | Collects every leaf under a node.                          |
| <code>catalogRows()</code>     | $O(n^2)$ | $O(n)$ | Applies the tree filter (then search and sort — module 3). |

**What n means here:** n = number of nodes in the tree (14: the root, 3 branches, 10 lines). Each node is visited at most once.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

The categories are a hierarchy, and a tree is the structure of a hierarchy. "All lines under Flower Bouquets" becomes "all leaves under this node" — the filter needs no list of which lines belong to which branch.

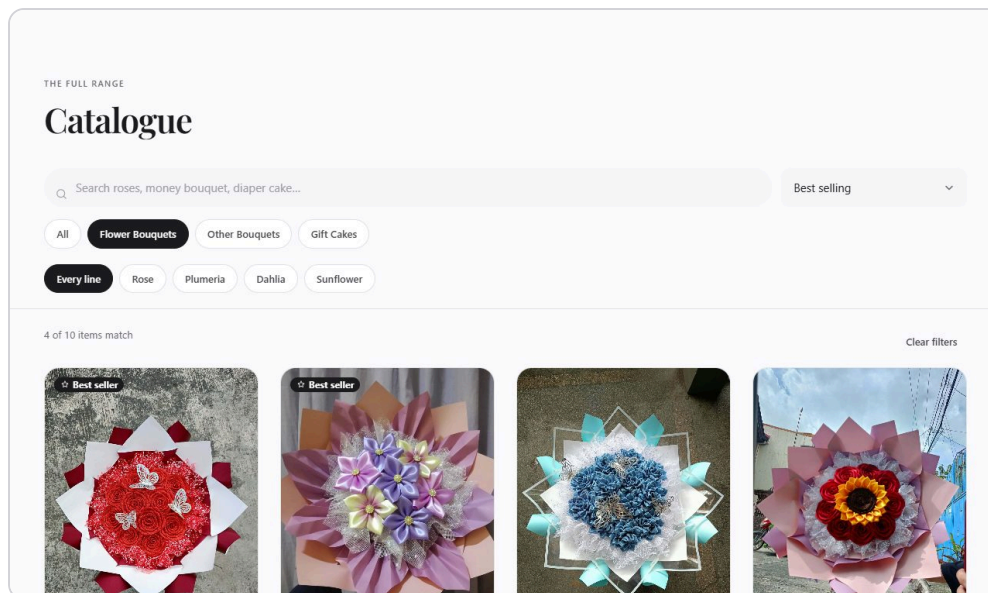
Adding a line (say, "Tulip") is one new leaf; nothing else changes. A flat list of strings would need the branch written on every product and special cases for "show everything under".

The walks use our own stack instead of recursion, which shows the depth-first order step by step. Storing children as index arrays keeps it a plain array of records, matching the procedural, array-only rule.

## 7 Demonstration

*Show that the module works correctly — the input, the process and the output.*

- 1 On the catalogue, press "Gift Cakes": two items, and the line chips "Diaper Cake" and "Beer-in-Can Cake" appear.
- 2 Press "Flower Bouquets": four items — every leaf under that branch.
- 3 Press "Rose": one item. Press "All" to clear.



Branch and line chips after choosing Flower Bouquets

## Trace — building and searching the tree (computed by our code)

### The tree as built by `treeFromOutline()`: one array of nodes

| Index | Label            | Slug            | Parent | Children      | Depth |
|-------|------------------|-----------------|--------|---------------|-------|
| 0     | All gifts        | root            | NIL    | [1, 6, 11]    | 0     |
| 1     | Flower Bouquets  | flower-bouquets | 0      | [2, 3, 4, 5]  | 1     |
| 2     | Rose             | rose            | 1      | []            | 2     |
| 3     | Plumeria         | plumeria        | 1      | []            | 2     |
| 4     | Dahlia           | dahlia          | 1      | []            | 2     |
| 5     | Sunflower        | sunflower       | 1      | []            | 2     |
| 6     | Other Bouquets   | other-bouquets  | 0      | [7, 8, 9, 10] | 1     |
| 7     | Paper Bills      | paper-bills     | 6      | []            | 2     |
| 8     | Makeup           | makeup          | 6      | []            | 2     |
| 9     | Pictures         | pictures        | 6      | []            | 2     |
| 10    | Sweets / Snacks  | sweets          | 6      | []            | 2     |
| 11    | Gift Cakes       | gift-cakes      | 0      | [12, 13]      | 1     |
| 12    | Diaper Cake      | diaper-cake     | 11     | []            | 2     |
| 13    | Beer-in-Can Cake | beer-cake       | 11     | []            | 2     |

### `treeFind(tree, "dahlia")` — the nodes popped off the stack, in order

All gifts → Flower Bouquets → Rose → Plumeria → Dahlia ⇒ index 4

## treeLeaves(tree, "Flower Bouquets")

Leaves: rose, plumeria, dahlia, sunflower. The catalogue then shows the 4 products whose line is one of these.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

**js/structures.js** · lines 389–401 · `treeAdd()` — Adds one node under a parent.

```
389  /* Attach a node under a parent.           Time O(1) · Space O(1) */
390  function treeAdd(tree, label, slug, parentIndex) {
391  ❶    var index = tree.nodes.length;
392      tree.nodes[index] = {
393          label: label,
394          slug: slug,
395          parent: parentIndex,
396          children: [],
397  ❷    depth: parentIndex === NIL ? 0 : tree.nodes[parentIndex].depth + 1
398      };
399  ❸    if (parentIndex !== NIL) listAdd(tree.nodes[parentIndex].children, index);
400      return index;
401  }
```

- ❶ line 391 The new node's index is the next free slot.
- ❷ line 397 Depth is the parent's depth + 1 (the root is 0).
- ❸ line 399 Link it: the parent remembers its new child's index.

Time O(1) · Space O(1)

**js/structures.js** · lines 456–466 · `treeFromOutline()` — Builds the whole tree from the order sheet outline.

```
456  /* Build from [{ label, slug, children: [{ label, slug }] }].
457                        Time O(n) · Space O(n) */
458  function treeFromOutline(rootLabel, branches) {
459  ❶    var tree = treeCreate(rootLabel);
460      for (var i = 0; i < branches.length; i++) {
461  ❷    var parent = treeAdd(tree, branches[i].label, branches[i].slug, tree.root);
462          var kids = branches[i].children || [];
463  ❸    for (var j = 0; j < kids.length; j++) treeAdd(tree, kids[j].label, kids[j].slug, parent);
464      }
465      return tree;
466  }
```

- ❶ line 459 Start with a root.
- ❷ line 461 Add each branch under the root...
- ❸ line 463 ...and each line under its branch.

Time O(n) · Space O(n)

**js/structures.js** · lines 403–414 · `treeFind()` — Finds a node by slug with our stack (depth-first).

```
403  /* Depth-first search for a slug, with an explicit stack.
404                        Time O(n) · Space O(n) */
405  function treeFind(tree, slug) {
406      var stack = stackCreate();
407  ❶    stackPush(stack, tree.root);
408      while (stack.top !== NIL) {
```

```

409 ②    var at = stackPop(stack), node = tree.nodes[at];
410 ③    if (node.slug === slug) return at;
411 ④    for (var i = node.children.length - 1; i >= 0; i--) stackPush(stack, node.children[i]);
412    }
413    return NIL;
414    }

```

- ① Line 407 Start at the root.
- ② Line 409 Take the most recently added node (last in, first out).
- ③ Line 410 Found it — return its index.
- ④ Line 411 Otherwise push its children, last first, so the first child is visited next.

Time  $O(n)$  · Space  $O(n)$

**js/structures.js** · lines 431–444 · `treeLeaves()` — Collects every leaf under a node.

```

431  /* Every leaf slug at or below a node - what the catalogue filters on.
432                                     Time  $O(n)$  · Space  $O(n)$  */
433  function treeLeaves(tree, index) {
434      var out = [];
435      if (index === NIL) return out;
436      var stack = stackCreate();
437      stackPush(stack, index);
438      while (stack.top !== NIL) {
439          var at = stackPop(stack), node = tree.nodes[at];
440  ①    if (node.children.length === 0) listAdd(out, node.slug);
441  ②    else for (var i = node.children.length - 1; i >= 0; i--) stackPush(stack, node.children[i]);
442      }
443      return out;
444  }

```

- ① Line 440 A node with no children is a leaf: keep its slug.
- ② Line 441 Otherwise descend into its children.

Time  $O(n)$  · Space  $O(n)$

**js/shop-catalog.js** · lines 129–143 · `catalogRows()` — Applies the tree filter (then search and sort — module 3).

```

129  /* The rows to show, in order.           Time  $O(n^2)$  · Space  $O(n)$  */
130  function catalogRows() {
131      var slug = catalogState.line || catalogState.branch;
132  ①    var leaves = slug ? treeLeaves(categoryTree, treeFind(categoryTree, slug)) : null;
133  ②    var inBranch = keepWhere(activeProducts(), function (p) { return leaves === null || isIn(leaves,
134      p.line); });
135
136      var found = linearSearch(inBranch, catalogState.query, searchText);
137
138      var tally = salesTally(), rows = [];
139      for (var i = 0; i < found.length; i++) {
140          var t = firstWhere(tally, function (r) { return r.product.id === found[i].id; });
141          listAdd(rows, { product: found[i], orders: t ? t.orders : 0, units: t ? t.units : 0 });
142      }
143      var sort = firstWhere(SORTS, function (s) { return s.id === catalogState.sort; }) || SORTS[0];
144      return insertionSort(rows, sort.compare);
145  }

```

- ① Line 132 The chosen chip's leaves, or no filter.
- ② Line 133 Keep active products whose line is one of those leaves.

Time  $O(n^2)$  · Space  $O(n)$

# Search & Sort

|         |                                                                                                                               |
|---------|-------------------------------------------------------------------------------------------------------------------------------|
| Screens | Catalogue — the search box and the sort menu (Best selling, Name A–Z, Starting price, Newest)                                 |
| Files   | js/core.js (normaliseKey, textHas) · js/algorithms.js (linearSearch, insertionSort) · js/shop-catalog.js (catalogRows, SORTS) |
| Data    | products — 10 records; SORTS — the four comparisons; each product's searchable text                                           |
| DSA     | <b>Linear search · Insertion sort</b>                                                                                         |

## 1 Business process

*What task or problem from the client are you solving?*

A customer who knows what they want types it: "matcha", "money", "diaper". The search has to find it in names, materials, branch names and even colour names, however it is typed (" MATCHA ", "Matcha rose").

They also want to order the results: best selling, by name, cheapest first, or newest — and the order must be steady, not shuffling equal items around.

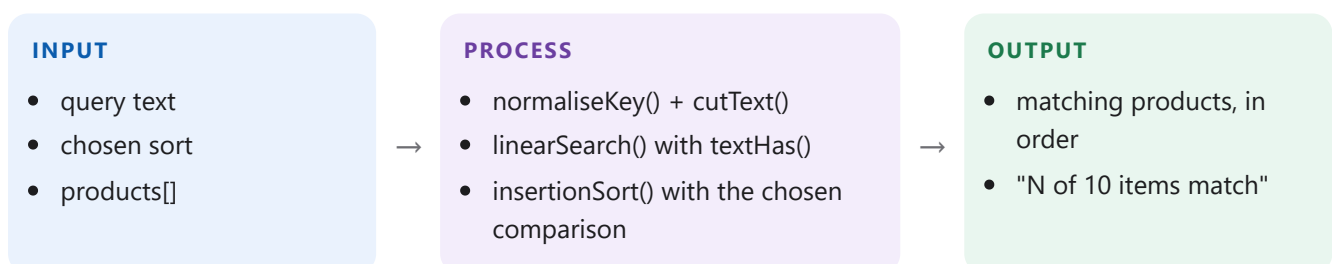
## 2 Input

*What data is needed by the module?*

| Input            | What it is                                                                             |
|------------------|----------------------------------------------------------------------------------------|
| Customer types   | Search words in the search box.                                                        |
| Customer picks   | A sort: Best selling, Name A–Z, Starting price, Newest.                                |
| Stored: products | Name, line, branch, short description, materials, colours, starting price, date added. |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Linear search · Insertion sort):

- 1 Normalise the query: lower-case, trim, collapse spaces, then cut it into words.
- 2 Linear search: for each product, build its searchable text and keep it only if every word appears in it — naive string matching, trying the word at every position of the text.
- 3 Insertion sort the matches with the chosen comparison: take each product in turn and slide the products before it that sort later one place right, until its place is found. Equal products never pass each other, so the order is stable.

## 4 Output

*What result does the module produce?*

- The catalogue grid, filtered and ordered.
- The result count, and a "Clear filters" button when filtered.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                    | Time                                             | Space       | Note                                                   |
|------------------------------|--------------------------------------------------|-------------|--------------------------------------------------------|
| <code>normaliseKey()</code>  | $O(n)$                                           | $O(n)$      | Puts text into one comparable form.                    |
| <code>textHas()</code>       | $O(n^2)$ – every start position × the needle     | $O(1)$      | Does the text contain the word? Naive string matching. |
| <code>linearSearch()</code>  | $O(n^2)$ – naive string matching on every record | $O(n)$      | Keeps every record whose text has every word.          |
| <code>insertionSort()</code> | $O(n^2)$ worst, $O(n)$ best                      | $O(n)$ copy | The sort used everywhere in the system.                |
| <code>catalogRows()</code>   | $O(n^2)$                                         | $O(n)$      | Search, then sort.                                     |

**What n means here:** n = number of products searched or sorted. Inside `textHas`, n = the length of the text being scanned.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Linear search is the right search here: the match is "contains these words anywhere in several fields". Binary search can only find an exact key in a sorted array, so it cannot answer a "contains" question; at n = 10 a full scan is instant.

Naive string matching tries the word at every starting position and compares letter by letter —  $O(n^2)$ , the cost the course gives for searching text across a table. It needs no extra index and handles any word.

Insertion sort is stable and simple:  $O(n)$  when the list is already in order,  $O(n^2)$  at worst — 45 comparisons for 10 products. It is one of the sorts from class; merge sort would be  $O(n \log n)$ , a notation the course does not use, and quick sort is not stable.

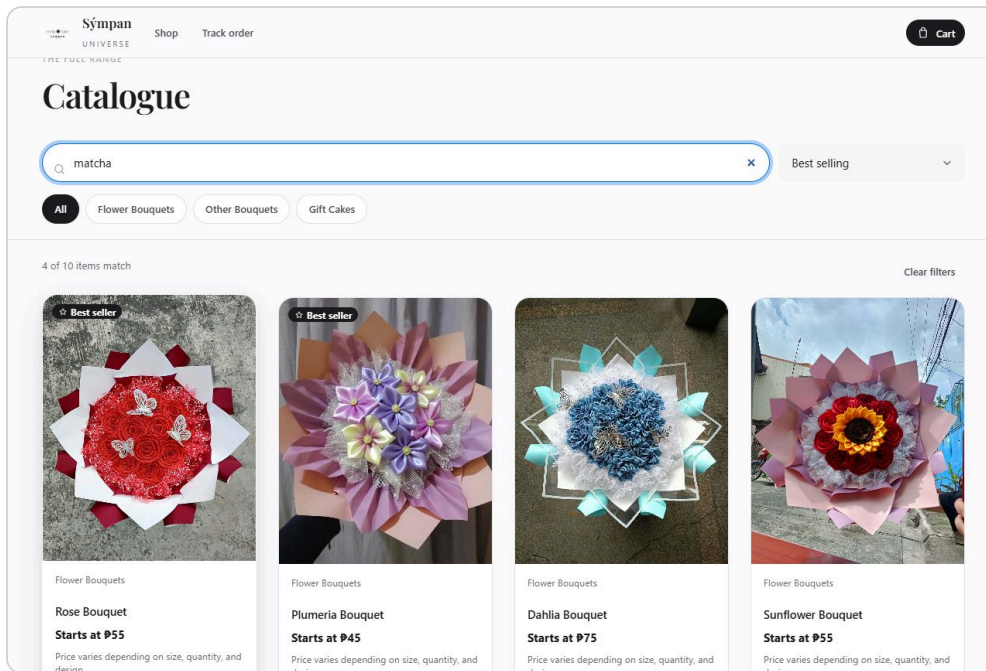
The same `insertionSort` serves every list in the system that needs ordering: completed orders, voids, the outbox, past quotations.

## 7 Demonstration

*Show that the module works correctly — the input, the process and the output.*

- 1 Type "matcha" in the search box: the four flower bouquets appear, because Matcha is one of their ribbon colours.
- 2 Type "money": only the Money Bouquet.

- 3 Change the sort to "Name, A-Z": Beer-in-Can Cake comes first.



Searching "matcha" finds the flowers by colour name

## Trace — the search and the insertion sort (computed by our code)

**linearSearch(products, " Matcha ROSE ") — normalised to the words "matcha", "rose"**

14 textHas() checks were made across 10 products (a product stops at its first missing word).

| Product text begins       | Word   | Found? |
|---------------------------|--------|--------|
| rose bouquet rose flow... | matcha | yes    |
| rose bouquet rose flow... | rose   | yes    |
| plumeria bouquet plume... | matcha | yes    |
| plumeria bouquet plume... | rose   | no     |
| dahlia bouquet dahlia ... | matcha | yes    |
| dahlia bouquet dahlia ... | rose   | no     |
| sunflower bouquet sunf... | matcha | yes    |
| sunflower bouquet sunf... | rose   | yes    |
| money bouquet paper bi... | matcha | no     |
| makeup bouquet makeup ... | matcha | no     |
| picture bouquet pictur... | matcha | no     |
| sweets & snacks bouque... | matcha | no     |
| diaper cake diaper cak... | matcha | no     |
| beer-in-can cake beer-... | matcha | no     |

Result: Rose Bouquet, Sunflower Bouquet

**insertionSort(10 products, by name) — each pass takes the next product and slides it left past the larger names**

| Pass | Product taken           | Comparisons | Result        |
|------|-------------------------|-------------|---------------|
| 1    | Plumeria Bouquet        | 1           | slides 1 left |
| 2    | Dahlia Bouquet          | 2           | slides 2 left |
| 3    | Sunflower Bouquet       | 1           | stays         |
| 4    | Money Bouquet           | 4           | slides 3 left |
| 5    | Makeup Bouquet          | 5           | slides 4 left |
| 6    | Picture Bouquet         | 4           | slides 3 left |
| 7    | Sweets & Snacks Bouquet | 1           | stays         |
| 8    | Diaper Cake             | 8           | slides 7 left |
| 9    | Beer-in-Can Cake        | 9           | slides 9 left |

35 comparisons in all. Worst case for 10 items is  $n(n-1)/2 = 45$  (a list in reverse order); best case is  $n - 1 = 9$  (already in order).

Result: Beer-in-Can Cake, Dahlia Bouquet, Diaper Cake, Makeup Bouquet, Money Bouquet, Picture Bouquet, Plumeria Bouquet, Rose Bouquet, Sunflower Bouquet, Sweets & Snacks Bouquet

**Best and worst case, counted on the real function**

| Input (10 numbers)               | Comparisons | Case             |
|----------------------------------|-------------|------------------|
| 1, 2, 3, ... 10 (already sorted) | 9           | best — $O(n)$    |
| 10, 9, 8, ... 1 (reversed)       | 45          | worst — $O(n^2)$ |

Orders, emails and receipts are appended in time order, so the "newest first" and "oldest first" lists the shop sorts are close to one of these two cases.

# Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

**js/core.js · lines 262–277 · normaliseKey()** — Puts text into one comparable form.

```
262  /* Lower-case, trimmed, inner runs of space collapsed - the form keys are
263     compared in, so " Quezon City " finds "quezon city". Time O(n) · Space O(n) */
264  function normaliseKey(text) {
265  ❶    var s = strip(toLower(text)), out = '', lastSpace = false;
266      for (var i = 0; i < s.length; i++) {
267          var c = s.charAt(i);
268  ❷    if (isSpace(c) || c === '.' || c === ',') {
269  ❸        if (!lastSpace) out += ' ';
270            lastSpace = true;
271        } else {
272            out += c;
273            lastSpace = false;
274        }
275    }
276    return strip(out);
277 }
```

- ❶ line 265 Lower-case and trim.
- ❷ line 268 Spaces, dots and commas all become one space...
- ❸ line 269 ...and runs of them collapse to a single space.

Time  $O(n)$  · Space  $O(n)$

**js/core.js** · lines 191–201 · `textHas()` — Does the text contain the word? Naive string matching.

```

191  /* Does the text contain the needle? Naive scan. Time  $O(n^2)$  - every start position × the needle · Space
    0(1) */
192  function textHas(haystack, needle) {
193      var text = String(haystack), find = String(needle);
194      if (find.length === 0) return true;
195  ❶   for (var start = 0; start + find.length <= text.length; start++) {
196          var k = 0;
197  ❷   while (k < find.length && text.charAt(start + k) === find.charAt(k)) k++;
198  ❸   if (k === find.length) return true;
199      }
200      return false;
201  }
```

- ❶ line 195 Try every starting position...
- ❷ line 197 ...and match the characters one by one (a loop inside a loop).
- ❸ line 198 All matched: found.

Time  $O(n^2)$  - every start position × the needle · Space  $O(1)$

**js/algorithms.js** · lines 100–118 · `linearSearch()` — Keeps every record whose text has every word.

```

100  /* Linear search: every record whose text contains every word typed.
    101      textOf builds the searchable text for one record.
    102          Time  $O(n^2)$  - naive string matching on every record · Space  $O(n)$ 
    */
103  function linearSearch(records, query, textOf) {
104  ❶   var words = cutText(normaliseKey(query), ' ');
105      var wanted = keepWhere(words, function (w) { return w.length > 0; });
106  ❷   if (wanted.length === 0) return copyArray(records);
107
108      var out = [];
109      for (var i = 0; i < records.length; i++) {
110  ❸   var haystack = normaliseKey(textOf(records[i]));
111          var all = true;
112          for (var w = 0; w < wanted.length && all; w++) {
113  ❹   if (!textHas(haystack, wanted[w])) all = false;
114          }
115  ❺   if (all) listAdd(out, records[i]);
116      }
117      return out;
118  }
```

- ❶ line 104 Normalise the query and cut it into words.
- ❷ line 106 No words: everything matches.
- ❸ line 110 Build this record's searchable text.
- ❹ line 113 A missing word rules the record out.
- ❺ line 115 Keep it.

Time  $O(n^2)$  - naive string matching on every record · Space  $O(n)$

js/algorithms.js · lines 36–52 · insertionSort() — The sort used everywhere in the system.

```
36  /* Returns a new sorted array; the input is left untouched. Each item is
37     taken in turn and the larger items before it slide one slot right until
38     its place is found. "> 0" (not ">= 0") keeps equal items in their
39     original order, so the sort is stable.
40                                     Time O(n²) worst, O(n) best · Space O(n) copy */
41  function insertionSort(list, compare) {
42  ❶    var work = copyArray(list);
43  ❷    for (var i = 1; i < work.length; i++) {
44        var held = work[i], j = i - 1;
45  ❸    while (j >= 0 && compare(work[j], held) > 0) {
46  ❹        work[j + 1] = work[j];
47        j--;
48    }
49  ❺    work[j + 1] = held;
50    }
51    return work;
52  }
```

❶ line 42 Sort a copy; the input is left untouched.

❷ line 43 Take each item in turn, from the second on.

❸ line 45 While the item before it sorts later — "> 0", not ">= 0", so equal items never pass each other (stable)...

❹ line 46 ...slide that item one place right.

❺ line 49 Drop the held item into the gap.

Time O(n²) worst, O(n) best · Space O(n) copy

js/shop-catalog.js · lines 129–143 · catalogRows() — Search, then sort.

```
129  /* The rows to show, in order.          Time O(n²) · Space O(n) */
130  function catalogRows() {
131      var slug = catalogState.line || catalogState.branch;
132      var leaves = slug ? treeLeaves(categoryTree, treeFind(categoryTree, slug)) : null;
133      var inBranch = keepWhere(activeProducts(), function (p) { return leaves === null || isIn(leaves,
134  ❶    p.line); });
135      var found = linearSearch(inBranch, catalogState.query, searchText);
136      var tally = salesTally(), rows = [];
137      for (var i = 0; i < found.length; i++) {
138          var t = firstWhere(tally, function (r) { return r.product.id === found[i].id; });
139          listAdd(rows, { product: found[i], orders: t ? t.orders : 0, units: t ? t.units : 0 });
140      }
141      var sort = firstWhere(SORTS, function (s) { return s.id === catalogState.sort; }) || SORTS[0];
142  ❷    return insertionSort(rows, sort.compare);
143  }
```

❶ line 134 Linear search over the products in the chosen branch.

❷ line 142 Order them with the chosen comparison.

Time O(n²) · Space O(n)

# Choosing a gift

Module 4 Flower Customiser · Module 5 Gift Customiser · Module 6 Cart

## 60-second pitch

These modules turn a product into something in the cart. The Flower Customiser finds the right photograph among 160 combinations with a hash table keyed "flower|arrangement|colour", and adds up the add-ons with a recursive sum. The Gift Customiser suggests the fewest bills for a money bouquet with a greedy algorithm. The Cart is a singly linked list: adding is  $O(1)$  at the tail, and removing re-points one link.

## Structures and algorithms to be able to explain

| Where               | Structure / algorithm                           | Time           | Space             |
|---------------------|-------------------------------------------------|----------------|-------------------|
| Find the photograph | Hash table with chaining (FNV-1a)               | $O(1)$ average | $O(n)$            |
| Add-on total        | Recursion                                       | $O(n)$         | $O(n)$ call stack |
| Fewest bills        | Greedy change-making                            | $O(n)$         | $O(n)$            |
| Add to cart         | Linked list append (tail pointer)               | $O(1)$         | $O(1)$            |
| Remove a line       | Linked list remove (walk to the node before it) | $O(n)$         | $O(1)$            |
| Show the cart       | Linked list walk from the head                  | $O(n)$         | $O(n)$            |

## Likely panel questions

### What happens when two keys land in the same bucket?

That is a collision. Each bucket holds a chain (a short list); the second key is added to the chain and a look-up compares keys along it. With 160 keys in 199 buckets the chains are about one long.

### Why 199 buckets?

A prime a little above the 160 keys: the load factor stays near 0.8 and the remainder spreads the FNV-1a hashes evenly over the buckets.

### Is the greedy answer always the fewest bills?

For the peso, yes. Its notes (1000, 500, 200, 100, 50, 20) form a canonical system, where taking the largest note that fits is optimal. For odd currencies greedy can fail and dynamic programming would be needed.

### Why a linked list for the cart instead of an array?

The cart adds at the end and removes from anywhere. Removing a node re-points one link; an array would shift every later item. The tail pointer makes adding  $O(1)$ .

### Where is the recursion, and what stops it?

sumRecursive adds the first item to the sum of the rest. The base case is the empty rest, which adds 0. The add-on list is at most four long, so the call stack stays tiny.

# Flower Customiser

|         |                                                                                                                                                                     |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | The product sheet of a flower bouquet (Rose, Plumeria, Dahlia, Sunflower)                                                                                           |
| Files   | js/core.js (fnv1a) · js/structures.js (hashPut, hashGet) · js/store.js (buildReferenceIndex, makeFlowerItem) · js/algorithms.js (sumRecursive) · js/shop-product.js |
| Data    | referenceIndex — hash table, 199 buckets, 160 keys; COLORS (20), ARRANGEMENTS, REAL_REFERENCES (14); the product's price list                                       |
| DSA     | <b>Hash table · Recursion</b>                                                                                                                                       |

## 1 Business process

*What task or problem from the client are you solving?*

A flower bouquet comes in two arrangements (Round, Layered), up to nine flower counts and twenty ribbon colours from the shop's colour chart. Customers need to see what they are ordering: the right photo for exactly that arrangement and colour, and the exact price.

There are 160 combinations of flower × arrangement × colour. 14 have real photos of the shop's work; the other 146 have previews made from the shop's own photos, recoloured to the chart colour. Add-ons (fairy lights, topper, card, glitter) add to the price.

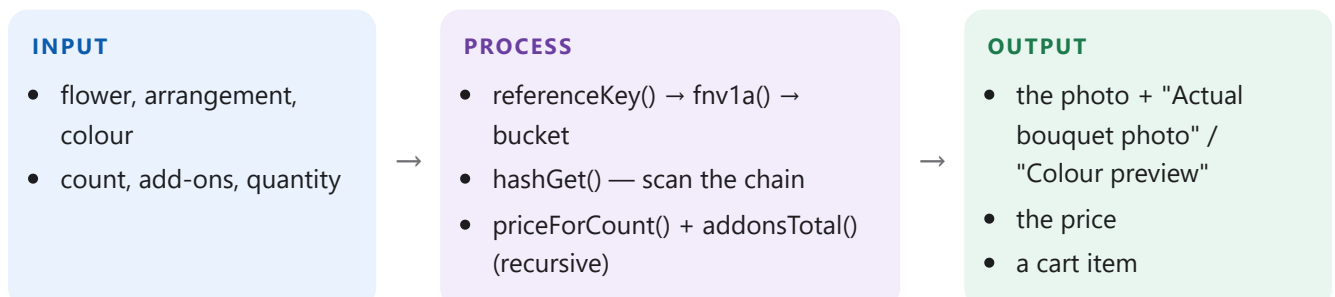
## 2 Input

*What data is needed by the module?*

| Input                                         | What it is                                                                                                        |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Customer picks                                | Arrangement, flower count, one of 20 colours, add-ons, card flower, quantity, notes, an optional reference image. |
| Stored: REAL_REFERENCES, COLORS, ARRANGEMENTS | Which photos are real; the colour chart; the counts each arrangement offers.                                      |
| Stored: the product's price list              | Price per flower count, set by the owner.                                                                         |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Hash table · Recursion):

- At start-up, build a hash table of all 160 photos keyed "flower|arrangement|colour": first every generated preview, then the 14 real photos written over their entries.

- 2 When the customer picks, build the key, hash it with FNV-1a, go to that bucket and scan its short chain for the key: the photo, and whether it is real.
- 3 Price = the price-list price for that flower count + the add-ons, added up recursively (each call adds one add-on to the sum of the rest).
- 4 Check the choice and build the cart item.

## 4 Output

*What result does the module produce?*

- The large photo and its caption on the product sheet.
- The price in the footer ("P555.00").
- A checked cart item.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                          | Time                                                                                            | Space             | Note                                        |
|------------------------------------|-------------------------------------------------------------------------------------------------|-------------------|---------------------------------------------|
| <code>fnv1a()</code>               | $O(n)$                                                                                          | $O(1)$            | Turns text into a 32-bit number (the hash). |
| <code>hashPut()</code>             | $O(1)$ average                                                                                  | $O(1)$            | Inserts or overwrites a key.                |
| <code>hashGet()</code>             | $O(1)$ average                                                                                  | $O(1)$            | Looks a key up.                             |
| <code>buildReferenceIndex()</code> | $O(n)$ – one <code>hashPut</code> per flower × arrangement × colour ( $n = 160$ ) and per photo | $O(n)$            | Fills the table with all 160 photos.        |
| <code>sumRecursive()</code>        | $O(n)$                                                                                          | $O(n)$ call stack | Adds up a list recursively.                 |
| <code>makeFlowerItem()</code>      | $O(n^2)$                                                                                        | $O(n)$            | Checks the choice and prices it.            |

**What n means here:** For the table's space,  $n$  = keys stored (160). Hashing a key is  $O(n)$  in its length. For the recursive sum,  $n$  = the add-ons chosen (at most four).

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Every click must find one photo among 160. A hash table answers "what is stored under this key?" in  $O(1)$  on average, and with 199 buckets for 160 keys the chains stay about one long. A linear scan of 160 records would grow with every colour or flower added; binary search would need the keys kept sorted.

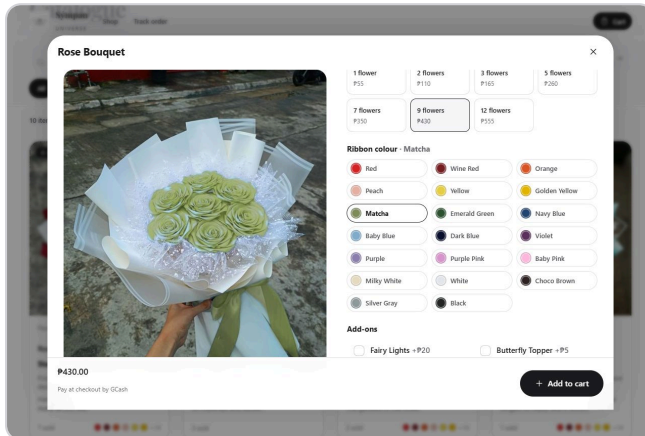
Chaining handles collisions simply: two keys landing in the same bucket just sit in the same short list.

Recursion fits a sum naturally: the total of a list is its first item plus the total of the rest, ending at the empty list. The add-on list is at most four long, so the call stack is tiny.

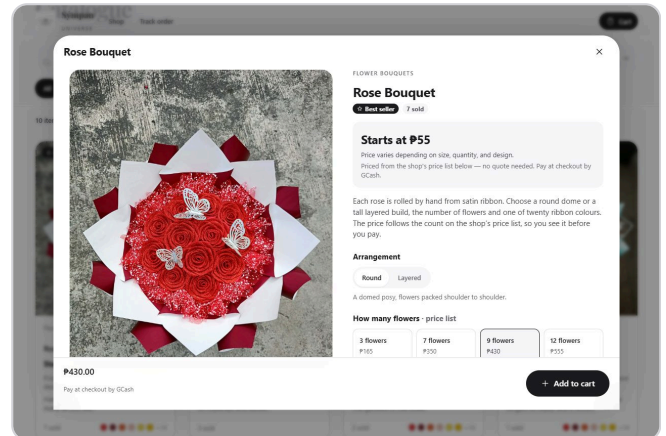
## 7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Open the Rose Bouquet. It starts Round, Red — caption "Actual bouquet photo".
- 2 Pick Emerald Green: the photo changes and the caption says "Colour preview".
- 3 Switch to Layered: a different photo — round and layered never share one.
- 4 Tick Fairy Lights and the card: the price in the footer goes up by ₱40.



Rose, Layered, Matcha — a colour preview



Rose, Round, Red — a real photograph

### Trace — the photo look-up and the add-on sum (computed by our code)

`referenceImage("rose", "layered", "matcha")`

| Step                                      | Value                                                               |
|-------------------------------------------|---------------------------------------------------------------------|
| 1. <code>referenceKey()</code>            | <code>rose layered matcha</code>                                    |
| 2. <code>fnv1a(key)</code>                | 281994334                                                           |
| 3. <code>hashIndex = fnv1a mod 199</code> | 190                                                                 |
| 4. chain length in bucket 190             | 2                                                                   |
| 5. compared keys until found              | 1                                                                   |
| Result                                    | <code>assets/colors/rose-layered-matcha.jpg</code> (colour preview) |

[190] `rose|layered|matcha` → `plumeria|round|emerald-green`

#### The whole table after `buildReferenceIndex()`

160 keys in 199 buckets: load factor 0.8, longest chain 5. Real photographs overwrote 14 previews (e.g. `rose|round|red` → `assets/products/roses-round-1.jpg`).

| Chain length | 0  | 1  | 2  | 3 | 4+ |
|--------------|----|----|----|---|----|
| Buckets      | 93 | 67 | 30 | 4 | 5  |

### addonsTotal(["lights", "card", "glitter"], 12 flowers) — the recursive calls, innermost first

| Call                 | This item           | Returns |
|----------------------|---------------------|---------|
| sumRecursive(..., 3) | (empty — base case) | ₹0      |
| sumRecursive(..., 2) | glitter = ₹15       | ₹15     |
| sumRecursive(..., 1) | card = ₹20          | ₹35     |
| sumRecursive(..., 0) | lights = ₹20        | ₹55     |

Total add-ons: ₹55 (glitter for 12 flowers is ₹15).

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/core.js · lines 517–527 · fnv1a() — Turns text into a 32-bit number (the hash).

```
517  /* 32-bit FNV-1a. Used for the hash table's bucket index and to compare the
518     order-desk passcode without keeping it in the page as plain text.
519                                     Time O(n) · Space O(1) */
520  function fnv1a(text) {
521    ❶ var s = String(text), hash = 2166136261;
522    for (var i = 0; i < s.length; i++) {
523      ❷ hash = hash ^ s.charCodeAt(i);
524      ❸ hash = Math.imul(hash, 16777619) >>> 0;
525    }
526    return hash >>> 0;
527  }
```

- ❶ line 521 Start from the FNV offset basis.
- ❷ line 523 Mix in each character with XOR...
- ❸ line 524 ...then multiply by the FNV prime, kept to 32 bits.

Time O(n) · Space O(1)

js/structures.js · lines 489–497 · hashPut() — Inserts or overwrites a key.

```
489  /* Insert or overwrite.                               Time O(1) average · Space O(1) */
490  function hashPut(table, key, value) {
491    ❶ var chain = table.buckets[hashIndex(table, key)];
492    for (var i = 0; i < chain.length; i++) {
493      ❷ if (chain[i].key === key) { chain[i].value = value; return; }
494    }
495    ❸ chain[chain.length] = { key: key, value: value };
496    table.size++;
497  }
```

- ❶ line 491 Hash the key to its bucket.
- ❷ line 493 Key already there: overwrite it (how real photos replace previews).
- ❸ line 495 Otherwise add it to the end of the chain.

Time O(1) average · Space O(1)

js/structures.js · lines 499–506 · hashGet() — Looks a key up.

```
499  /* The value for a key, or null.                       Time O(1) average · Space O(1) */
500  function hashGet(table, key) {
```

```

501 ❶ var chain = table.buckets[hashIndex(table, key)];
502   for (var i = 0; i < chain.length; i++) {
503 ❷   if (chain[i].key === key) return chain[i].value;
504   }
505   return null;
506 }

```

- ❶ Line 501 Go straight to the bucket.
- ❷ Line 503 Scan the short chain for the exact key.

Time  $O(1)$  average · Space  $O(1)$

js/store.js · lines 205–228 · buildReferenceIndex() — Fills the table with all 160 photos.

```

205  /* Index every combination: generated previews first, then the real
206   photographs written over the entries they exist for.
207                                     Time  $O(n)$  – one hashPut per flower × arrangement × colour (n =
160) and per photo · Space  $O(n)$  */
208  function buildReferenceIndex() {
209    // 160 keys over 199 buckets: a load factor near 0.8 keeps chains short.
210 ❶ var table = hashCreate(199);
211    for (var f = 0; f < FLOWERS.length; f++) {
212      for (var a = 0; a < ARRANGEMENTS.length; a++) {
213        for (var c = 0; c < COLORS.length; c++) {
214          var flower = FLOWERS[f], arrangement = ARRANGEMENTS[a].id, color = COLORS[c].id;
215          hashPut(table, referenceKey(flower, arrangement, color), {
216 ❷          src: IMG_COLORS + flower + '-' + arrangement + '-' + color + '.jpg',
217            real: false
218          });
219        }
220      }
221    }
222    for (var r = 0; r < REAL_REFERENCES.length; r++) {
223      var ref = REAL_REFERENCES[r];
224      hashPut(table, referenceKey(ref.flower, ref.arrangement, ref.color),
225 ❸      { src: IMG_PRODUCTS + ref.file, real: true });
226    }
227    return table;
228  }

```

- ❶ Line 210 199 buckets: a prime a little above 160 keeps chains short.
- ❷ Line 216 Every combination first gets its generated preview...
- ❸ Line 225 ...then the 14 real photos overwrite their entries.

Time  $O(n)$  – one hashPut per flower × arrangement × colour (n = 160) and per photo · Space  $O(n)$

js/algorithms.js · lines 124–131 · sumRecursive() — Adds up a list recursively.

```

124  /* Sum a list by peeling off one element at a time. The base case is the
125   empty tail. Used for the add-on total and the quote's line totals.
126                                     Time  $O(n)$  · Space  $O(n)$  call stack */
127  function sumRecursive(items, valueOf, index) {
128    var at = index === undefined ? 0 : index;
129 ❶ if (at >= items.length) return 0;
130 ❷ return (Number(valueOf(items[at])) || 0) + sumRecursive(items, valueOf, at + 1);
131  }

```

- ❶ Line 129 Base case: the empty rest adds nothing.
- ❷ Line 130 This item plus the sum of the rest.

Time  $O(n)$  · Space  $O(n)$  call stack

```

270  /* Validate a flower choice and build its item.  Time O(n²) · Space O(n)
271     choice: { arrangement, count, color, quantity, addons, cardFlower, notes, reference } */
272  function makeFlowerItem(product, choice) {
273      if (!product || !product.active || product.kind !== 'flower') return { ok: false, error: 'This bouquet
is not available.' };
274      if (!isIn(product.arrangements, choice.arrangement)) return { ok: false, error: 'Choose an
arrangement.' };
275      ❶ if (!isIn(countsFor(product, choice.arrangement), choice.count)) return { ok: false, error: 'Choose how
many flowers.' };
276      if (!isIn(product.colors, choice.color)) return { ok: false, error: 'Choose a ribbon colour.' };
277      if (!isWholeIn(choice.quantity, 1, MAX_QUANTITY)) return { ok: false, error: 'Quantity must be 1 to ' +
MAX_QUANTITY + '.' };
278      if (String(choice.notes || '').length > MAX_NOTE) return { ok: false, error: 'Notes are limited to ' +
MAX_NOTE + ' characters.' };
279
280      var addons = keepWhere(choice.addons || [], function (id) { return addonById(id) !== null; });
281      ❷ var unit = priceForCount(product, choice.count) + addonsTotal(addons, choice.count);
282      return { ok: true, item: {
283          productId: product.id, productName: product.name, kind: 'flower', quoteType: 'flower',
284          arrangement: choice.arrangement, count: choice.count, color: choice.color,
285          size: '', detail: '', provided: false,
286          quantity: choice.quantity, addons: addons,
287          cardFlower: isIn(addons, 'card') ? (choice.cardFlower || CARD_FLOWERS[0]) : '',
288          notes: strip(choice.notes), reference: choice.reference || null,
289      ❸ unitEstimate: unit, estimate: unit * choice.quantity,
290          materials: 0, labor: 0, itemExpense: 0
291      } };
292  }

```

- ❶ Line 275 The count must be one this arrangement offers.
- ❷ Line 281 Price list + add-ons for one bouquet.
- ❸ Line 289 Times the quantity.

Time  $O(n^2)$  · Space  $O(n)$

# Gift Customiser

|         |                                                                                                                                   |
|---------|-----------------------------------------------------------------------------------------------------------------------------------|
| Screens | The product sheet of the five quote gifts (money, makeup, sweets & snacks, diaper cake, beer-in-can cake) and the picture bouquet |
| Files   | js/algorithms.js (breakIntoBills, describeBills) · js/store.js (makeQuoteItem) · js/shop-product.js (billHint)                    |
| Data    | DENOMINATIONS — 1000, 500, 200, 100, 50, 20; QUOTE_TYPES; the product's sizes                                                     |
| DSA     | <b>Greedy algorithm</b>                                                                                                           |

## 1 Business process

*What task or problem from the client are you solving?*

Five gifts have no price list and are quoted by the owner: the money bouquet, makeup bouquet, sweets & snacks bouquet, diaper cake and beer-in-can cake. The customer describes what they want — size, how many bills or cans, whether they will bring the items themselves — and the owner prices it. The picture bouquet has one set price (₱650).

For a money bouquet the customer usually knows the total they want to give (say ₱5,870) but not how many bills that is. The number of bills decides the size of the bouquet and the labour, so the site suggests the fewest bills for that amount.

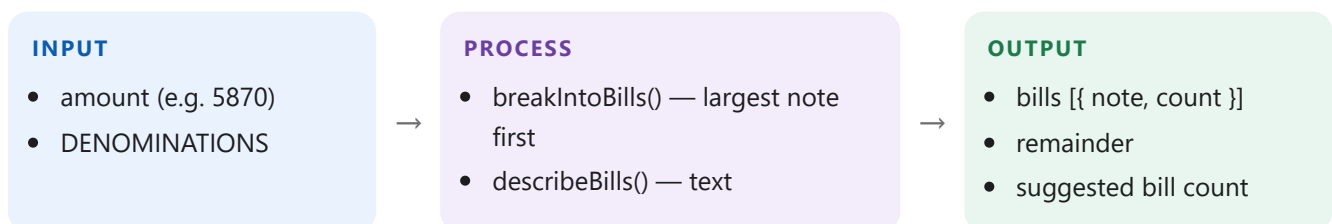
## 2 Input

*What data is needed by the module?*

| Input                    | What it is                                                                                                                         |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Customer types / picks   | Size, number of bills / items / cans, whether they provide the items, brand or details, add-ons, quantity, notes, reference image. |
| Customer types (money)   | The total amount to put inside, e.g. 5870.                                                                                         |
| Stored:<br>DENOMINATIONS | Philippine notes, largest first: 1000, 500, 200, 100, 50, 20.                                                                      |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Greedy algorithm):

- 1 Greedy change-making: start with the amount; for each note from the largest down, take as many as fit (amount ÷ note, rounded down) and subtract them.
- 2 Whatever is left at the end (under ₱20) cannot be made in bills and is reported.

3 Show "Fewest bills: ₱1,000 × 5 · ₱500 × 1 · ... = 10 bills" with a button to use that count.

4 Check the gift and add it to the cart as a quote item (priced later by the owner).

## 4 Output

*What result does the module produce?*

- The bill suggestion under the amount box, and the "Use N bills" button.
- A quote item in the cart, labelled "Quote".

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                     | Time   | Space  | Note                                    |
|-------------------------------|--------|--------|-----------------------------------------|
| <code>breakIntoBills()</code> | $O(n)$ | $O(n)$ | Greedy change-making.                   |
| <code>billHint()</code>       | $O(n)$ | $O(n)$ | Turns the result into the hint text.    |
| <code>makeQuoteItem()</code>  | $O(n)$ | $O(n)$ | Checks a gift and builds its cart item. |

**What n means here:**  $n$  = number of note denominations (6).

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

For a canonical currency like the peso, greedy is provably optimal: taking the largest note that fits always gives the fewest notes. It runs in  $O(n)$  — six steps — with no search.

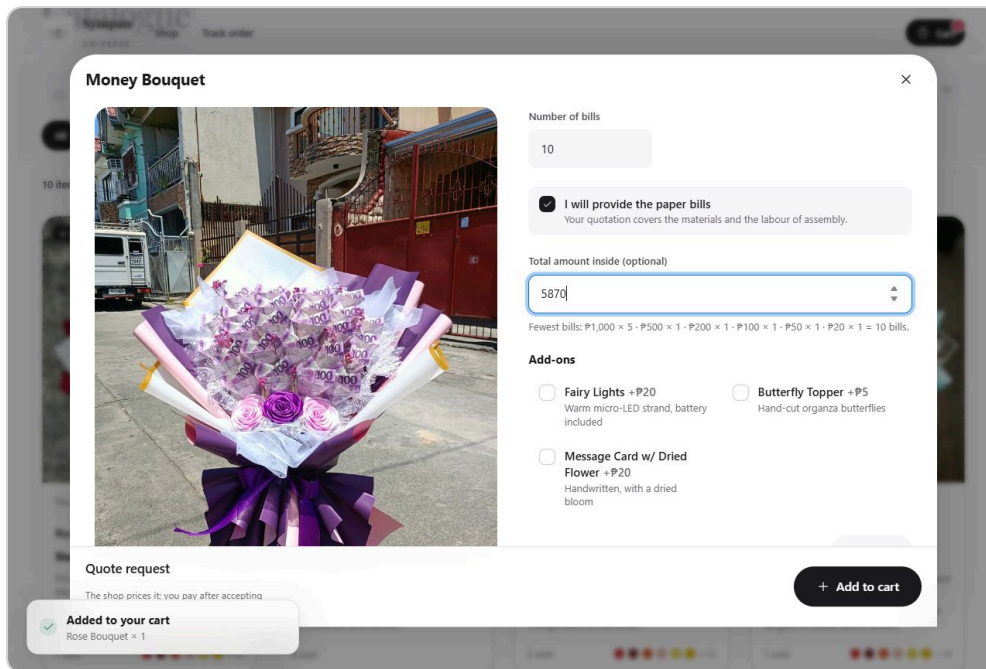
The alternative, dynamic programming over every amount up to ₱5,870, is needed only for odd currencies where greedy can fail; here it would be thousands of steps to reach the same answer.

Fewer bills means a smaller, cheaper bouquet — which is what a customer asking "how many bills?" usually wants.

## 7 Demonstration

*Show that the module works correctly — the input, the process and the output.*

- 1 Open the Money Bouquet. Type 5870 in "Total amount inside".
- 2 The hint shows ₱1,000 × 5 · ₱500 × 1 · ₱200 × 1 · ₱100 × 1 · ₱50 × 1 · ₱20 × 1 = 10 bills.
- 3 Press "Use 10 bills": the bill count updates.



The money bouquet with the greedy bill suggestion

## Trace — the fewest bills (computed by our code)

**breakIntoBills(5870, [1000, 500, 200, 100, 50, 20])**

| Note  | Remaining before | Taken | Remaining after |
|-------|------------------|-------|-----------------|
| ₱1000 | ₱5870            | 5     | ₱870            |
| ₱500  | ₱870             | 1     | ₱370            |
| ₱200  | ₱370             | 1     | ₱170            |
| ₱100  | ₱170             | 1     | ₱70             |
| ₱50   | ₱70              | 1     | ₱20             |
| ₱20   | ₱20              | 1     | ₱0              |

Result:  $\text{P}1,000 \times 5 \cdot \text{P}500 \times 1 \cdot \text{P}200 \times 1 \cdot \text{P}100 \times 1 \cdot \text{P}50 \times 1 \cdot \text{P}20 \times 1 = 10$  bills.

**breakIntoBills(3460, [1000, 500, 200, 100, 50, 20])**

| Note  | Remaining before | Taken | Remaining after |
|-------|------------------|-------|-----------------|
| ₱1000 | ₱3460            | 3     | ₱460            |
| ₱500  | ₱460             | 0     | ₱460            |
| ₱200  | ₱460             | 2     | ₱60             |
| ₱100  | ₱60              | 0     | ₱60             |
| ₱50   | ₱60              | 1     | ₱10             |
| ₱20   | ₱10              | 0     | ₱10             |

Result:  $\text{P}1,000 \times 3 \cdot \text{P}200 \times 2 \cdot \text{P}50 \times 1 = 6$  bills; ₱10 cannot be made in bills.

### breakIntoBills(15, [1000, 500, 200, 100, 50, 20])

| Note  | Remaining before | Taken | Remaining after |
|-------|------------------|-------|-----------------|
| ₱1000 | ₱15              | 0     | ₱15             |
| ₱500  | ₱15              | 0     | ₱15             |
| ₱200  | ₱15              | 0     | ₱15             |
| ₱100  | ₱15              | 0     | ₱15             |
| ₱50   | ₱15              | 0     | ₱15             |
| ₱20   | ₱15              | 0     | ₱15             |

Result: no bills = 0 bills; ₱15 cannot be made in bills.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/algorithms.js · lines 141–154 · breakIntoBills() — Greedy change-making.

```
141 function breakIntoBills(amount, denominations) {
142 ① var remaining = Math.floor(Number(amount) || 0);
143    if (remaining < 0) remaining = 0;
144    var bills = [];
145 ② for (var i = 0; i < denominations.length; i++) {
146      var note = denominations[i];
147 ③ var count = Math.floor(remaining / note);
148      if (count > 0) {
149        listAdd(bills, { note: note, count: count });
150 ④ remaining -= note * count;
151      }
152    }
153 ⑤ return { bills: bills, remainder: remaining, total: sumRecursive(bills, function (b) { return b.count;
    }) };
154 }
```

- ① line 142 Start with the whole amount.
- ② line 145 Go through the notes, largest first.
- ③ line 147 Take as many of this note as fit.
- ④ line 150 Subtract them; move on to the next smaller note.
- ⑤ line 153 Return the bills, what could not be made, and the total count.

Time  $O(n)$  · Space  $O(n)$

js/shop-product.js · lines 167–176 · billHint() — Turns the result into the hint text.

```
167 /* The fewest bills for the amount typed – greedy change-making.
168      Time  $O(n)$  · Space  $O(n)$  */
169 function billHint() {
170   var amount = toNumber(productState.detail);
171   if (!(amount > 0)) return 'Optional. Tell us the total and we will suggest the fewest bills.';
172 ① var b = breakIntoBills(amount, DENOMINATIONS);
173   if (b.total === 0) return 'Bills start at ₱20.';
174 ② return 'Fewest bills: ' + describeBills(b) + ' = ' + b.total + ' ' + plural(b.total, 'bill') +
175         (b.remainder > 0 ? ' (₱' + b.remainder + ' cannot be made in bills)' : '') + '.';
176 }
```

- 1 line 172 Run the greedy break-down.
- 2 line 174 Show the notes and the total.

Time  $O(n)$  · Space  $O(n)$

js/store.js · lines 294–326 · makeQuoteItem() — Checks a gift and builds its cart item.

```
294  /* Validate a gift choice and build its item. Quote products (money,
295     makeup, sweets, diaper, beer) are priced later by the owner; a
296     fixed-price product (the picture bouquet) is priced here.
297                                     Time  $O(n)$  · Space  $O(n)$ 
298     choice: { size, count, detail, provided, quantity, addons, cardFlower, notes, reference } */
299  function makeQuoteItem(product, choice) {
300      if (!product || !product.active || (product.kind !== 'quote' && product.kind !== 'fixed')) {
301          return { ok: false, error: 'This item is not available.' };
302      }
303      var type = quoteTypeById(product.quoteType);
304      1 if (!isIn(product.sizes, choice.size)) return { ok: false, error: 'Choose a size.' };
305      2 if (!isWholeIn(choice.count, 1, 500)) return { ok: false, error: type.countLabel + ' must be a whole
        number from 1 to 500.' };
306      if (!isWholeIn(choice.quantity, 1, MAX_QUANTITY)) return { ok: false, error: 'Quantity must be 1 to ' +
        MAX_QUANTITY + '.' };
307      if (String(choice.detail || '').length > 160) return { ok: false, error: 'Keep the details under 160
        characters.' };
308      if (String(choice.notes || '').length > MAX_NOTE) return { ok: false, error: 'Notes are limited to ' +
        MAX_NOTE + ' characters.' };
309
310      var addons = keepWhere(choice.addons || [], function (id) {
311          var addon = addonById(id);
312          return addon !== null && !addon.flowersOnly;
313      });
314      3 var unit = (product.kind === 'fixed' ? product.price : 0) + addonsTotal(addons, 0);
315      return { ok: true, item: {
316          productId: product.id, productName: product.name, kind: product.kind, quoteType: product.quoteType,
317          arrangement: '', count: choice.count, color: '',
318          size: choice.size, detail: strip(choice.detail),
319          provided: type.provided ? choice.provided === true : false,
320          quantity: choice.quantity, addons: addons,
321          cardFlower: isIn(addons, 'card') ? (choice.cardFlower || CARD_FLOWERS[0]) : '',
322          notes: strip(choice.notes), reference: choice.reference || null,
323          unitEstimate: unit, estimate: unit * choice.quantity,
324          materials: 0, labor: 0, itemExpense: 0
325      } };
326  }
```

- 1 Line 304 The size must be one the product offers.
- 2 Line 305 The count must be a whole number from 1 to 500.
- 3 Line 314 Quote items carry only their add-ons; the picture bouquet carries its set price.

Time  $O(n)$  · Space  $O(n)$

# Cart

|         |                                                                                                                                           |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | The cart sheet (the bag button) — lines, + / – and Remove, the subtotal                                                                   |
| Files   | js/structures.js (llAppend, llRemove, llUpdate, llEntries) · js/store.js (basketAdd, basketRemove, basketSetQuantity) · js/shop-basket.js |
| Data    | basket — linked list { nodes, head, tail, size }, each node { value, next, live }                                                         |
| DSA     | <b>Singly linked list</b>                                                                                                                 |

## 1 Business process

*What task or problem from the client are you solving?*

Customers collect several items before checking out — a rose bouquet, a picture bouquet — change quantities, and remove things they no longer want, in any order.

The cart must keep the items in the order they were added and remove any one of them cleanly.

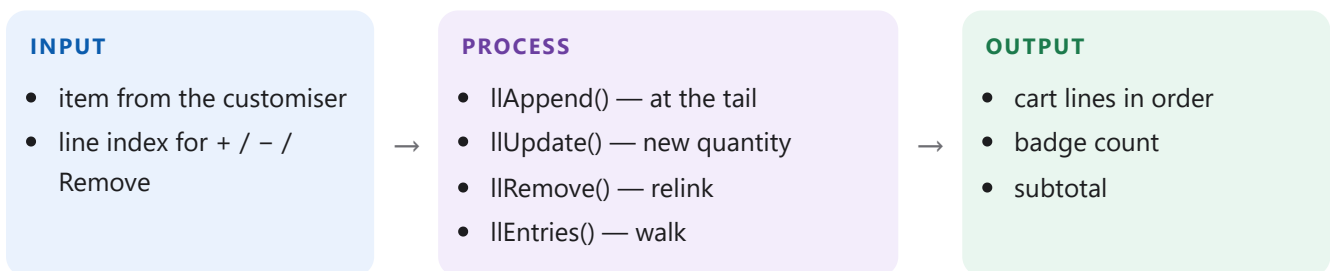
## 2 Input

*What data is needed by the module?*

| Input                    | What it is                                             |
|--------------------------|--------------------------------------------------------|
| Customer picks           | "Add to cart" on a product; + / – on a line; "Remove". |
| Built by modules 4 and 5 | The checked item: product, choices, quantity, price.   |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Singly linked list):

- 1 Append: the new item becomes a node at the tail; the old tail's "next" points to it ( $O(1)$ , thanks to the tail pointer).
- 2 Change quantity: replace that node's value with a copy carrying the new quantity and price.
- 3 Remove: walk from the head to find the node before it, re-point that node's "next" past it, and mark the removed node dead — nothing else moves.
- 4 Show: walk from the head along the "next" pointers.

## 4 Output

*What result does the module produce?*

- The cart sheet with its lines, the Cart badge count, and the subtotal.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                        | Time   | Space  | Note                       |
|----------------------------------|--------|--------|----------------------------|
| <code>llAppend()</code>          | $O(1)$ | $O(1)$ | Adds a node at the tail.   |
| <code>llRemove()</code>          | $O(n)$ | $O(1)$ | Unlinks one node.          |
| <code>llEntries()</code>         | $O(n)$ | $O(n)$ | Walks the list in order.   |
| <code>basketSetQuantity()</code> | $O(1)$ | $O(1)$ | Changes a line's quantity. |

**What n means here:**  $n$  = number of lines in the cart.

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

A linked list is the textbook structure for "add at the end, remove from anywhere, keep the order". Removing a node re-points one link instead of shifting every later item down as an array would.

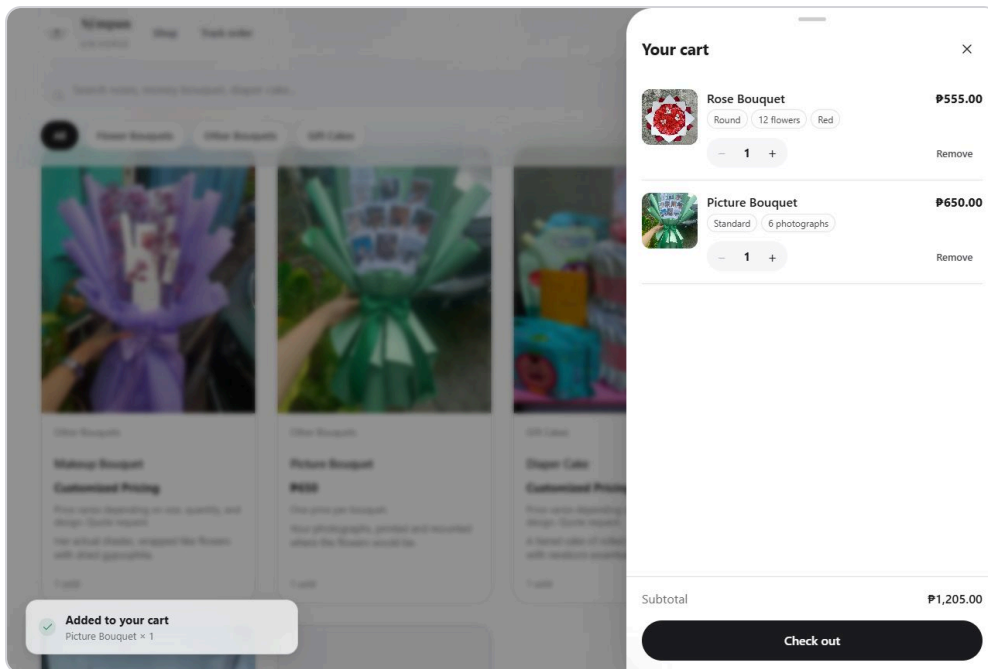
The tail pointer makes adding  $O(1)$ . Keeping the nodes in one array with whole-number "next" links shows the pointer mechanics plainly while still obeying the array-only rule.

The cart is short, so the  $O(n)$  walk to find the node before the one removed costs nothing in practice.

## 7 Demonstration

*Show that the module works correctly — the input, the process and the output.*

- 1 Add a Rose Bouquet and a Picture Bouquet; open the cart: two lines, in that order.
- 2 Press + on the first line: its quantity and price update.
- 3 Remove the first line: the second stays exactly as it was.



The cart with two lines

## Trace — the cart's nodes and links (computed by our code)

After `basketAdd()` three times



head = 0 · tail = 2 · size = 3

**basketRemove(1) — the middle node: node 0 now points past it; nothing moved**



head = 0 · tail = 2 · size = 2

**basketAdd(Money Bouquet) at the tail, and basketSetQuantity(0, 2)**



head = 0 · tail = 3 · size = 3

**basketRemove(0) — the head: head moves to node 2**



head = 2 · tail = 3 · size = 2

Walking from the head (llEntries): Dahlia Bouquet × 1 → Money Bouquet × 1

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

**js/structures.js** · lines 29–39 · `llAppend()` — Adds a node at the tail.

```
29  /* Append at the tail. Keeping a tail pointer is what makes this O(1)
30     rather than a walk to the end.           Time O(1) · Space O(1) */
31  function llAppend(list, value) {
32      var index = list.nodes.length;
33  ❶  list.nodes[index] = { value: value, next: NIL, live: true };
34  ❷  if (list.head === NIL) list.head = index;
35  ❸  else list.nodes[list.tail].next = index;
36  ❹  list.tail = index;
37      list.size++;
38      return index;
39  }
```

❶ line 33 Make a node in the next free slot; it points nowhere yet.

❷ line 34 If the list was empty it becomes the head...

❸ line 35 ...otherwise the old tail points to it.

❹ line 36 It is the new tail.

Time O(1) · Space O(1)

**js/structures.js** · lines 41–62 · `llRemove()` — Unlinks one node.

```
41  /* Unlink the node at an index. A singly linked list has no back pointer,
42     so the predecessor is found by walking from the head.
43                                     Time O(n) · Space O(1) */
44  function llRemove(list, index) {
45      var node = list.nodes[index];
46      if (!node || !node.live) return false;
47
48  ❶  if (list.head === index) {
49      list.head = node.next;
50      if (list.tail === index) list.tail = NIL;
51  } else {
52      var prev = list.head;
53  ❷  while (prev !== NIL && list.nodes[prev].next !== index) prev = list.nodes[prev].next;
54      if (prev === NIL) return false;
55  ❸  list.nodes[prev].next = node.next;
56      if (list.tail === index) list.tail = prev;
57  }
58  ❹  node.live = false;
59      node.next = NIL;
60      list.size--;
61      return true;
62  }
```

❶ line 48 Removing the head: the head moves to the next node.

❷ line 53 Otherwise walk to the node before it.

❸ line 55 Skip over it.

❹ line 58 Mark it dead; nothing else moves.

Time O(n) · Space O(1)

**js/structures.js** · lines 78–88 · `llEntries()` — Walks the list in order.

```
78  /* Walk head to tail: [{ index, value }]. Time O(n) · Space O(n) */
79  function llEntries(list) {
80  ❶    var out = [], cursor = list.head, guard = 0;
81      while (cursor !== NIL && guard <= list.nodes.length) {
82          var node = list.nodes[cursor];
83          out[out.length] = { index: cursor, value: node.value };
84  ❷    cursor = node.next;
85          guard++;
86      }
87      return out;
88  }
```

❶ line 80 Start at the head.

❷ line 84 Follow each "next" pointer until NIL.

Time  $O(n)$  · Space  $O(n)$

**js/store.js** · lines 381–386 · `basketSetQuantity()` — Changes a line's quantity.

```
381  /* Change one line's quantity; its estimate follows. Time O(1) · Space O(1) */
382  function basketSetQuantity(index, quantity) {
383      var item = llGet(basket, index);
384      if (!item || !isWholeIn(quantity, 1, MAX_QUANTITY)) return false;
385  ❶    return llUpdate(basket, index, copyRecord(item, { quantity: quantity, estimate: item.unitEstimate *
        quantity }));
386  }
```

❶ line 385 Replace the node's value with a copy carrying the new quantity and price.

Time  $O(1)$  · Space  $O(1)$

## PART 3

# Checkout and payment

Module 7 Checkout & Delivery · Module 8 Quotation Desk · Module 9 Payment, Receipts & Email

## 60-second pitch

These modules take the customer from cart to paid order. Checkout & Delivery finds the delivery fee for whatever city is typed with a hash table of 45 names and aliases, and checks every field by scanning its characters — no regular expressions. The Quotation Desk answers quote requests first come, first served with a circular queue, and pre-fills each quotation from the owner's past quotes, put newest first with insertion sort. Payment, Receipts & Email takes GCash payments, snapshots each one into a receipt, and sends every email in order through a second circular queue.

## Structures and algorithms to be able to explain

| Where                   | Structure / algorithm                               | Time                     | Space  |
|-------------------------|-----------------------------------------------------|--------------------------|--------|
| City → delivery area    | Hash table (normalised names and aliases)           | $O(1)$ average           | $O(n)$ |
| Check the checkout form | Character scans; each cart line's product looked up | $O(n^2)$                 | $O(n)$ |
| Quote requests in order | Circular queue (FIFO)                               | $O(1)$ enqueue / dequeue | $O(n)$ |
| Pre-fill a quotation    | Insertion sort of past quotes + linear search       | $O(n^2)$                 | $O(n)$ |
| Reference used before?  | Linear search of the receipts                       | $O(n)$                   | $O(1)$ |
| Send emails in order    | Circular queue (FIFO)                               | $O(1)$ per email         | $O(n)$ |

## Likely panel questions

### Why does a circular queue never shift its items?

The head and the tail move forward and wrap around with "% capacity". Dequeueing only moves the head; its slot is reused later. A plain array that removes from the front would move every remaining item:  $O(n)$ .

### What happens when the queue is full?

cqGrow doubles the buffer and copies the items in queue order, so the head lands at slot 0. That copy is  $O(n)$  but rare, so enqueue stays  $O(1)$  amortised.

### How is " QUEZON city " understood?

normaliseKey lower-cases, trims and collapses runs of spaces, dots and commas, character by character. Every city and alias went into the table the same way, so any spelling hashes to the same bucket.

### How is a GCash reference stopped from paying twice?

gcashProblem checks for 13 digits, then receiptWithRef searches the receipts for the same reference — a linear search,  $O(n)$ .

### Why is a receipt a copy of the order?

A receipt records what was paid at that moment. If the order is edited later, the receipt must not change.

**Why send emails through a queue?**

"Order confirmed" must arrive before "ready for pickup", and only one request is sent at a time. The queue keeps that order, and a failed email can be queued again without being lost.

# Checkout & Delivery

|         |                                                                                                                                                                      |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | Checkout step 2 — contact details, pickup (a date) or delivery (address, courier, date and time)                                                                     |
| Files   | js/store.js (buildAreaIndex, lookupArea, deliveryQuote, slotLoad, pickupPlacesLeft) · js/orders.js (validateRequest, isValidEmail, isValidPhone) · js/shop-basket.js |
| Data    | areaIndex — hash table, 53 buckets; DELIVERY_AREAS (35 cities with aliases); COURIERS; orders (bookings already taken)                                               |
| DSA     | <b>Hash table · Validation</b>                                                                                                                                       |

## 1 Business process

*What task or problem from the client are you solving?*

At checkout the customer gives their name, email and mobile number and chooses pickup (a date only, Monday to Saturday) or delivery (address, date and time).

The delivery fee depends on the city: free in Brgy. Lawa, ₱20–₱50 for nearby towns the shop delivers to itself (Friday to Sunday), and a courier rate (Flash Express or Lalamove) for Metro Manila and the rest of Bulacan. The fee must be worked out the moment the city is typed, however it is typed.

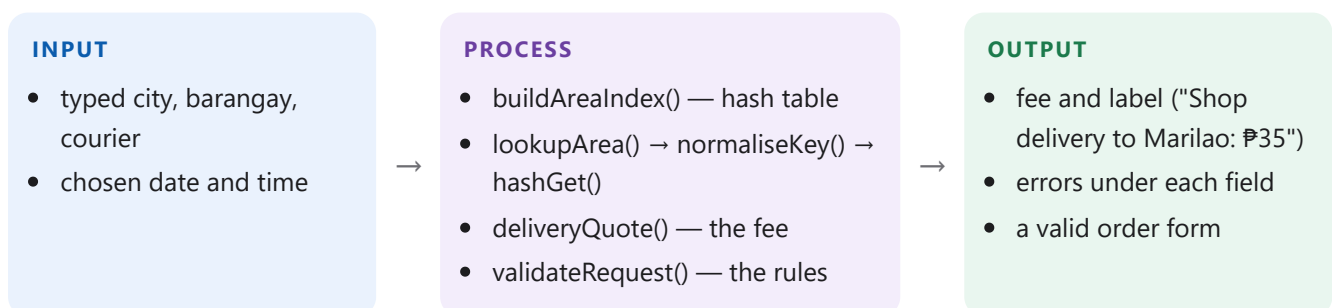
## 2 Input

*What data is needed by the module?*

| Input                  | What it is                                                                                           |
|------------------------|------------------------------------------------------------------------------------------------------|
| Customer types / picks | Name, email, mobile, pickup or delivery; street, barangay, city; courier; date; delivery time; rush. |
| Stored: DELIVERY_AREAS | 35 cities with aliases, their service (shop or courier) and fee or region.                           |
| Stored: COURIERS       | Each courier's rate per region.                                                                      |
| Stored: orders         | Bookings already taken, for time-slot and pickup-day capacity.                                       |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Hash table · Validation):

- At start-up, put every city name and alias, normalised, into a hash table (53 buckets).
- When a city is typed: normalise it (" QUEZON city " → "quezon city"), hash it, scan that bucket — the area record in O(1) on average. If "Marilao, Bulacan" is not found, try the part before the comma.

- 3 Shop area: the fee is the area's fee, or free if the barangay is Lawa. Courier area: look up the chosen courier's rate for the area's region.
- 4 Check everything by scanning characters (no regular expressions): the email format, the mobile number, the date against the lead time, Sundays (pickup) or Friday–Sunday (shop delivery), and the places left that day or time.

## 4 Output

*What result does the module produce?*

- The fee notice under the address, the courier choices with their rates, the places left for each time or pickup date, and field errors.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                      | Time                                     | Space  | Note                                         |
|--------------------------------|------------------------------------------|--------|----------------------------------------------|
| <code>buildAreaIndex()</code>  | $O(n)$                                   | $O(n)$ | Puts every city and alias in the hash table. |
| <code>lookupArea()</code>      | $O(1)$ average + $O(n)$ to tidy the text | $O(n)$ | Finds the area for whatever was typed.       |
| <code>deliveryQuote()</code>   | $O(n)$                                   | $O(1)$ | Works out the fee.                           |
| <code>validateRequest()</code> | $O(n^2)$                                 | $O(n)$ | Checks the checkout form.                    |

**What n means here:** For the look-up,  $n$  = the length of the typed city (to normalise it); the table holds 45 names. For the form check,  $n$  = the cart lines (each one's product is looked up — a loop inside a loop) and the orders (for capacity).

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Customers type city names in every possible way. Normalising then hashing turns any spelling into a direct look-up —  $O(1)$  on average — instead of comparing the input with all 45 names on every keystroke.

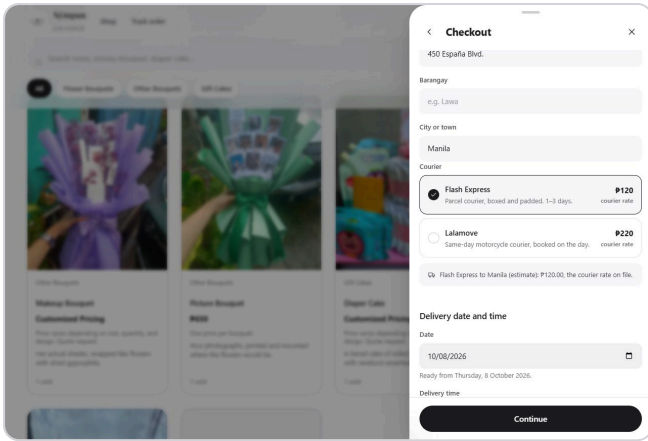
Aliases ("QC", "Sta Maria", "Paranaque") are simply extra keys pointing at the same record, which a hash table handles naturally.

Checking by scanning characters is required by the project rules (no regular expressions) and gives a precise message for each kind of mistake. Fees live in data, not code: the owner can add a city by adding one record.

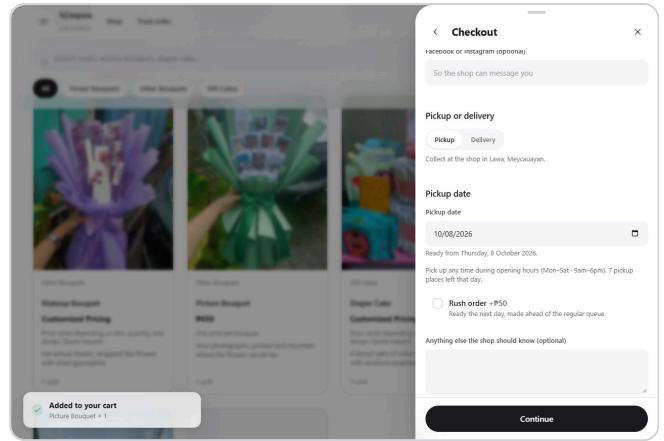
## 7 Demonstration

*Show that the module works correctly — the input, the process and the output.*

- 1 Add a bouquet and check out. Choose Delivery and type "quezon CITY ": the courier choices appear with ₱120 / ₱220.
- 2 Type "Marilao": "Shop delivery to Marilao: ₱35", Friday to Sunday.
- 3 Choose Pickup: only a date — no time slots — and the places left that day.



Delivery to Manila: courier choices with their rates



Pickup: a date only

## Trace — the delivery-area look-ups (computed by our code)

### lookupArea() and deliveryQuote() for six typed addresses

| Typed              | normaliseKey    | Bucket | Area        | Label                                   | Fee  | Status   |
|--------------------|-----------------|--------|-------------|-----------------------------------------|------|----------|
| " quezon CITY "    | quezon city     | 46     | Quezon City | Lalamove to Quezon City (estimate)      | ₱220 | estimate |
| "Marilao, Bulacan" | marilao bulacan | 46     | Marilao     | Shop delivery to Marilao                | ₱35  | final    |
| "Meycauayan"       | meycauayan      | 3      | Meycauayan  | Free delivery within Brgy. Lawa         | ₱0   | final    |
| "meycauayan city"  | meycauayan city | 40     | Meycauayan  | Shop delivery to Meycauayan             | ₱20  | final    |
| "QC"               | qc              | 31     | Quezon City | Flash Express to Quezon City (estimate) | ₱120 | estimate |
| "Davao"            | davao           | 49     | not found   | Flash Express — fee set by the shop     | ₱0   | manual   |

### The table built by buildAreaIndex()

45 names and aliases in 53 buckets; 30 buckets in use; longest chain 3; load factor 0.85.

[46] quezon city → pasig → hagonoy

[31] qc → navotas

[48] marilao

### validateRequest() — a pickup on Sunday 11 October with a bad email

| Field | Message                                                                    |
|-------|----------------------------------------------------------------------------|
| email | Enter an email address like name@gmail.com — your confirmation goes there. |
| date  | The shop is closed on Sundays. Choose another pickup date.                 |

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/store.js · lines 428–437 · buildAreaIndex() — Puts every city and alias in the hash table.

```
428  /* Every city and alias, normalised, to its area.  Time O(n) · Space O(n) */
429  function buildAreaIndex() {
430  ❶    var table = hashCreate(53);
431      for (var i = 0; i < DELIVERY_AREAS.length; i++) {
432          var area = DELIVERY_AREAS[i];
433  ❷    hashPut(table, normaliseKey(area.city), area);
434  ❸    for (var k = 0; k < area.aliases.length; k++) hashPut(table, normaliseKey(area.aliases[k]), area);
435      }
436      return table;
437  }
```

❶ Line 430 53 buckets for 45 keys.

❷ Line 433 The city's normalised name...

❸ Line 434 ...and each alias, all pointing at the same record.

Time O(n) · Space O(n)

js/store.js · lines 439–447 · lookupArea() — Finds the area for whatever was typed.

```
439  /* " quezon city " finds Quezon City.  Time O(1) average + O(n) to tidy the text · Space O(n) */
440  function lookupArea(city) {
441      if (isBlank(city)) return null;
442  ❶    var found = hashGet(areaIndex, normaliseKey(city));
443      if (found) return found;
444      // "Marilao, Bulacan" - try the part before the first comma.
445  ❷    var head = cutText(city, ',')[0];
446      return hashGet(areaIndex, normaliseKey(head));
447  }
```

❶ Line 442 Normalise and look up.

❷ Line 445 Not found? Try the part before a comma ("Marilao, Bulacan").

Time O(1) average + O(n) to tidy the text · Space O(n)

js/store.js · lines 468–496 · deliveryQuote() — Works out the fee.

```
468  /* Work out the delivery fee for an address.  Time O(n) · Space O(1)
469      status: final    the shop's own fee, nothing to confirm
470      estimate a courier planning rate, the owner confirms it
471      manual no rate known, the owner enters it */
472  function deliveryQuote(f) {
473      if (!f || f.mode !== 'delivery') {
474  ❶    return { service: 'pickup', fee: 0, status: 'final', label: 'Pickup at the shop', area: '',
475              courier: '' };
476      }
477      var area = lookupArea(f.city);
478      if (area && area.service === 'inhouse') {
479  ❷    var free = isIn(area.freeBarangays || [], barangayKey(f.barangay));
480      return {
481  ❸    service: 'inhouse', fee: free ? 0 : area.fee, status: 'final', area: area.city, courier: '',
482          label: free ? 'Free delivery within Brgy. Lawa' : 'Shop delivery to ' + area.city
483      };
484      }
485      var courier = courierById(f.courier);
486      var courierName = courier ? courier.name : 'Courier';
487      if (area && courier) {
488  ❹    var rate = courierRate(courier.id, area.region);
489      if (rate > 0) {
490          return { service: 'courier', fee: rate, status: 'estimate', area: area.city, courier:
491              courier.id,
492              label: courierName + ' to ' + area.city + ' (estimate)' };
493      }
494      }
495      }
496  }
```

```

491     }
492 }
493 ⑤ return { service: 'courier', fee: 0, status: 'manual', area: area ? area.city : strip(f.city),
494           courier: courier ? courier.id : '',
495           label: courierName + ' - fee set by the shop' };
496 }

```

- ① Line 474 Pickup: free.
- ② Line 478 Shop area: free in Brgy. Lawa...
- ③ Line 480 ...otherwise the area's fee.
- ④ Line 487 Courier area: the chosen courier's rate for the region.
- ⑤ Line 493 No rate on file: the fee must be set by the owner.

Time  $O(n)$  · Space  $O(1)$

**js/orders.js** · lines 144–200 · `validateRequest()` — Checks the checkout form.

```

144  /* form: { name, phone, handle, mode, date, slot, address, barangay, city,
145           courier, rush, notes }
146   Returns a list of { field, message }; empty means valid.
147           Time  $O(n^2)$  · Space  $O(n)$  */
148  function validateRequest(form, items, fromIso, ignoreId) {
149      var errors = [];
150      /* Record one problem.           Time  $O(1)$  · Space  $O(1)$  */
151      function fail(field, message) { listAdd(errors, { field: field, message: message }); }
152
153      if (items.length === 0) fail('basket', 'Add at least one item to your cart.');
```

```

154      for (var i = 0; i < items.length; i++) {
155          var product = productById(items[i].productId);
156          if (!product || !product.active) fail('basket', items[i].productName + ' is no longer available.
Remove it to continue.');
```

```

157      }
158      var name = strip(form.name);
159      if (name.length < 2 || name.length > 60) fail('name', 'Enter your name (2 to 60 characters).');
```

```

160      if (!isValidPhone(form.phone)) fail('phone', 'Enter a mobile number like 0917 123 4567.');
```

```

161  ① if (!isValidEmail(form.email)) fail('email', 'Enter an email address like name@gmail.com - your
confirmation goes there.');
```

```

162      if (String(form.handle || '').length > 60) fail('handle', 'Keep the Facebook or Instagram name under 60
characters.');
```

```

163      if (String(form.notes || '').length > MAX_NOTE) fail('notes', 'Notes are limited to ' + MAX_NOTE + '
characters.');
```

```

164
165      if (!modeById(form.mode)) fail('mode', 'Choose pickup or delivery.');
```

```

166      var earliest = earliestDate(items, form.rush === true, fromIso);
167      if (!dateFromIso(form.date)) fail('date', 'Choose a date.');
```

```

168  ② else if (form.date < earliest) fail('date', 'The earliest the shop can have this ready is ' +
formatDateLong(earliest) + '.');
```

```

169
170      if (form.mode === 'delivery') {
171          if (isBlank(form.address)) fail('address', 'Enter the street and house number.');
```

```

172          if (isBlank(form.city)) fail('city', 'Enter the city or town.');
```

```

173          else if (needsCourier(form.city)) {
174              if (!courierById(form.courier)) fail('courier', 'Choose a courier for this address.');
```

```

175          } else if (dateFromIso(form.date) && !isInhouseDay(form.date)) {
176  ③ fail('date', 'The shop delivers on ' + INHOUSE_DAY_NAMES + ' only.');
```

```

177          }
178      }
179
180      if (form.mode === 'delivery') {
181          if (!isIn(TIME_SLOTS, form.slot)) fail('slot', 'Choose a delivery time.');
```

```

182          else if (dateFromIso(form.date) && slotLoad(form.date, form.slot, ignoreId) >= SLOT_CAPACITY) {
183              fail('slot', 'That time is fully booked. Choose another.');
```

```

184          }

```

```

185     } else if (dateFromIso(form.date) && weekdayOf(form.date) === 0) {
186 ❹      fail('date', 'The shop is closed on Sundays. Choose another pickup date.');
```

```

187     } else if (dateFromIso(form.date) && pickupPlacesLeft(form.date, ignoreId) === 0) {
188         // Pickups are booked by date alone.
189         fail('date', 'Pickups are fully booked on ' + formatDateLong(form.date) + '. Choose another
date.');
```

```

190     }
191     // A price-list cart is never a quote request, so it needs a delivery
192     // rate on file. Only a cart with a quote product can ask for one.
193     var hasQuoteItem = countWhere(items, function (item) { return item.kind === 'quote'; }) > 0;
194     if (form.mode === 'delivery' && !isBlank(form.city) && courierById(form.courier) && !hasQuoteItem &&
195         deliveryQuote(form).status === 'manual') {
196 ❺      fail('city', 'We have no delivery rate for ' + strip(form.city) + ' yet. Choose pickup or one of
the listed cities, or message us at ' +
197         SHOP_INFO.social + '.');
```

```

198     }
199     return errors;
200 }
```

- ❶ Line 161 An email is required — confirmations go there.
- ❷ Line 168 Not earlier than the lead time allows.
- ❸ Line 176 Shop deliveries only Friday to Sunday.
- ❹ Line 186 Pickups: Monday to Saturday, by date only.
- ❺ Line 196 A price-list cart needs a rate on file — it is never turned into a quote.

Time  $O(n^2)$  · Space  $O(n)$

# Quotation Desk

|         |                                                                                                                                                                                       |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | Order desk → "Quote requests" tab, and the quote editor                                                                                                                               |
| Files   | js/structures.js (cqEnqueue, cqDequeue, cqGrow) · js/orders.js (leaveQueue, quoteHistory, recentQuotes, pastQuoteFrom, quoteDraft, sendQuote) · js/admin-desk.js · js/admin-orders.js |
| Data    | quoteQueue — circular queue of order ids { items, head, tail, count, capacity }; orders (past quotations)                                                                             |
| DSA     | <b>Circular queue · Insertion sort</b>                                                                                                                                                |

## 1 Business process

*What task or problem from the client are you solving?*

Quote requests for the five custom gifts arrive while the owner is busy. They must be answered fairly — first come, first served — and pricing each one by hand from scratch is slow.

So requests wait in a first-in, first-out queue, and each quotation is pre-filled from the owner's own past quotes for the same product and size; usually the owner only checks it and presses send.

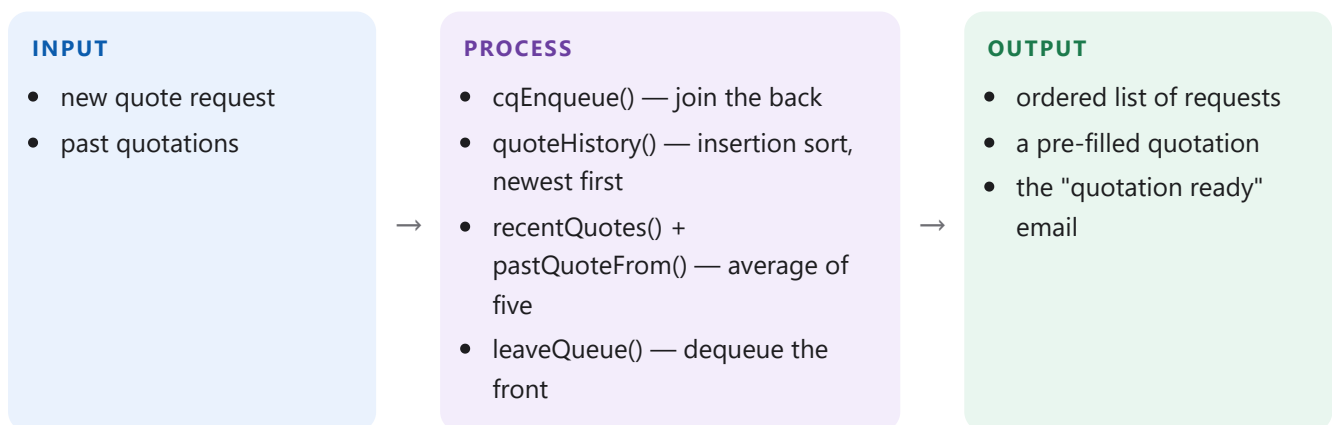
## 2 Input

*What data is needed by the module?*

| Input              | What it is                                                                |
|--------------------|---------------------------------------------------------------------------|
| Stored: quoteQueue | Order ids waiting for a quotation, in arrival order.                      |
| Stored: orders     | Past quotations: product, size, materials, labour, quantity, date.        |
| Owner types        | Materials, labour and item cost per line; delivery and rush fees; a note. |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Circular queue · Insertion sort):

- 1 A request joins the back of a circular queue (enqueue). The desk lists the queue front to back; the front is next.

- 2 Pre-fill: quoteHistory() reads every quoted order once and collects its quoted lines, per piece and without their add-ons; insertion sort puts them newest first. For each line of the request, recentQuotes() walks that list and stops at the first five for the same product and size (or any size, marked); their average, times the quantity, fills the line.
- 3 When the quotation is sent, the request leaves the queue — from the front in  $O(1)$  when it is the next in line (dequeue) — and the customer is emailed.
- 4 When the buffer is full it doubles in size, unrolling so the head is at slot 0.

## 4 Output

*What result does the module produce?*

- The "Quote requests" tab in order, with position numbers.
- The quote editor, pre-filled and marked "Filled in from your last N quotes".
- The quotation email to the customer.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation       | Time                                 | Space  | Note                                                  |
|-----------------|--------------------------------------|--------|-------------------------------------------------------|
| cqEnqueue()     | $O(1)$ amortised ( $O(n)$ on a grow) | $O(1)$ | Joins the back of the queue.                          |
| cqDequeue()     | $O(1)$                               | $O(1)$ | Leaves from the front.                                |
| cqGrow()        | $O(n)$                               | $O(n)$ | Doubles a full buffer.                                |
| leaveQueue()    | $O(n)$                               | $O(n)$ | Takes an id out when it is quoted or voided.          |
| quoteHistory()  | $O(n^2)$                             | $O(n)$ | Every quoted line the owner has priced, newest first. |
| recentQuotes()  | $O(n)$                               | $O(1)$ | The five most recent quotes for one product.          |
| pastQuoteFrom() | $O(n)$                               | $O(1)$ | What the owner charged before.                        |
| sendQuote()     | $O(n)$                               | $O(n)$ | Sends the quotation.                                  |

**What n means here:** For the queue,  $n$  = requests waiting. For the pre-fill,  $n$  = past quoted lines — every line of every quoted order.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

A queue is the fair-service structure: first in, first out. A circular buffer reuses the slots freed at the front, so enqueue and dequeue are  $O(1)$  and the array never creeps forward as removing from the front of a plain array would.

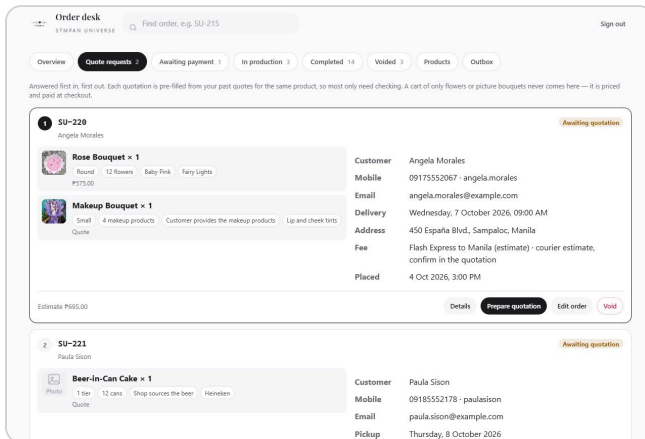
Insertion sort puts the past quotes in date order and is stable. The list is read back to front first, so it is already nearly newest first — close to the  $O(n)$  best case. The history is built once per draft, so each line of the request searches one list instead of every order again.

Averaging the five most recent quotes follows the owner's current prices rather than old ones, and pre-filling from the owner's own history automates the job without inventing prices.

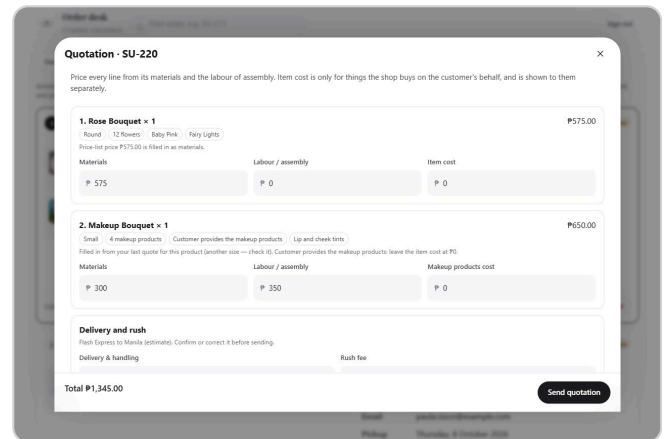
## 7 Demonstration

Show that the module works correctly — the input, the process and the output.

- 1 Check out a cart with a Money Bouquet: it becomes a quote request.
- 2 In the order desk, open "Quote requests": it is last in line; SU-220 is first.
- 3 Press "Prepare quotation": the figures are already filled in from past Money Bouquet quotes. Send it.



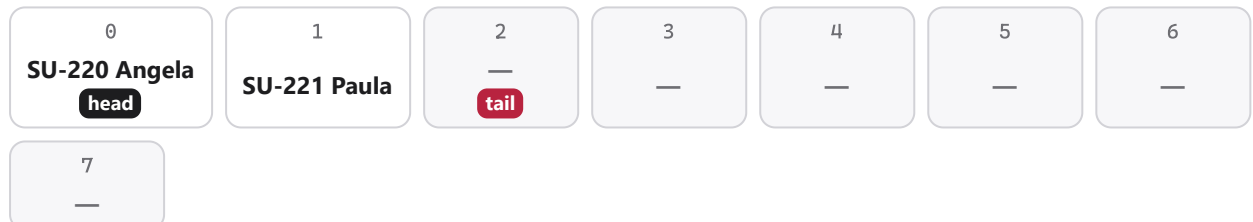
Quote requests, first in first out



The quote editor, pre-filled from past quotes

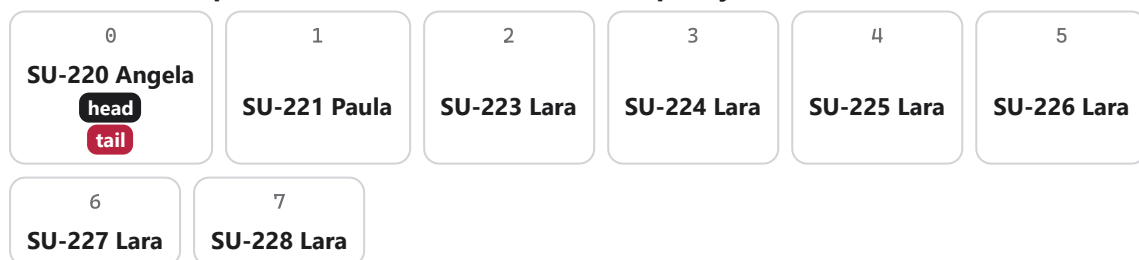
## Trace — the quote queue and the pre-fill (computed by our code)

### The quote queue after the demo history



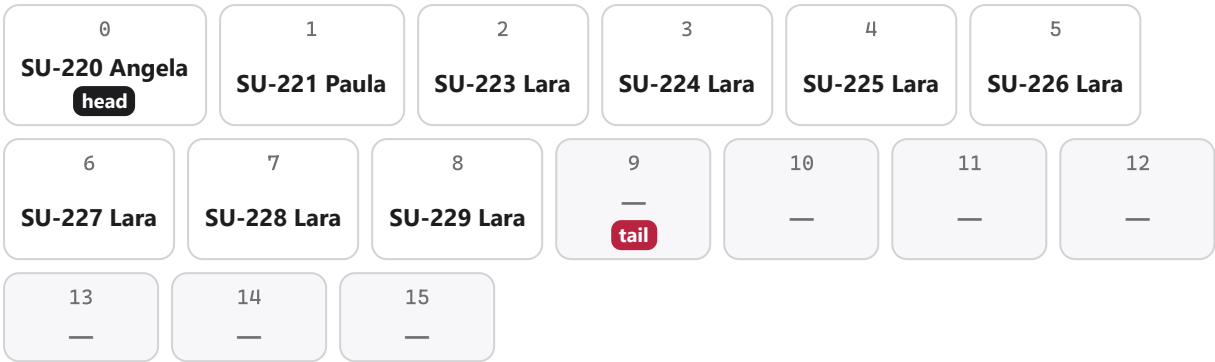
$$\text{capacity} = 8 \cdot \text{count} = 2 \cdot \text{head} = 0 \cdot \text{tail} = 2$$

### After 6 more requests: the buffer is full (count = capacity)



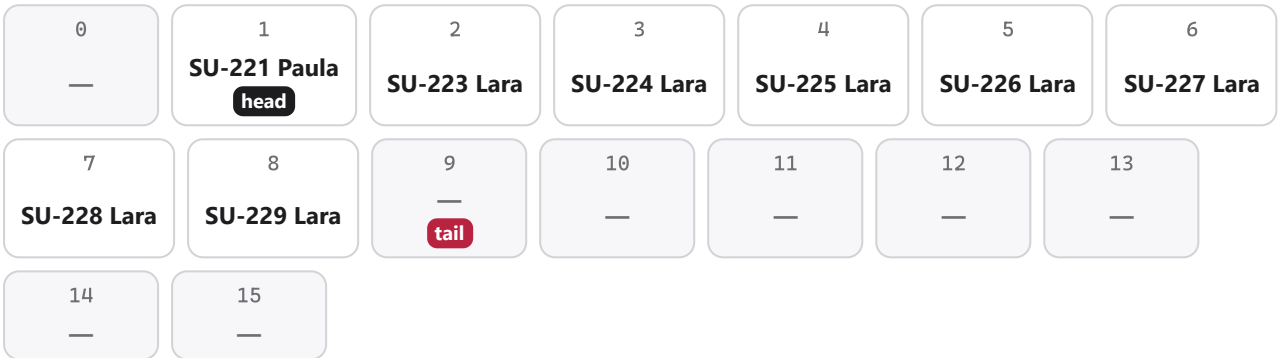
$$\text{capacity} = 8 \cdot \text{count} = 8 \cdot \text{head} = 0 \cdot \text{tail} = 0$$

The 7th request: `cqEnqueue()` finds it full, `cqGrow()` doubles it and unrolls it so the head is at 0



capacity = 16 · count = 9 · head = 0 · tail = 9

`sendQuote()` on the front request: `leaveQueue()` dequeues it in  $O(1)$



capacity = 16 · count = 8 · head = 1 · tail = 9

`quoteDraft()` for a Medium Money Bouquet — the owner's past quotes, per piece without add-ons

`quoteHistory()` read every quoted order once and insertion-sorted its 9 quoted lines newest first.

`recentQuotes()` then walked that list and stopped after the first five for this product and size:

| Quoted on            | Materials | Labour |
|----------------------|-----------|--------|
| 28 Aug 2026, 4:00 PM | ₱250      | ₱300   |

Their average: materials ₱250, labour ₱300 per piece (from 1 quote). `quoteDraft()` fills these in for the owner, scaled by the quantity.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

`js/structures.js` · lines 137–144 · `cqEnqueue()` — Joins the back of the queue.

```
137  /* Join at the back.           Time O(1) amortised (O(n) on a grow) · Space O(1) */
138  function cqEnqueue(queue, value) {
139  ❶    if (queue.count === queue.capacity) cqGrow(queue);
140  ❷    queue.items[queue.tail] = value;
141  ❸    queue.tail = (queue.tail + 1) % queue.capacity;
142    queue.count++;
143    return queue.count;
144  }
```

❶ line 139 Full? Double the buffer first.

❷ line 140 Write at the tail...

3 line 141 ...and move the tail on, wrapping around with modulo.

Time  $O(1)$  amortised ( $O(n)$  on a grow) · Space  $O(1)$

js/structures.js · lines 146–154 · cqDequeue() — Leaves from the front.

```
146  /* Leave from the front, or null when empty. Time  $O(1)$  · Space  $O(1)$  */
147  function cqDequeue(queue) {
148      if (queue.count === 0) return null;
149  1  var value = queue.items[queue.head];
150      queue.items[queue.head] = null;
151  2  queue.head = (queue.head + 1) % queue.capacity;
152      queue.count--;
153      return value;
154  }
```

1 Line 149 Read the front.

2 Line 151 Move the head on, wrapping around.

Time  $O(1)$  · Space  $O(1)$

js/structures.js · lines 125–135 · cqGrow() — Doubles a full buffer.

```
125  /* Double the buffer when full, unrolling it so head is index 0.
126                                     Time  $O(n)$  · Space  $O(n)$  */
127  function cqGrow(queue) {
128      var bigger = [], cap = queue.capacity * 2;
129      for (var i = 0; i < cap; i++) bigger[i] = null;
130  1  for (var k = 0; k < queue.count; k++) bigger[k] = queue.items[(queue.head + k) % queue.capacity];
131      queue.items = bigger;
132      queue.capacity = cap;
133      queue.head = 0;
134      queue.tail = queue.count;
135  }
```

1 Line 130 Copy the items in queue order, so the head lands at slot 0.

Time  $O(n)$  · Space  $O(n)$

js/orders.js · lines 320–326 · leaveQueue() — Takes an id out when it is quoted or voided.

```
320  /* Take an id out of a queue: from the front in  $O(1)$  when it is the next
321      in line – the usual case – or from the middle in  $O(n)$  when it is not.
322                                     Time  $O(n)$  · Space  $O(n)$  */
323  function leaveQueue(queue, id) {
324  1  if (cqFront(queue) === id) return cqDequeue(queue) === id;
325  2  return cqRemove(queue, id);
326  }
```

1 Line 324 Normally it is at the front:  $O(1)$  dequeue.

2 Line 325 Otherwise remove it from the middle, keeping the order.

Time  $O(n)$  · Space  $O(n)$

js/orders.js · lines 339–364 · quoteHistory() — Every quoted line the owner has priced, newest first.

```
339  /* Every quoted line of a quote product the shop has priced, per piece and
340      without its add-ons, newest quotation first. Learned from the order
341      records, so it improves as the shop quotes more. Built once per draft:
342      each line of the new order then searches this one list.
343                                     Time  $O(n^2)$  · Space  $O(n)$  */
344  function quoteHistory(ignoreId) {
345      var seen = [];
```

```

346     for (var o = 0; o < orders.length; o++) {
347         var order = orders[o];
348 ❶ if (order.id === ignoreId || order.quotedAt === 0 || order.status === 'requested') continue;
349         for (var i = 0; i < order.items.length; i++) {
350             var past = order.items[i];
351             if (past.kind === 'quote' && lineTotal(past) > 0) {
352                 // Take the past add-ons out, so the new line's own add-ons are
353                 // not counted twice when they are added back.
354 ❷ var base = past.materials - past.estimate;
355                 listAdd(seen, { seq: seen.length, productId: past.productId, size: past.size, stamp:
order.quotedAt,
356                                     materials: (base > 0 ? base : 0) / past.quantity, labor: past.labor /
past.quantity });
357             }
358         }
359     }
360     // Newest first. Orders are kept oldest first, so reading the list back to
361     // front leaves it nearly in order – insertion sort's best case. Lines
362     // quoted at the same moment keep their order (seq), first line first.
363 ❸ return insertionSort(backwards(seen), function (a, b) { return b.stamp - a.stamp || a.seq - b.seq; });
364 }

```

❶ Line 348 Only orders the owner has already quoted.

❷ Line 354 Per piece, with the add-ons taken out.

❸ Line 363 Read back to front, then insertion sort newest first; lines quoted at the same moment keep their order.

Time  $O(n^2)$  · Space  $O(n)$

**js/orders.js** · lines 366–376 · recentQuotes() — The five most recent quotes for one product.

```

366  /* The five most recent history lines for the item's product – only at
367     the same size when sameSize is true. The history is newest first, so
368     a linear search that stops at five is enough. Time  $O(n)$  · Space  $O(1)$  */
369  function recentQuotes(history, item, sameSize) {
370      var out = [];
371 ❶ for (var h = 0; h < history.length && out.length < 5; h++) {
372          var line = history[h];
373 ❷ if (line.productId === item.productId && (!sameSize || line.size === item.size)) listAdd(out,
line);
374      }
375      return out;
376  }

```

❶ Line 371 Walk the history from the newest, stopping after five.

❷ Line 373 Keep lines of the same product (and size).

Time  $O(n)$  · Space  $O(1)$

**js/orders.js** · lines 378–391 · pastQuoteFrom() — What the owner charged before.

```

378  /* What the owner charged before for the same product and size, per piece:
379     the average of the five most recent quotations, or of any size when
380     that size was never quoted. Returns { materials, labor, count, sameSize }
381     per piece, or null with no history. Time  $O(n)$  · Space  $O(1)$  */
382  function pastQuoteFrom(history, item) {
383 ❶ var recent = recentQuotes(history, item, true), sameSize = recent.length > 0;
384 ❷ if (!sameSize) recent = recentQuotes(history, item, false);
385     if (recent.length === 0) return null;
386     return {
387 ❸ materials: roundMoney(sumRecursive(recent, function (r) { return r.materials; }) / recent.length),
labor: roundMoney(sumRecursive(recent, function (r) { return r.labor; }) / recent.length),
389     count: recent.length, sameSize: sameSize
390 };

```

```
391 }
```

- 1 line 383 The same size first...
- 2 line 384 ...or any size if there is none.
- 3 line 387 Average them.

Time  $O(n)$  · Space  $O(1)$

js/orders.js · lines 455–485 · sendQuote() — Sends the quotation.

```
455  /* Send (or revise) the quotation.          Time  $O(n)$  · Space  $O(n)$  */
456  function sendQuote(id, draft, stamp) {
457      var order = orderById(id);
458      if (!order) return { ok: false, errors: [{ field: 'order', message: 'No such order.' }] };
459      if (order.status !== 'requested' && order.status !== 'quoted') {
460          return { ok: false, errors: [{ field: 'order', message: order.ref + ' is past the quotation stage.'
461      }} ];
462      }
463      1 var errors = validateQuote(order, draft);
464      if (errors.length > 0) return { ok: false, errors: errors };
465
466      var priced = [];
467      for (var i = 0; i < order.items.length; i++) {
468          listAdd(priced, copyRecord(order.items[i], {
469              materials: roundMoney(draft.lines[i].materials),
470              labor: roundMoney(draft.lines[i].labor),
471              itemExpense: roundMoney(draft.lines[i].itemExpense)
472          }));
473      }
474      var revising = order.status === 'quoted';
475      order.items = priced;
476      order.deliveryFee = roundMoney(draft.deliveryFee);
477      order.rushFee = roundMoney(draft.rushFee);
478      order.quoteNote = strip(draft.note);
479      order.quotedAt = stamp;
480      if (order.delivery.service !== 'pickup') order.delivery = settledDelivery(order, order.delivery,
481      order.deliveryFee);
482      2 if (!revising) leaveQueue(quoteQueue, order.id);
483      order.status = 'quoted';
484      addHistory(order, stamp, (revising ? 'Quotation revised: ' : 'Quotation sent: ') +
485      peso(orderTotal(order)));
486      3 emailQuoteReady(order, stamp);
487      return { ok: true, order: order };
488  }
```

- 1 line 462 Every line must be priced; a courier fee must be set.
- 2 line 480 Leave the queue.
- 3 line 483 Email the customer that it is approved.

Time  $O(n)$  · Space  $O(n)$

# Payment, Receipts & Email

|         |                                                                                                                                                              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | Checkout step 3 (payment), the receipt, Track order (paying a quotation), Order desk → Outbox                                                                |
| Files   | js/orders.js (gcashProblem, receiptWithRef, placeOrder, acceptQuote, issueReceipt) · js/notify.js (queueEmail) · js/mail.js (drainMail) · js/email-config.js |
| Data    | receipts — array of snapshots; outbox — array of emails; mailQueue — circular queue of outbox positions; EMAIL_CONFIG                                        |
| DSA     | <b>Circular queue (outgoing mail)</b>                                                                                                                        |

## 1 Business process

*What task or problem from the client are you solving?*

The shop takes only two payments: a 50% down payment or full payment, both by GCash with the 13-digit reference number. Every payment needs a receipt, and the customer must be told by email: order confirmed (with the tracking number), quotation ready, payment received, ready for pickup, cancelled.

A GCash reference can only pay once. Emails must go out in the order they were written, and none may be lost if the connection drops.

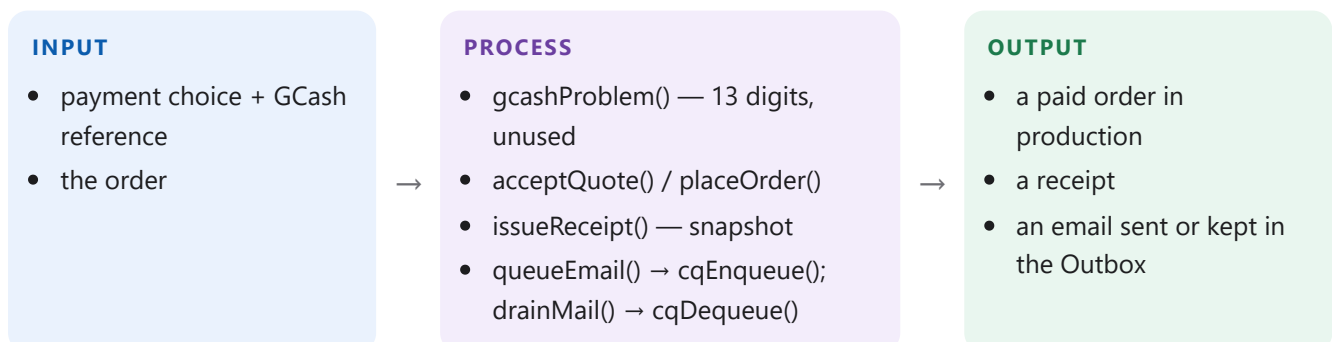
## 2 Input

*What data is needed by the module?*

| Input                  | What it is                                                              |
|------------------------|-------------------------------------------------------------------------|
| Customer picks / types | 50% or 100%; the GCash reference number.                                |
| Stored: the order      | Lines, totals, customer email.                                          |
| Stored: receipts       | Every reference already used.                                           |
| Stored: EMAIL_CONFIG   | The EmailJS IDs (blank = emails kept in the Outbox, marked "not sent"). |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Circular queue (outgoing mail)):

- 1 Check the reference: 13 digits, and not already on any receipt (a linear search).
- 2 Price-list cart: place the order and take the payment in one step; if anything fails, no order is left behind.

- 3 Record the payment; take a snapshot of the order into a numbered receipt (OR-0001...), so later edits never change it.
- 4 Write the email, store it in the outbox array and put its position in a circular queue. Every 1.5 s the sender takes the front of the queue and posts it to EmailJS, one at a time, marking it sent or failed.

## 4 Output

*What result does the module produce?*

- The receipt (printable) with paid and outstanding amounts.
- The "Order confirmed" screen with the tracking number.
- Emails to the customer, and the Outbox tab listing each one.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation      | Time                                    | Space  | Note                                                       |
|----------------|-----------------------------------------|--------|------------------------------------------------------------|
| gcashProblem() | $O(n)$                                  | $O(n)$ | The checks every payment shares.                           |
| placeOrder()   | $O(n^2)$                                | $O(n)$ | Checkout for a price-list cart: place and pay in one step. |
| acceptQuote()  | $O(n)$                                  | $O(n)$ | Records the payment.                                       |
| issueReceipt() | $O(n)$                                  | $O(n)$ | Snapshots the order into a receipt.                        |
| queueEmail()   | $O(n)$                                  | $O(n)$ | Writes an email and queues it.                             |
| drainMail()    | $O(n^2)$ – each send redraws the outbox | $O(1)$ | Sends the queue, one at a time.                            |

**What n means here:** For the reference check,  $n$  = receipts so far. For the queue,  $n$  = emails waiting. For a receipt,  $n$  = the order's lines.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Emails must leave in the order they were written ("confirmed" before "ready"), and only one may be in flight at a time — a first-in, first-out queue is exactly that. The circular buffer keeps enqueue and dequeue  $O(1)$ .

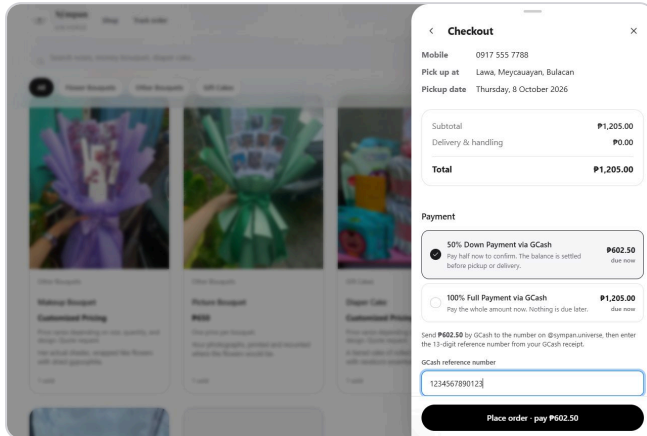
Receipts are snapshots (copies), not live views of the order, because a receipt records what was paid at that moment.

Checking the reference against every receipt is a linear search,  $O(n)$ ; with a few hundred receipts a year that is instant and needs no extra index.

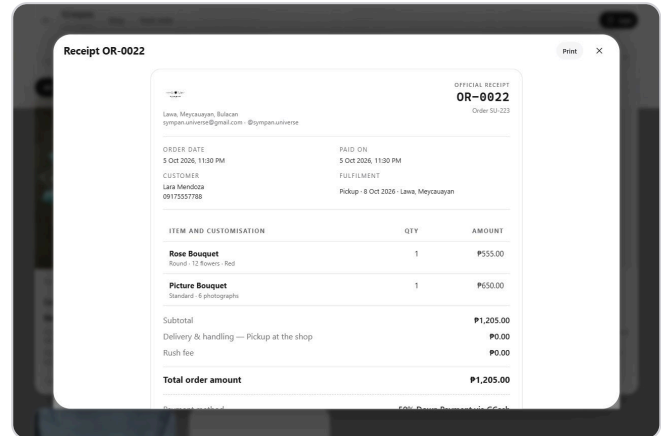
## 7 Demonstration

*Show that the module works correctly — the input, the process and the output.*

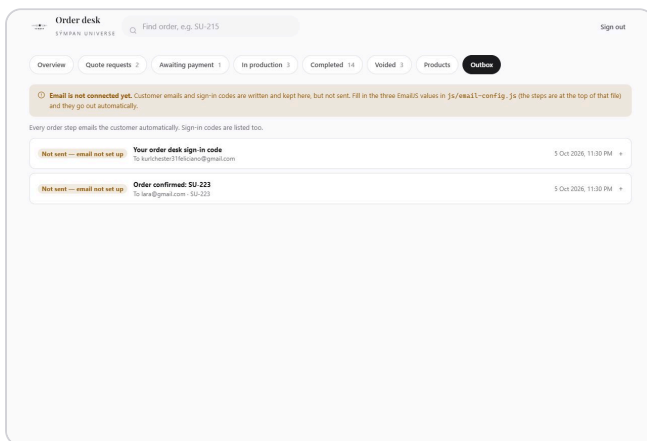
- 1 Add a Rose Bouquet, check out with pickup, choose "100% Full Payment via GCash", enter 1234567890123 and place the order.
- 2 The receipt opens; the confirmation shows the tracking number.
- 3 Try to pay another order with the same reference: refused.
- 4 In the order desk, open "Outbox": the "Order confirmed" email is there.



Checkout: the two GCash payments



The receipt



The Outbox

## Trace — a payment, its receipt and its email (computed by our code)

### placeOrder() with a 5-digit reference

Refused before anything is created: "Enter the 13-digit GCash reference number.". Orders still: 22.

### placeOrder() with "1234 567 890123", 50% down payment

| Receipt field        | Value                                                  |
|----------------------|--------------------------------------------------------|
| Receipt no.          | OR-0022                                                |
| Order (tracking) no. | SU-223                                                 |
| Customer             | Lara Mendoza, 09175557788, lara@gmail.com              |
| Items                | Rose Bouquet × 1 — Round, 12 flowers, Red, Card · Rose |
| Subtotal             | ₱575.00                                                |
| Delivery & handling  | ₱0.00                                                  |
| Total                | ₱575.00                                                |
| Method               | 50% Down Payment via GCash                             |
| Amount paid          | ₱287.50                                                |
| Remaining balance    | ₱287.50                                                |
| Status               | Partially paid                                         |

Order SU-223 is now "In production".

### payBalance() with the same reference

"That GCash reference is already on receipt OR-0022. Check the number."

## queueEmail() — the confirmation joins the mail queue

outbox[0]: to lara@gmail.com, subject "Order confirmed: SU-223", status queued.



capacity = 8 · count = 1 · head = 0 · tail = 1

Hi Lara Mendoza,

Your order is confirmed and we are making it now.

- Rose Bouquet × 1 (Round, 12 flowers, Red, Card · Rose) – ₱575.00

Total: ₱575.00

Down payment: ₱287.50 by GCash (ref 1234567890123)

Remaining balance: ₱287.50 – due before pickup

Receipt: OR-0022

Pickup: Thursday, 8 October 2026

Tracking number: SU-223

Keep this number. Quote it when you message us, or use Track order on our site with your email or mobile number.

Thank you,

Sýmpan Universe

Lawa, Meycauayan, Bulacan · @sympan.universe

## drainMail() with EmailJS not set up

Dequeued; status is now "simulated" — kept in the Outbox, not sent. Queue count: 0.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/orders.js · lines 588–594 · gcashProblem() — The checks every payment shares.

```
588  /* The checks every GCash payment shares: '' when it may go ahead.
589                                     Time O(n) · Space O(n) */
590  function gcashProblem(gcashRef) {
591  ❶    if (!isValidGcashRef(gcashRef)) return 'Enter the 13-digit GCash reference number.';
592  ❷    var used = receiptWithRef(gcashRef);
593      return used ? 'That GCash reference is already on receipt ' + used.no + '. Check the number.' : '';
594  }
```

❶ line 591 Exactly 13 digits.

❷ line 592 Not already used on a receipt.

Time O(n) · Space O(n)

js/orders.js · lines 287–309 · placeOrder() — Checkout for a price-list cart: place and pay in one step.

```
287  /* Checkout for a priced cart: the order is placed and paid in one step, so
288     nothing waits on the owner. Nothing is created unless the payment
289     details are valid.                               Time O(n²) · Space O(n) */
290  function placeOrder(form, methodId, gcashRef, stamp) {
291      var items = repricedCart();
292      ❶ var errors = validateRequest(form, items, isoFromStamp(stamp));
293      if (errors.length > 0) return { ok: false, errors: errors };
294      var preview = deliveryQuote(form);
295      if (needsQuote(items, preview)) return { ok: false, errors: [{ field: 'basket', message: 'This cart
needs a quote from the shop first.' }] };
296      if (!paymentById(methodId)) return { ok: false, errors: [{ field: 'payment', message: 'Choose 50% down
payment or full payment.' }] };
297      ❷ var problem = gcashProblem(gcashRef);
298      if (problem) return { ok: false, errors: [{ field: 'gcash', message: problem }] };
299
300      ❸ var made = submitRequest(form, stamp);
301      if (!made.ok) return made;
302      ❹ var paid = acceptQuote(made.order.id, methodId, gcashRef, stamp);
303      if (!paid.ok) {
304          // Should never happen after the checks above; never leave a half-made order.
305      ❺ voidOrder(made.order.id, 'Payment could not be recorded', 'system', stamp);
306          return { ok: false, errors: [{ field: 'gcash', message: paid.error }] };
307      }
308      return { ok: true, order: made.order, receipt: paid.receipt };
309  }
```

- ❶ line 292 Check the form first.
- ❷ line 297 Check the payment before anything is created.
- ❸ line 300 Create the order (priced automatically).
- ❹ line 302 Take the payment.
- ❺ line 305 Never leave a half-made order.

Time  $O(n^2)$  · Space  $O(n)$

js/orders.js · lines 596–619 · acceptQuote() — Records the payment.

```
596  /* The customer accepts the quotation and pays. Time O(n) · Space O(n) */
597  function acceptQuote(id, methodId, gcashRef, stamp) {
598      var order = orderById(id);
599      if (!order || order.status !== 'quoted') return { ok: false, error: 'This order has no quotation
waiting for payment.' };
600      ❶ if (stamp - order.quotedAt > QUOTE_EXPIRY_DAYS * 24 * 3600000) {
601          voidOrder(order.id, 'Quotation expired - not paid within ' + QUOTE_EXPIRY_DAYS + ' days', 'system',
stamp);
602          return { ok: false, error: 'This quotation has expired. Send a new request and the shop will quote
it again.' };
603      }
604      var method = paymentById(methodId);
605      if (!method) return { ok: false, error: 'Choose 50% down payment or full payment.' };
606      var problem = gcashProblem(gcashRef);
607      if (problem) return { ok: false, error: problem };
608      if (orderTotal(order) <= 0) return { ok: false, error: 'This quotation has no amount to pay.' };
609
610      var total = orderTotal(order);
611      order.paymentMethod = method.id;
612      ❷ order.amountPaid = method.share === 1 ? total : roundMoney(total * method.share);
613      order.status = 'paid';
614      ❸ enterProduction(order);
615      addHistory(order, stamp, 'Quotation accepted - ' + peso(order.amountPaid) + ' paid by GCash');
616      ❹ var receipt = issueReceipt(order, method.share === 1 ? 'Full payment' : 'Down payment',
order.amountPaid, gcashRef, stamp);
```

```

617 5 emailOrderConfirmed(order, receipt, stamp);
618     return { ok: true, order: order, receipt: receipt };
619 }

```

- ❶ Line 600 An expired quotation cannot be paid.
- ❷ Line 612 Full or half of the total.
- ❸ Line 614 Into the production line (module 10).
- ❹ Line 616 Issue the receipt...
- ❺ Line 617 ...and the confirmation email.

Time  $O(n)$  · Space  $O(n)$

**js/orders.js** · lines 531–564 · `issueReceipt()` — Snapshots the order into a receipt.

```

531  /* Snapshot the order as it stands at the moment money changes hands, so a
532     receipt never shifts when the order is edited later.
533                                     Time  $O(n)$  · Space  $O(n)$  */
534  function issueReceipt(order, kind, amount, gcashRef, stamp) {
535      var lines = [];
536      for (var i = 0; i < order.items.length; i++) {
537          var item = order.items[i];
538          listAdd(lines, { name: item.productName, quantity: item.quantity, specs: itemSpecs(item),
539                          notes: item.notes, amount: lineTotal(item) });
540      }
541      var method = paymentById(order.paymentMethod);
542      // A balance is paid by GCash after the original payment, so say so
543      // rather than repeating "100% Full Payment" on a top-up receipt.
544      var methodName = kind === 'Balance payment'
545          ? 'GCash - balance after ' + (method ? toLower(method.short) : 'down payment')
546          : (method ? method.name : 'GCash');
547      var receipt = {
548 ❶      no: 'OR-' + leftPad(counters.receipt, 4, '0'),
549          orderId: order.id, ref: order.ref, stamp: stamp, orderDate: order.createdAt,
550          kind: kind, method: methodName, gcashRef: digitsOnly(gcashRef),
551          amount: roundMoney(amount),
552          customer: copyRecord(order.customer),
553          fulfilment: fulfilmentSummary(order),
554          items: lines,
555          subtotal: orderSubtotal(order), deliveryFee: order.deliveryFee,
556          deliveryLabel: order.delivery.label, rushFee: order.rushFee,
557          total: orderTotal(order), paidToDate: order.amountPaid,
558 ❷      balance: orderBalance(order), status: paymentStatus(order)
559      };
560      counters.receipt++;
561      listAdd(receipts, receipt);
562      listAdd(order.receipts, receipt);
563      return receipt;
564  }

```

- ❶ Line 548 The next receipt number.
- ❷ Line 558 Paid to date, balance and status, as they are now.

Time  $O(n)$  · Space  $O(n)$

**js/notify.js** · lines 12–26 · `queueEmail()` — Writes an email and queues it.

```

12  /* Record an email and queue it for sending. Returns the record, or null
13     while mail is off.                                     Time  $O(n)$  · Space  $O(n)$  */
14  function queueEmail(to, subject, lines, kind, orderId, stamp) {
15      // The demo customers live on example.com, a domain that can never
16      // receive mail: their messages are not even queued.
17 ❶  if (!mailEnabled || !isValidEmail(to) || textHas(toLower(to), '@example.com')) return null;

```

```

18     var record = {
19         id: counters.mail, to: toLower(strip(to)), subject: subject, body: glue(lines, '\n'),
20         kind: kind, orderId: orderId, stamp: stamp, status: 'queued', note: ''
21     };
22     counters.mail++;
23 ❷ listAdd(outbox, record);
24 ❸ cqEnqueue(mailQueue, outbox.length - 1);
25     return record;
26 }

```

- ❶ Line 17 Never email the demo customers.
- ❷ Line 23 Keep it in the outbox...
- ❸ Line 24 ...and queue its position for sending.

Time  $O(n)$  · Space  $O(n)$

js/mail.js · lines 20–60 · drainMail() — Sends the queue, one at a time.

```

20     /* Send the next queued email, then the next, one at a time.
21                                     Time  $O(n^2)$  – each send redraws the outbox · Space  $O(1)$  */
22     function drainMail() {
23         if (mailSending || cqIsEmpty(mailQueue)) return;
24 ❶ var record = outbox[cqDequeue(mailQueue)];
25         if (!mailConfigured()) {
26 ❷     record.status = 'simulated';
27         record.note = 'Email is not set up yet (js/email-config.js).';
28         drainMail();
29         return;
30     }
31     mailSending = true;
32     record.status = 'sending';
33 ❸ fetch(EMAILJS_URL, {
34         method: 'POST',
35         headers: { 'Content-Type': 'application/json' },
36         body: JSON.stringify({
37             service_id: EMAIL_CONFIG.serviceId,
38             template_id: EMAIL_CONFIG.templateId,
39             user_id: EMAIL_CONFIG.publicKey,
40             template_params: {
41                 to_email: record.to, subject: record.subject, message: record.body,
42                 from_name: EMAIL_CONFIG.fromName, reply_to: EMAIL_CONFIG.replyTo
43             }
44         })
45     }).then(function (response) {
46 ❹     record.status = response.ok ? 'sent' : 'failed';
47     record.note = response.ok ? '' : 'EmailJS answered ' + response.status;
48     return response.ok ? '' : response.text();
49 }).then(function (detail) {
50     if (detail) record.note = record.note + ': ' + detail;
51 }, function () {
52     record.status = 'failed';
53     record.note = 'No connection to EmailJS.';
54 }).then(function () {
55     mailSending = false;
56     if (typeof onMailChanged === 'function') onMailChanged();
57     if (typeof onSignInMailChanged === 'function') onSignInMailChanged();
58     drainMail();
59 });
60 }

```

- ❶ Line 24 Take the front of the queue.
- ❷ Line 26 EmailJS not set up: keep it, marked "not sent".
- ❸ Line 33 Otherwise post it to EmailJS.

4 line 46 Mark the result; then send the next one.

Time  $O(n^2)$  – each send redraws the outbox · Space  $O(1)$

## PART 4

# Behind the counter

Module 10 Production & Tracking · Module 11 Order Records · Module 12 Order Desk

## 60-second pitch

These modules are the owner's side. Production & Tracking keeps rush orders in a min-heap by due date and regular orders in a circular queue by payment order, and finds any order for a customer with a binary search on the order number. Order Records keeps every completed and voided order, sorted by date with insertion sort, and undoes "Mark ready" with a stack. The Order Desk signs the owner in with a hashed password and an emailed six-digit code, and its Overview puts every order and receipt into its day, month or year by reading the record's own date.

## Structures and algorithms to be able to explain

| Where                    | Structure / algorithm              | Time                                   | Space  |
|--------------------------|------------------------------------|----------------------------------------|--------|
| Rush lane                | Min-heap keyed (due date, arrival) | $O(\log n)$ insert / extract           | $O(n)$ |
| Standard lane            | Circular queue (FIFO)              | $O(1)$                                 | $O(n)$ |
| Track an order           | Binary search on the order number  | $O(\log n)$                            | $O(1)$ |
| Undo "Mark ready"        | Stack (LIFO)                       | $O(1)$ push / pop                      | $O(n)$ |
| Completed orders by date | Insertion sort + one grouping pass | $O(n)$ nearly in order, $O(n^2)$ worst | $O(n)$ |
| Sign-in                  | FNV-1a hash compare                | $O(n)$ in the password length          | $O(1)$ |
| Overview by period       | Bucketing by the record's date     | $O(n^2)$ – the lines of every order    | $O(n)$ |

## Likely panel questions

### Why a heap for rush orders and not a sorted array?

Inserting into a sorted array shifts items:  $O(n)$ . A heap inserts, and removes the earliest due date, in  $O(\log n)$  by sifting along one path of the tree.

### How is a tie on the due date broken?

The key is (due date, arrival number). `heapCompare` compares dates first, then arrival, so equal dates are served in the order they were paid — the heap is stable.

### Why is the orders array always sorted?

Order numbers come from a counter that only goes up, and new orders are appended, so the array stays in number order without sorting — which is what makes binary search possible.

### What is the worst case of your binary search?

About  $\log_2 n$  comparisons: 5 for the 22 demo orders, 10 for 1,000 orders. A linear search could need all  $n$ .

**How does undo put an order back exactly?**

Each "Mark ready" pushes { order id, its place in line }. Undo pops the newest entry and puts the order back: into the heap with its original key, or into the queue at its old position.

**Is the two-step sign-in real security?**

It keeps customers off the desk: only hashes of the password and the code are kept, codes expire after five minutes and allow three tries, and five wrong passwords lock the desk for 30 seconds. Real security needs a server — anything in a browser can be read with developer tools.

# Production & Tracking

|         |                                                                                                                                                                                                    |
|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | Order desk → "In production" tab and "Mark ready"; Track order on the shop                                                                                                                         |
| Files   | js/structures.js (heapInsert, heapSiftUp, heapExtractMin, heapCompare) · js/orders.js (enterProduction, productionLine, completeNext, orderById, customerLookup) · js/algorithms.js (binarySearch) |
| Data    | rushLane — min-heap keyed (due date, arrival); standardLane — circular queue; orders — array sorted by order number                                                                                |
| DSA     | <b>Min-heap · Circular queue · Binary search</b>                                                                                                                                                   |

## 1 Business process

*What task or problem from the client are you solving?*

Paid orders must be made in the right order. Rush orders (₱50 extra, ready next day) go first, earliest due date first. Regular orders are made in the order they were paid. An order still owing a balance cannot be released, but it must not hold up the paid orders behind it.

Customers check their order with the tracking number (e.g. SU-215) and their mobile number or email, so the shop needs to find an order by number quickly.

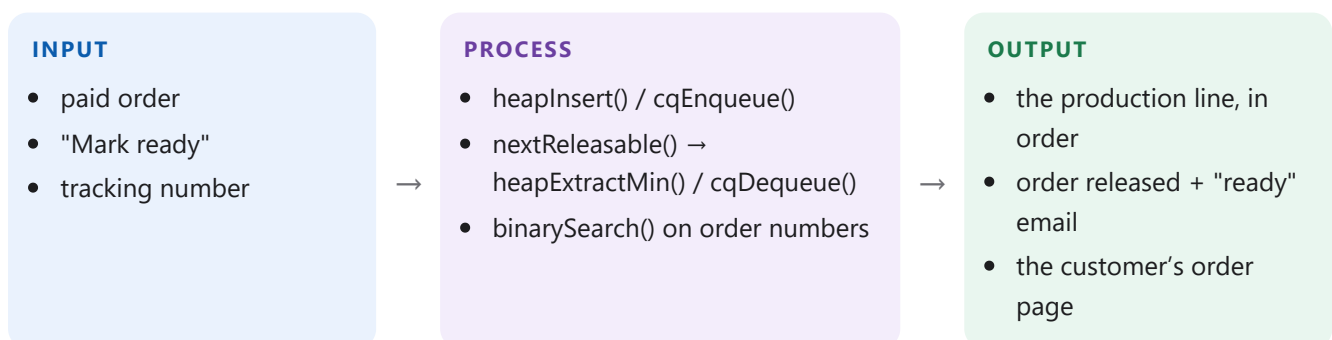
## 2 Input

*What data is needed by the module?*

| Input                                         | What it is                                  |
|-----------------------------------------------|---------------------------------------------|
| Stored: rushLane (heap), standardLane (queue) | Order ids in production.                    |
| Stored: orders                                | In ascending order-number order.            |
| Customer types                                | Tracking number and mobile number or email. |
| Owner presses                                 | "Mark ready".                               |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Min-heap · Circular queue · Binary search):

- 1 Paid rush order: insert it into a min-heap keyed by (due date, arrival number) — it sifts up past later dates. Paid standard order: enqueue it.

- 2 Mark ready: walk the line (heap order, then queue order) to the first order that is fully paid; take it out (the heap minimum or the queue front in the usual case) and email the customer. Listing the line in heap order sorts a copy of the heap's array by the same key with insertion sort; the heap itself is untouched.
- 3 Tracking: orders are numbered in increasing order and only ever appended, so the array is already sorted — binary search halves the range at each step.

## 4 Output

*What result does the module produce?*

- The "In production" tab with positions, rush badges and the "Mark ready" button.
- The customer's tracking page with the timeline.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation        | Time                            | Space  | Note                                 |
|------------------|---------------------------------|--------|--------------------------------------|
| heapInsert()     | $O(\log n)$                     | $O(1)$ | Adds an order to the rush lane.      |
| heapSiftUp()     | $O(\log n)$                     | $O(1)$ | Restores the heap upward.            |
| heapExtractMin() | $O(\log n)$                     | $O(1)$ | Removes the earliest-due order.      |
| heapCompare()    | $O(1)$                          | $O(1)$ | Which order goes first.              |
| completeNext()   | $O(n^2)$                        | $O(n)$ | "Mark ready".                        |
| binarySearch()   | $O(\log n)$                     | $O(1)$ | Finds an order by number.            |
| customerLookup() | $O(\log n) + O(n)$ for the text | $O(n)$ | Shows an order only to its customer. |

**What n means here:** For the heap and the queue,  $n$  = orders in production. For binary search,  $n$  = all orders.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

A min-heap always gives the earliest due date, in  $O(\log n)$  per insert or removal — keeping a sorted list instead would cost  $O(n)$  per insert. Adding the arrival number to the key makes it stable: equal dates are served in the order they were paid.

Standard orders are fair first-in, first-out: a circular queue,  $O(1)$  at both ends.

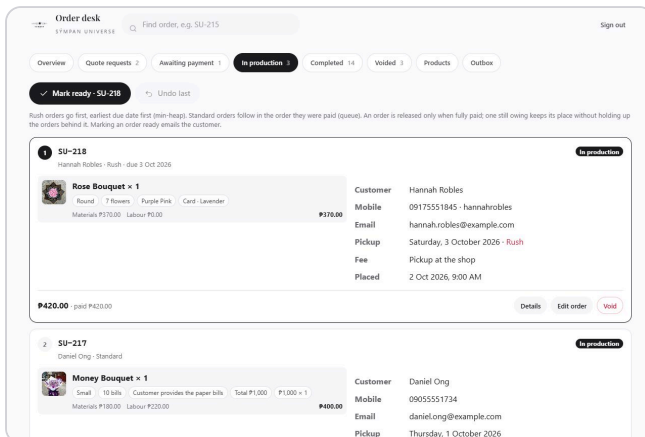
Binary search finds an order among  $n$  in  $O(\log n)$  — about 10 steps for 1,000 orders — and costs nothing extra because order numbers are handed out in increasing order.

## 7 Demonstration

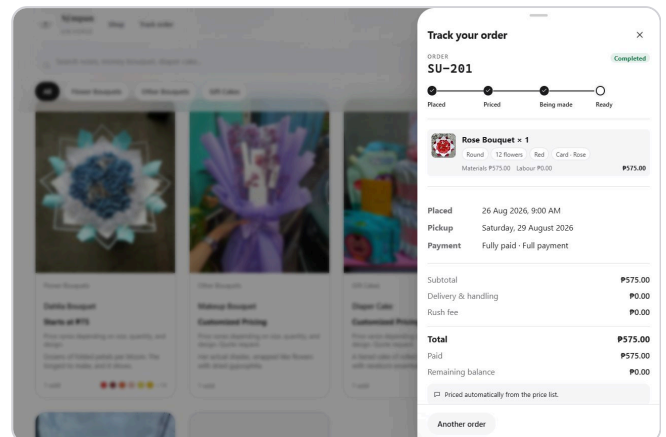
*Show that the module works correctly — the input, the process and the output.*

- 1 In the order desk, open "In production": rush orders first, then standard.

- 2 Press "Mark ready": the first fully paid order is released and the customer emailed; an order owing a balance keeps its place.
- 3 On the shop, press "Track order" and enter SU-201 with 0917 555 0142.



The production line



Tracking an order

## Trace — the rush heap and the order search (computed by our code)

### The rush lane after the demo history

[0]  
**SU-218 Hannah**  
key (20261003, 1)

### heapInsert() for SU-223, due 2026-10-10 (key 20261010)

[0]  
**SU-218 Hannah**  
key (20261003, 1)

[1]  
**SU-223 Lara**  
key (20261010, 2)

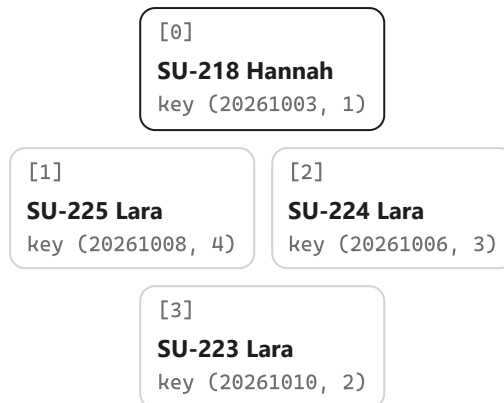
### heapInsert() for SU-224, due 2026-10-06 (key 20261006)

[0]  
**SU-218 Hannah**  
key (20261003, 1)

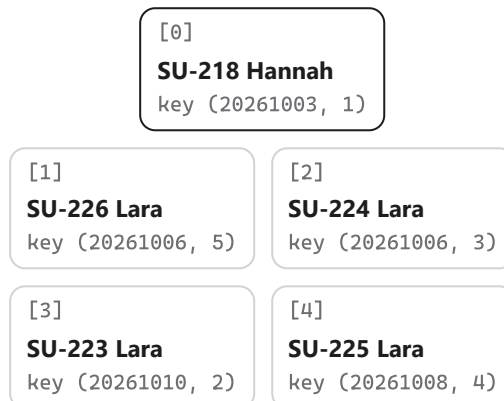
[1]  
**SU-223 Lara**  
key (20261010, 2)

[2]  
**SU-224 Lara**  
key (20261006, 3)

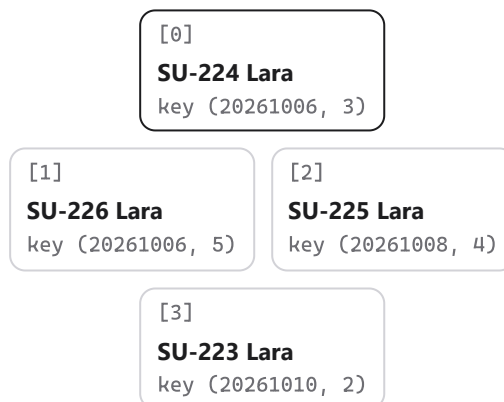
**heapInsert() for SU-225, due 2026-10-08 (key 20261008)**



**heapInsert() for SU-226, due 2026-10-06 (key 20261006)**



**completeNext() — "Mark ready" takes SU-218, the earliest due date; the last leaf sinks from the root**



Equal dates are served in payment order, because the key's second part is the arrival number.

**binarySearch(orders, 215) — the order numbers looked at**

213 → 220 → 216 → 214 → 215 ⇒ index 14 (SU-215) in 5 steps among 26 orders; a linear scan would need 15.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

**js/structures.js** · lines 272–281 · **heapInsert()** — Adds an order to the rush lane.

```
272  /* Insert with a priority. `seq` may be given to restore an item at its
273     exact former key.                               Time O(log n) · Space O(1) */
274  function heapInsert(heap, value, priority, seq) {
```

```

275     var arrival = seq === undefined ? heap.nextSeq : seq;
276     if (arrival >= heap.nextSeq) heap.nextSeq = arrival + 1;
277 ❶ heap.items[heap.size] = { value: value, priority: priority === undefined ? 1 : priority, seq: arrival
    };
278     heap.size++;
279 ❷ heapSiftUp(heap, heap.size - 1);
280     return heap.size;
281 }

```

❶ Line 277 Put it at the end of the array (the next leaf).

❷ Line 279 Sift it up while it beats its parent.

Time  $O(\log n)$  · Space  $O(1)$

**js/structures.js** · lines 248–257 · `heapSiftUp()` — Restores the heap upward.

```

248  /* Bubble a node up while it beats its parent. Time  $O(\log n)$  · Space  $O(1)$  */
249  function heapSiftUp(heap, index) {
250      var child = index;
251      while (child > 0) {
252 ❶     var parent = Math.floor((child - 1) / 2);
253 ❷     if (heapCompare(heap.items[child], heap.items[parent]) >= 0) break;
254 ❸     heapSwap(heap, child, parent);
255         child = parent;
256     }
257 }

```

❶ Line 252 The parent of index  $i$  is  $(i - 1) \div 2$ .

❷ Line 253 Stop once the parent is not later.

❸ Line 254 Otherwise swap and continue upward.

Time  $O(\log n)$  · Space  $O(1)$

**js/structures.js** · lines 283–292 · `heapExtractMin()` — Removes the earliest-due order.

```

283  /* Remove and return the smallest key's value. Time  $O(\log n)$  · Space  $O(1)$  */
284  function heapExtractMin(heap) {
285      if (heap.size === 0) return null;
286 ❶     var top = heap.items[0];
287         heap.size--;
288 ❷     heap.items[0] = heap.items[heap.size];
289         heap.items.length = heap.size;
290 ❸     if (heap.size > 0) heapSiftDown(heap, 0);
291         return top.value;
292 }

```

❶ Line 286 The minimum is always at the root.

❷ Line 288 Move the last leaf to the root...

❸ Line 290 ...and sink it to its place.

Time  $O(\log n)$  · Space  $O(1)$

**js/structures.js** · lines 235–239 · `heapCompare()` — Which order goes first.

```

235  /* Negative when a must leave before b. Time  $O(1)$  · Space  $O(1)$  */
236  function heapCompare(a, b) {
237 ❶     if (a.priority !== b.priority) return a.priority - b.priority;
238 ❷     return a.seq - b.seq;
239 }

```

❶ Line 237 Earlier due date first...

2 line 238 ...then whoever paid first.

Time  $O(1)$  · Space  $O(1)$

js/orders.js · lines 689–719 · completeNext() — "Mark ready".

```
689  /* Release the next fully paid order. Records where it stood, so undo can
690     put it back exactly. Finding it walks the line,  $O(n^2)$ ; taking it
691     from the front is  $O(1)$  for the queue,  $O(\log n)$  for the heap.
692                                     Time  $O(n^2)$  · Space  $O(n)$  */
693  function completeNext(stamp) {
694      var line = productionLine();
695      if (line.length === 0) return { ok: false, error: 'Nothing is in production.' };
696  1   var order = nextReleasable();
697      if (!order) {
698          var owing = renderEach(line, function (o) { return o.ref; }, ', ');
699          return { ok: false, error: 'Every order in production still owes a balance (' + owing + '). Record
a balance payment first.' };
700      }
701      var skipped = [];
702      for (var i = 0; i < line.length && line[i] !== order; i++) listAdd(skipped, line[i]);
703      var position = -1;
704      if (order.lane === 'rush') {
705          // Rush comes first, so a releasable rush order is the heap's minimum
706          // unless one ahead of it still owes.
707  2   if (heapPeek(rushLane) === order.id) heapExtractMin(rushLane);
708          else heapRemove(rushLane, order.id);
709      } else {
710          position = positionIn(cqValues(standardLane), order.id);
711  3   leaveQueue(standardLane, order.id);
712      }
713      order.status = 'completed';
714      order.completedAt = stamp;
715  4   stackPush(completedStack, { id: order.id, position: position });
716      addHistory(order, stamp, 'Ready and released');
717  5   emailOrderReady(order, stamp);
718      return { ok: true, order: order, skipped: skipped };
719  }
```

1 line 696 The first fully paid order in line.

2 line 707 Rush: usually the heap minimum.

3 line 711 Standard: usually the queue front.

4 line 715 Remember it for undo (module 11).

5 line 717 Email "ready for pickup / on its way".

Time  $O(n^2)$  · Space  $O(n)$

js/algorithms.js · lines 84–98 · binarySearch() — Finds an order by number.

```
84  /* Binary search over an array sorted ascending by keyOf. Order numbers are
85     handed out in increasing sequence and orders are only ever appended, so
86     the order list is sorted for free. Returns the index or -1.
87                                     Time  $O(\log n)$  · Space  $O(1)$  */
88  function binarySearch(sorted, target, keyOf) {
89      var low = 0, high = sorted.length - 1;
90      while (low <= high) {
91  1   var mid = Math.floor((low + high) / 2);
92      var key = keyOf(sorted[mid]);
93  2   if (key === target) return mid;
94  3   if (key < target) low = mid + 1;
95  4   else high = mid - 1;
96      }
97      return -1;
}
```

- ❶ line 91 Look at the middle.
- ❷ line 93 Found.
- ❸ line 94 Too small: search the right half...
- ❹ line 95 ...too big: search the left half.

Time  $O(\log n)$  · Space  $O(1)$

**js/orders.js** · lines 65–73 · `customerLookup()` — Shows an order only to its customer.

```

65  /* A customer sees an order only with its tracking number and the mobile
66     number or email used on it.           Time  $O(\log n) + O(n)$  for the text · Space  $O(n)$  */
67  function customerLookup(ref, contact) {
68  ❶    var order = orderByRef(ref);
69      if (!order || isBlank(contact)) return null;
70      var byEmail = order.customer.email && toLower(strip(contact)) === order.customer.email;
71      var byPhone = digitsOnly(contact).length > 0 && normalisePhone(order.customer.phone) ===
        normalisePhone(contact);
72  ❷    return byEmail || byPhone ? order : null;
73  }
```

- ❶ line 68 Binary search by number.
- ❷ line 72 Only if the mobile number or email matches.

Time  $O(\log n) + O(n)$  for the text · Space  $O(n)$

# Order Records

|         |                                                                                                                                |
|---------|--------------------------------------------------------------------------------------------------------------------------------|
| Screens | Order desk → "Completed" and "Voided" tabs; "Undo last" in production                                                          |
| Files   | js/structures.js (stackPush, stackPop) · js/orders.js (completeNext, undoCompletion, completedByDate, voidOrder, expireQuotes) |
| Data    | completedStack — stack of { id, position }; orders (completion and void times, reasons)                                        |
| DSA     | <b>Stack · Insertion sort</b>                                                                                                  |

## 1 Business process

*What task or problem from the client are you solving?*

Every finished order must be kept with all its details, organised by date. Cancelled orders are kept too, marked "VOIDED – NON-REFUNDABLE", never deleted. Quotations the customer never pays are cancelled automatically after three days.

People make mistakes: if the owner marks the wrong order ready, they need to undo it — and the order must go back exactly where it was.

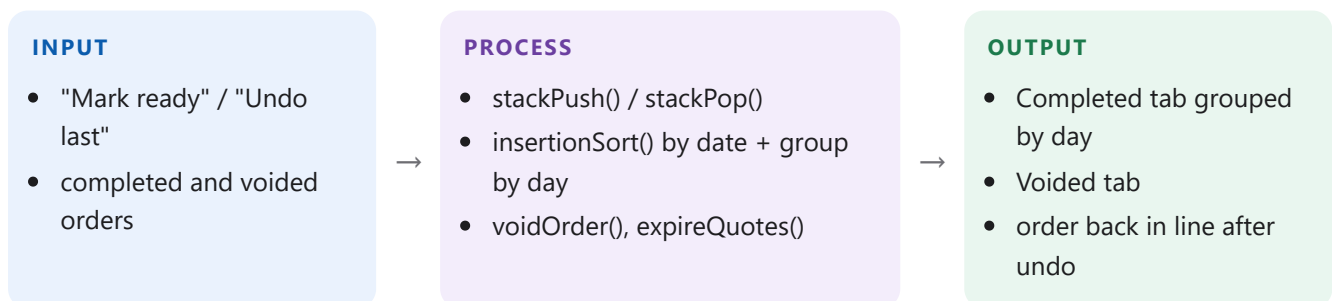
## 2 Input

*What data is needed by the module?*

| Input                  | What it is                                        |
|------------------------|---------------------------------------------------|
| Stored: orders         | Completion and void times, reasons, payments.     |
| Stored: completedStack | The completions made this session, newest on top. |
| Owner presses          | "Undo last"; "Void" with a reason.                |
| The clock              | Every minute, for expiring quotations.            |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Stack · Insertion sort):

- 1 Each "Mark ready" pushes { order, its place in line } onto a stack.
- 2 Undo pops the top — the most recent completion — and puts the order back: into the heap with its original key, or into the queue at its old position.

- 3 Completed orders: read the list back to front (it is kept oldest first), insertion sort it by completion time, newest first, then group consecutive orders of the same day with a day total.
- 4 Voiding takes the order out of its queue or lane, keeps the record and any payment, and emails the customer. Every minute, quotations older than three days are voided by "System".

## 4 Output

*What result does the module produce?*

- The "Completed" tab (days with totals; each order opens in full).
- The "Voided" tab with reason, who and when.
- The cancellation email.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                      | Time                                          | Space  | Note                           |
|--------------------------------|-----------------------------------------------|--------|--------------------------------|
| <code>stackPush()</code>       | $O(1)$                                        | $O(1)$ | Pushes onto the stack.         |
| <code>stackPop()</code>        | $O(1)$                                        | $O(1)$ | Pops the top.                  |
| <code>undoCompletion()</code>  | $O(n)$ worst                                  | $O(n)$ | Undo the last "Mark ready".    |
| <code>completedByDate()</code> | $O(n)$ here (nearly in order), $O(n^2)$ worst | $O(n)$ | Completed orders by day.       |
| <code>voidOrder()</code>       | $O(n)$                                        | $O(n)$ | Cancels, but keeps the record. |
| <code>expireQuotes()</code>    | $O(n^2)$                                      | $O(n)$ | The automatic clean-up.        |

**What n means here:** n = number of orders (completed or voided); for the stack, n = completions this session.

Each Time and Space is read from the comment above the function in the source, where every function in js/ states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Undo is last in, first out by nature — you undo the most recent action first — which is exactly a stack,  $O(1)$  push and pop.

Storing the position with each completion means undo restores the order exactly, not just "somewhere in production".

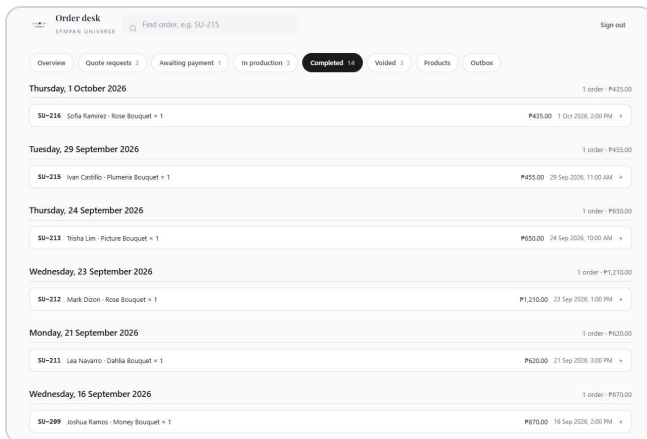
Insertion sort gives a stable date order. Orders are completed mostly in the order they were placed, so reading the oldest-first array back to front leaves it nearly newest first already: the sort runs close to its  $O(n)$  best case (15 comparisons for 15 orders, instead of the worst case of 105). Grouping consecutive same-day orders is then a single  $O(n)$  pass.

## 7 Demonstration

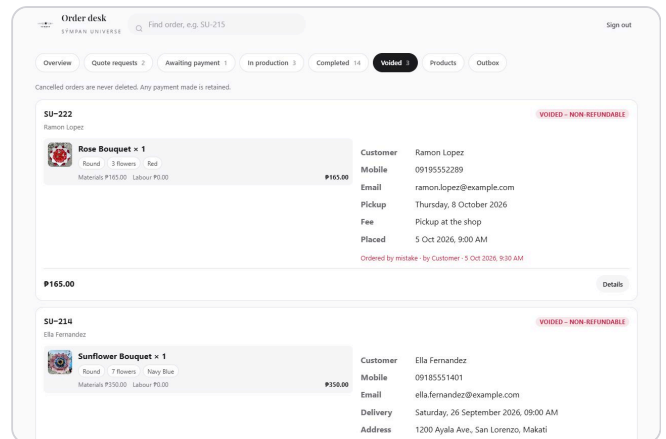
*Show that the module works correctly — the input, the process and the output.*

- 1 In "In production" press "Mark ready", then "Undo last": the order is back at its place.
- 2 Open "Completed": orders grouped by day, newest first, each with a day total.

### 3 Void an order awaiting payment: it moves to "Voided" as VOIDED – NON-REFUNDABLE.



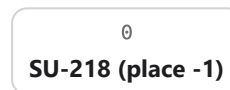
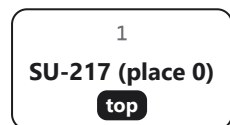
Completed orders grouped by day



Voided orders kept on record

## Trace — undo and the records by date (computed by our code)

Two "Mark ready": SU-218 then SU-217 are pushed



undoCompletion() pops SU-217 (last in, first out) and puts it back in its lane



SU-217 is "In production" again.

completedByDate() — insertion sort newest first, then grouped by day (first six days)

| Day                          | Orders | Day total |
|------------------------------|--------|-----------|
| Monday, 5 October 2026       | SU-218 | ₱420.00   |
| Thursday, 1 October 2026     | SU-216 | ₱435.00   |
| Tuesday, 29 September 2026   | SU-215 | ₱455.00   |
| Thursday, 24 September 2026  | SU-213 | ₱650.00   |
| Wednesday, 23 September 2026 | SU-212 | ₱1,210.00 |
| Monday, 21 September 2026    | SU-211 | ₱620.00   |

insertionSort made 15 comparisons for the 15 completed orders. They are read back to front first, so the list is already nearly newest first — close to the best case,  $n - 1 = 14$ . Sorting the oldest-first array directly would be the worst case,  $n(n - 1)/2 = 105$ .

### voidedOrders() — kept on record as VOIDED – NON-REFUNDABLE

| Order  | Reason                                      | By         | Retained |
|--------|---------------------------------------------|------------|----------|
| SU-222 | Ordered by mistake                          | Customer   | ₱0.00    |
| SU-214 | Customer cancelled after production started | Order desk | ₱235.00  |
| SU-210 | Found another gift                          | Customer   | ₱0.00    |

### expireQuotes(four days later)

Voided automatically: SU-219 — "Quotation expired — not paid within 3 days" by System.

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/structures.js · lines 341–346 · `stackPush()` — Pushes onto the stack.

```
341  /*                               Time O(1) · Space O(1) */
342  function stackPush(stack, value) {
343  ❶    stack.top++;
344  ❷    stack.items[stack.top] = value;
345      return stack.top + 1;
346  }
```

❶ line 343 Move the top up one...

❷ line 344 ...and store the value there.

Time O(1) · Space O(1)

js/structures.js · lines 348–355 · `stackPop()` — Pops the top.

```
348  /* Remove and return the top, or null.   Time O(1) · Space O(1) */
349  function stackPop(stack) {
350  ❶    if (stack.top === NIL) return null;
351  ❷    var value = stack.items[stack.top];
352      stack.items.length = stack.top;
353  ❸    stack.top--;
354      return value;
355  }
```

❶ line 350 Empty: nothing to undo.

❷ line 351 Take the top value...

❸ line 353 ...and move the top down.

Time O(1) · Space O(1)

js/orders.js · lines 721–741 · `undoCompletion()` — Undo the last "Mark ready".

```
721  /* Put the last completion back exactly where it came from: the front of
722     its lane in O(1) / O(log n), or its old place in O(n) if it had been
723     released from behind an order still owing.   Time O(n) worst · Space O(n) */
724  function undoCompletion(stamp) {
725  ❶    var entry = stackPop(completedStack);
726      if (entry === null) return { ok: false, error: 'There is nothing to undo.' };
727      var order = orderById(entry.id);
728      order.status = 'paid';
729      order.completedAt = 0;
```

```

730     if (order.lane === 'rush') {
731         // An order moved to the rush lane after completing has no arrival
732         // number yet; it takes the next one.
733         if (order.laneSeq < 0) order.laneSeq = rushLane.nextSeq;
734 ❷ heapInsert(rushLane, order.id, dueKey(order), order.laneSeq);
735     } else {
736 ❸ if (entry.position <= 0) cqRequeueFront(standardLane, order.id);
737 ❹ else cqInsertAt(standardLane, entry.position, order.id);
738     }
739     addHistory(order, stamp, 'Completion undone – back in production');
740     return { ok: true, order: order };
741 }

```

- ❶ Line 725 The most recent completion.
- ❷ Line 734 Rush: back in the heap with its original key.
- ❸ Line 736 Standard: back at the front...
- ❹ Line 737 ...or at its old place if it was released from behind an order still owing.

Time  $O(n)$  worst · Space  $O(n)$

**js/orders.js** · lines 743–762 · `completedByDate()` — Completed orders by day.

```

743  /* Completed orders grouped by the day they were completed, newest first.
744     Orders are kept oldest first and are mostly completed in that order, so
745     the list is read back to front before the insertion sort: it is then
746     nearly in order already, the sort's best case.
747                                     Time  $O(n)$  here (nearly in order),  $O(n^2)$  worst · Space  $O(n)$  */
748     function completedByDate() {
749 ❶ var sorted = insertionSort(backwards(ordersWithStatus('completed')), function (a, b) { return
    b.completedAt - a.completedAt || a.id - b.id; });
750     var groups = [];
751     for (var i = 0; i < sorted.length; i++) {
752         var day = isoFromStamp(sorted[i].completedAt);
753         var last = groups.length > 0 ? groups[groups.length - 1] : null;
754 ❷ if (!last || last.date !== day) {
755             last = { date: day, orders: [], total: 0 };
756             listAdd(groups, last);
757         }
758         listAdd(last.orders, sorted[i]);
759 ❸ last.total = roundMoney(last.total + orderTotal(sorted[i]));
760     }
761     return groups;
762 }

```

- ❶ Line 749 Read back to front, then insertion sort newest first — nearly in order already.
- ❷ Line 754 A new day starts a new group...
- ❸ Line 759 ...and the day total grows.

Time  $O(n)$  here (nearly in order),  $O(n^2)$  worst · Space  $O(n)$

**js/orders.js** · lines 768–785 · `voidOrder()` — Cancels, but keeps the record.

```

768  /*                                     Time  $O(n)$  · Space  $O(n)$  */
769     function voidOrder(id, reason, by, stamp) {
770         var order = orderById(id);
771         if (!order) return { ok: false, error: 'No such order.' };
772         if (order.status === 'completed') return { ok: false, error: order.ref + ' is completed and cannot be
    voided.' };
773         if (order.status === 'voided') return { ok: false, error: order.ref + ' is already voided.' };
774
775 ❶ if (order.status === 'requested') leaveQueue(queue, order.id);
776 ❷ if (order.status === 'paid') leaveProduction(order);

```

```

777 ❸ order.status = 'voided';
778    order.voidedAt = stamp;
779    order.voidedBy = by === 'admin' ? 'Order desk' : (by === 'system' ? 'System' : 'Customer');
780    order.voidReason = strip(reason) || (by === 'admin' ? 'Cancelled by the order desk' : 'Cancelled by the
customer');
781    addHistory(order, stamp, 'VOIDED – NON-REFUNDABLE' +
782      (order.amountPaid > 0 ? ' (' + peso(order.amountPaid) + ' retained)' : ''));
783 ❹ emailOrderVoided(order, stamp);
784    return { ok: true, order: order };
785  }

```

❶ line 775 Leave the quote queue...

❷ line 776 ...or the production line.

❸ line 777 Marked VOIDED – NON-REFUNDABLE; payments are retained.

❹ line 783 Tell the customer.

Time  $O(n)$  · Space  $O(n)$

**js/orders.js** · lines 787–798 · `expireQuotes()` — The automatic clean-up.

```

787  /* Void every quotation left unpaid for QUOTE_EXPIRY_DAYS, so the owner
788     never has to chase one. Run on start-up and every minute after.
789                                     Time  $O(n^2)$  · Space  $O(n)$  */
790  function expireQuotes(now) {
791    var limit = QUOTE_EXPIRY_DAYS * 24 * 3600000, expired = [];
792  ❶ var stale = keepWhere(orders, function (o) { return o.status === 'quoted' && now - o.quotedAt > limit;
});
793    for (var i = 0; i < stale.length; i++) {
794  ❷ voidOrder(stale[i].id, 'Quotation expired – not paid within ' + QUOTE_EXPIRY_DAYS + ' days',
'system', now);
795      listAdd(expired, stale[i]);
796    }
797    return expired;
798  }

```

❶ line 792 Quotations unpaid for more than three days...

❷ line 794 ...are voided by "System".

Time  $O(n^2)$  · Space  $O(n)$

# Order Desk

|         |                                                                                                                                                         |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screens | index.html#admin — sign-in, the six-digit code, the Overview (daily, monthly, yearly)                                                                   |
| Files   | js/admin-signin.js (deskSignIn, issueOtp, verifyOtp) · js/core.js (fnv1a) · js/reports.js (bucketOf, tallyPeriods, periodReport) · js/admin-overview.js |
| Data    | ADMIN_PASS_HASH and EMAIL_CONFIG.adminEmail; deskState (tries, lock, the code's hash); orders and receipts                                              |
| DSA     | <b>Two-step sign-in · Bucketing by date</b>                                                                                                             |

## 1 Business process

*What task or problem from the client are you solving?*

The owner runs the shop from the order desk, at index.html#admin — linked from nowhere on the shop. It needs a sign-in: email and password, then a six-digit code sent to the owner's email (two-step verification).

Once in, the Overview answers "how is the shop doing?" for any day, month or year: money collected, sales, orders placed, pieces sold, orders made ready, voids, and that period's best sellers. The owner can also edit any order or product.

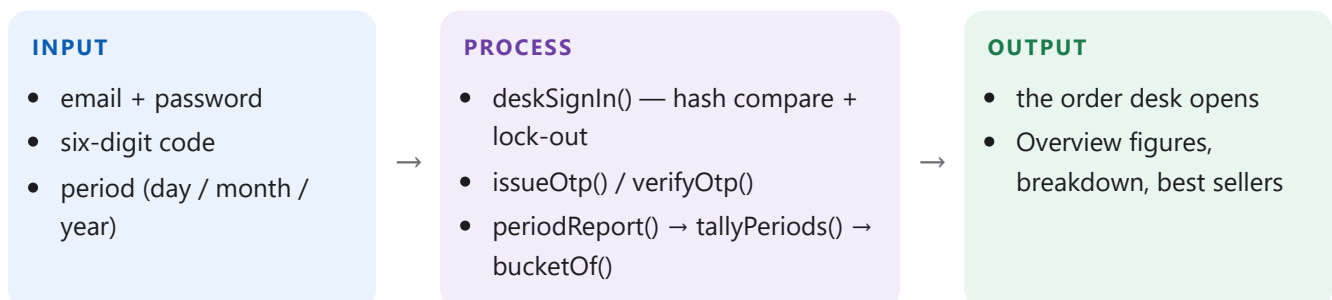
## 2 Input

*What data is needed by the module?*

| Input                                            | What it is                                               |
|--------------------------------------------------|----------------------------------------------------------|
| Owner types                                      | Email, password, the emailed code; a day, month or year. |
| Stored: ADMIN_PASS_HASH, EMAIL_CONFIG.adminEmail | The password's hash and the owner's email.               |
| Stored: orders, receipts                         | Everything the Overview counts.                          |

## 3 Process

*What data structure and/or algorithm did you use? How does it work step by step?*



**Step by step** (Two-step sign-in · Bucketing by date):

- 1 Sign-in: compare the email exactly and the password by its FNV-1a hash; five wrong tries lock the desk for 30 seconds.
- 2 Two-step: make a random six-digit code, keep only its hash, email it; it lasts five minutes, allows three tries, and a new one can be sent after 30 seconds (three per sign-in).

- 3 Overview: a period is a key "2026-10-05", "2026-10" or "2026", and a record falls in it when its date begins with the key. For the breakdown, `bucketOf()` reads the day (month view) or the month (year view) straight off the record's date and uses it as the row number, so one pass over the orders and receipts fills every row.

## 4 Output

*What result does the module produce?*

- The sign-in and code screens.
- The Overview: figure cards, day-by-day or month-by-month table, best sellers.
- The tabs: quote requests, payments, production, completed, voided, products, outbox.

## 5 Complexity analysis

*What is the time complexity? What is the space complexity?*

| Operation                   | Time                                | Space  | Note                             |
|-----------------------------|-------------------------------------|--------|----------------------------------|
| <code>deskSignIn()</code>   | $O(n)$                              | $O(1)$ | Step 1: email and password.      |
| <code>issueOtp()</code>     | $O(1)$                              | $O(1)$ | Makes a code.                    |
| <code>verifyOtp()</code>    | $O(n)$                              | $O(1)$ | Step 2: the code.                |
| <code>bucketOf()</code>     | $O(1)$                              | $O(1)$ | Which row a moment belongs to.   |
| <code>tallyPeriods()</code> | $O(n^2)$ – the lines of every order | $O(1)$ | One pass fills every row.        |
| <code>periodReport()</code> | $O(n^2)$                            | $O(n)$ | The whole Overview for a period. |

**What  $n$  means here:** For hashing,  $n$  = characters of the password or code. For the Overview,  $n$  = orders and receipts — and each order's lines are added up (a loop inside a loop).

Each Time and Space is read from the comment above the function in the source, where every function in `js/` states its own cost.

## 6 Justification

*Why did you choose that data structure or algorithm? Why is it appropriate?*

Storing hashes (not the password or the code) means the page never holds them as text. Codes expire and are limited in tries, so guessing is impractical. This keeps customers off the desk; real security needs a server, because anyone can read a browser-only site's code.

Every date in the system is "YYYY-MM-DD", so "is this in October 2026?" is "does it begin with 2026-10?" — one rule handles days, months and years, and a year is the sum of its months.

Reading each record's own day or month as its row number fills all 31 rows of a month in one pass, instead of scanning every order again for each row.

## 7 Demonstration

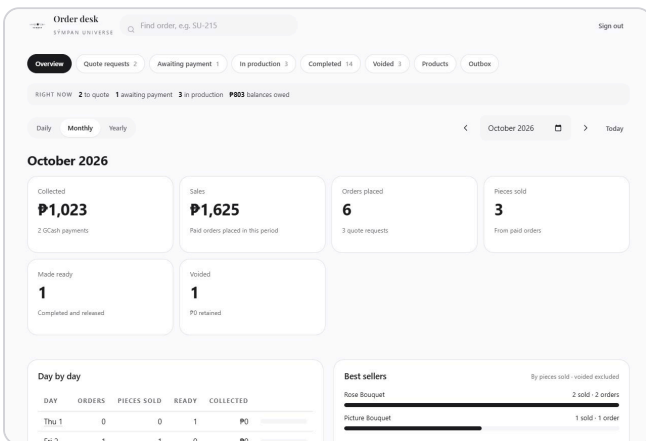
*Show that the module works correctly — the input, the process and the output.*

- 1 Open `index.html#admin`. Sign in with `kurlchester31feliciano@gmail.com` and `admin123`.
- 2 Enter the six-digit code (emailed; shown on screen while `EmailJS` is not set up).

3 The Overview opens on today. Switch to Monthly, then Yearly; use < > to move.

Step 1: email and password

Step 2: the six-digit code



The Overview, monthly

## Trace — the sign-in and the Overview (computed by our code)

### deskSignIn() — step 1

| Email / password                                | Result                                             |
|-------------------------------------------------|----------------------------------------------------|
| "kurlchester31feliciano@gmail.com" / "wrong"    | That email or password is not right. 4 tries left. |
| "someone@gmail.com" / "admin123"                | That email or password is not right. 3 tries left. |
| "kurlchester31feliciano@gmail.com" / "admin123" | accepted → code step                               |

Password hash stored in data.js: 1883603724; fnv1a("admin123") = 1883603724.

### issueOtp() and verifyOtp() — step 2

| Action              | Result                                                             |
|---------------------|--------------------------------------------------------------------|
| issueOtp()          | code 919916 — only its hash 41796610 is kept; expires in 5 minutes |
| verifyOtp("000000") | That code is not right. 2 tries left.                              |
| verifyOtp("919916") | signed in (authed = true)                                          |

**periodReport("month", "2026-09")**

| Figure        | Value                    |
|---------------|--------------------------|
| Collected     | ₱10,522.50 (18 payments) |
| Sales         | ₱9,895.00                |
| Orders placed | 14                       |
| Pieces sold   | 13                       |
| Made ready    | 12                       |
| Voided        | 2 (₱235.00 retained)     |

Day by day (days with activity). All 30 day rows were filled by one tallyPeriods() pass: bucketOf() read each order's and receipt's date and returned its day number as the row index — 120 bucketOf() calls in all for the month view, instead of one scan of every order per day.

| Day        | Orders | Pieces | Ready | Collected |
|------------|--------|--------|-------|-----------|
| 2026-09-01 | 0      | 0      | 1     | ₱592.50   |
| 2026-09-02 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-03 | 0      | 0      | 1     | ₱375.00   |
| 2026-09-05 | 1      | 2      | 0     | ₱0.00     |
| 2026-09-06 | 0      | 0      | 1     | ₱1,115.00 |
| 2026-09-07 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-08 | 0      | 0      | 1     | ₱750.00   |
| 2026-09-09 | 1      | 1      | 0     | ₱915.00   |
| 2026-09-10 | 0      | 0      | 2     | ₱0.00     |
| 2026-09-11 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-12 | 0      | 0      | 0     | ₱1,050.00 |
| 2026-09-13 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-14 | 0      | 0      | 0     | ₱870.00   |
| 2026-09-15 | 1      | 0      | 1     | ₱1,050.00 |
| 2026-09-16 | 0      | 0      | 1     | ₱0.00     |
| 2026-09-17 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-18 | 0      | 0      | 0     | ₱310.00   |
| 2026-09-19 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-20 | 0      | 0      | 0     | ₱1,210.00 |
| 2026-09-21 | 1      | 1      | 1     | ₱310.00   |
| 2026-09-22 | 0      | 0      | 0     | ₱650.00   |
| 2026-09-23 | 1      | 0      | 1     | ₱0.00     |
| 2026-09-24 | 0      | 0      | 1     | ₱235.00   |
| 2026-09-25 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-26 | 0      | 0      | 0     | ₱455.00   |

| Day        | Orders | Pieces | Ready | Collected |
|------------|--------|--------|-------|-----------|
| 2026-09-27 | 1      | 1      | 0     | ₱0.00     |
| 2026-09-28 | 0      | 0      | 0     | ₱435.00   |
| 2026-09-29 | 1      | 1      | 1     | ₱0.00     |
| 2026-09-30 | 0      | 0      | 0     | ₱200.00   |

### periodReport("year", "2026") — a year is the sum of its months

Year collected: ₱11,810.00; sum of the 12 month rows: ₱11,810.00. Best sellers of the year: Rose Bouquet (7), Money Bouquet (3), Plumeria Bouquet (2), Dahlia Bouquet (1), Makeup Bouquet (1).

## Code walkthrough

The real functions, with their line numbers in the files today. The numbered markers in the code match the explanation under it.

js/admin-signin.js · lines 35–56 · deskSignIn() — Step 1: email and password.

```

35  /* Step 1. Returns '' when the code can be sent, or the message to show.
36     Never says which of the two was wrong. Time O(n) · Space O(1) */
37  function deskSignIn(email, password, now) {
38  ❶    if (now < deskState.lockedUntil) {
39      return 'Too many attempts. Try again in ' + Math.ceil((deskState.lockedUntil - now) / 1000) + '
seconds.';
40    }
41  ❷    var emailOk = toLower(strip(email)) === toLower(strip(EMAIL_CONFIG.adminEmail));
42  ❸    if (emailOk && fnv1a(password) === ADMIN_PASS_HASH) {
43      deskState.attempts = 0;
44      deskState.stage = 'otp';
45      deskState.codesSent = 0;
46      return '';
47    }
48    deskState.attempts++;
49    if (deskState.attempts >= ADMIN_MAX_ATTEMPTS) {
50      deskState.attempts = 0;
51  ❹    deskState.lockedUntil = now + ADMIN_LOCK_MS;
52      return 'Too many attempts. The desk is locked for 30 seconds.';
53    }
54    return 'That email or password is not right. ' + (ADMIN_MAX_ATTEMPTS - deskState.attempts) + ' ' +
plural(ADMIN_MAX_ATTEMPTS - deskState.attempts, 'try', 'tries') + ' left.';
55  }
56  }
```

- ❶ Line 38 Locked? Say how long.
- ❷ Line 41 The email must match exactly.
- ❸ Line 42 The password is compared by its hash.
- ❹ Line 51 Five misses lock the desk for 30 seconds.

Time O(n) · Space O(1)

js/admin-signin.js · lines 58–65 · issueOtp() — Makes a code.

```

58  /* Make a fresh code and keep only its hash. Returns the code so it can be
59     emailed. Time O(1) · Space O(1) */
60  function issueOtp(now) {
61  ❶    var code = otpCode();
62      deskState.codesSent = (deskState.codesSent || 0) + 1;
63  ❷    deskState.otp = { hash: fnv1a(code), expires: now + OTP_LIFETIME_MS, tries: 0, sentAt: now };
64      return code;
65  }
```

65 }

❶ Line 61 Six random digits from the browser's secure random source.

❷ Line 63 Keep only its hash, its expiry and a try counter.

Time  $O(1)$  · Space  $O(1)$

js/admin-signin.js · lines 75–91 · verifyOtp() — Step 2: the code.

```
75  /* Step 2. Returns '' when signed in, or the message to show.
76                                Time  $O(n)$  · Space  $O(1)$  */
77  function verifyOtp(code, now) {
78      var otp = deskState.otp;
79      if (deskState.stage !== 'otp' || !otp) return 'Start again with your email and password.';
80  ❶ if (now > otp.expires) { deskState.otp = null; return 'That code has expired. Send a new one.'; }
81      if (otp.tries >= OTP_MAX_TRIES) return 'Too many wrong codes. Send a new one.';
82  ❷ if (fnv1a(digitsOnly(code)) === otp.hash && digitsOnly(code).length === OTP_LENGTH) {
83          deskState.authed = true;
84          deskState.stage = 'password';
85          deskState.otp = null;
86          return '';
87      }
88  ❸ otp.tries++;
89      var left = OTP_MAX_TRIES - otp.tries;
90      return left > 0 ? 'That code is not right. ' + left + ' ' + plural(left, 'try', 'tries') + ' left.' :
    'Too many wrong codes. Send a new one.';
91  }
```

❶ Line 80 Expired after five minutes.

❷ Line 82 Right code: signed in.

❸ Line 88 Wrong code: one try fewer (three in all).

Time  $O(n)$  · Space  $O(1)$

js/reports.js · lines 49–60 · bucketOf() — Which row a moment belongs to.

```
49  /* Which bucket a moment falls in: -1 outside the period; otherwise 0, or
50     with a breakdown the day of the month ('day') or the month of the year
51     ('month') minus one - the bucket's index, read straight off the date.
52                                Time  $O(1)$  · Space  $O(1)$  */
53  function bucketOf(stamp, key, part) {
54      if (!(stamp > 0)) return -1;
55      var iso = isoFromStamp(stamp);
56  ❶ if (!beginsWith(iso, key)) return -1;
57  ❷ if (part === 'day') return readNumber(iso, 8, 10) - 1;
58  ❸ if (part === 'month') return readNumber(iso, 5, 7) - 1;
59      return 0;
60  }
```

❶ Line 56 Outside the period: no row.

❷ Line 57 Month view: the day of the month is the row.

❸ Line 58 Year view: the month is the row.

Time  $O(1)$  · Space  $O(1)$

js/reports.js · lines 62–96 · tallyPeriods() — One pass fills every row.

```
62  /* Add every order and receipt into the bucket its date picks. One pass
63     fills every bar of the chart at once; nothing is read twice.
64                                Time  $O(n^2)$  - the lines of every order · Space  $O(1)$  */
65  function tallyPeriods(buckets, key, part) {
```

```

66     for (var o = 0; o < orders.length; o++) {
67 ❶     var order = orders[o], at = bucketOf(order.createdAt, key, part);
68         if (at >= 0) {
69             var t = buckets[at];
70             t.placed++;
71             if (countWhere(order.items, function (item) { return item.kind === 'quote'; }) > 0)
                t.quoteRequests++;
72             if (countsAsSale(order)) {
73                 t.sales = roundMoney(t.sales + orderTotal(order));
74 ❷     t.units += sumRecursive(order.items, function (item) { return item.quantity; });
75             }
76         }
77         if (order.status === 'completed') {
78             at = bucketOf(order.completedAt, key, part);
79             if (at >= 0) buckets[at].completed++;
80         }
81         if (order.status === 'voided') {
82             at = bucketOf(order.voidedAt, key, part);
83             if (at >= 0) {
84                 buckets[at].voided++;
85                 buckets[at].retained = roundMoney(buckets[at].retained + order.amountPaid);
86             }
87         }
88     }
89     for (var r = 0; r < receipts.length; r++) {
90 ❸     at = bucketOf(receipts[r].stamp, key, part);
91         if (at >= 0) {
92             buckets[at].collected = roundMoney(buckets[at].collected + receipts[r].amount);
93             buckets[at].payments++;
94         }
95     }
96 }

```

❶ line 67 Each order goes straight to its row...

❷ line 74 ...adding its pieces if it was paid.

❸ line 90 Money collected: each receipt to its row.

Time  $O(n^2)$  – the lines of every order · Space  $O(1)$

**js/reports.js** · lines 106–133 · `periodReport()` — The whole Overview for a period.

```

106  /* Everything the Overview shows for one period, with a breakdown one
107     level down: the day's orders, the month's days, the year's months.
108     The breakdown's bars are filled by one tallyPeriods pass.
109                                     Time  $O(n^2)$  · Space  $O(n)$  */
110  function periodReport(kind, key) {
111     var rows = [], part = '';
112     if (kind === 'year') {
113 ❶     part = 'month';
114         for (var m = 1; m <= 12; m++) listAdd(rows, { key: key + '-' + twoDigits(m), label: MONTH_SHORT[m - 1], totals: emptyTotals() });
115         } else if (kind === 'month') {
116 ❷     part = 'day';
117         var days = daysInMonth(key);
118         for (var d = 1; d <= days; d++) listAdd(rows, { key: key + '-' + twoDigits(d), label: String(d), totals: emptyTotals() });
119     }
120     if (rows.length > 0) {
121         var buckets = [];
122         for (var b = 0; b < rows.length; b++) listAdd(buckets, rows[b].totals);
123 ❸     tallyPeriods(buckets, key, part);
124     }
125     var placed = kind === 'day'

```

```

126         ? insertionSort(keepWhere(orders, function (o) { return inPeriod(o.createdAt, key); })), function
    (a, b) { return a.createdAt - b.createdAt; })
127         : [];
128     return {
129         kind: kind, key: key, label: periodLabel(kind, key),
130         totals: periodTotals(key), rows: rows, orders: placed,
131         ④ best: topSellers(salesTally(function (o) { return inPeriod(o.createdAt, key); })), 5)
132     };
133 }

```

- ① line 113 A year: twelve month rows...
- ② line 116 ...a month: one row per day.
- ③ line 123 One pass over the orders and receipts fills them all.
- ④ line 131 The period's best sellers (module 1).

Time  $O(n^2)$  · Space  $O(n)$

# Complexity summary — all 12 modules

The main operation of each module, written with only the four notations from class:  $O(1)$ ,  $O(\log n)$ ,  $O(n)$  and  $O(n^2)$ . Details are on each module's pages.

| #  | Module                    | Part   | Main structure / algorithm                                | Time                                                 | Space  |
|----|---------------------------|--------|-----------------------------------------------------------|------------------------------------------------------|--------|
| 1  | Catalogue & Best Sellers  | Part 1 | Hash-table tally · selection sort (stable)                | $O(n^2)$                                             | $O(n)$ |
| 2  | Category Navigation       | Part 1 | N-ary tree · depth-first search with a stack              | $O(n)$                                               | $O(n)$ |
| 3  | Search & Sort             | Part 1 | Linear search + naive string matching · insertion sort    | $O(n^2)$                                             | $O(n)$ |
| 4  | Flower Customiser         | Part 2 | Hash table (FNV-1a, chaining) · recursive sum             | $O(1)$ average                                       | $O(n)$ |
| 5  | Gift Customiser           | Part 2 | Greedy change-making                                      | $O(n)$                                               | $O(n)$ |
| 6  | Cart                      | Part 2 | Singly linked list                                        | $O(1)$ append · $O(n)$ remove                        | $O(n)$ |
| 7  | Checkout & Delivery       | Part 3 | Hash table (city → area) · character-scan validation      | $O(1)$ average look-up · $O(n^2)$ form check         | $O(n)$ |
| 8  | Quotation Desk            | Part 3 | Circular queue · insertion sort of past quotes            | $O(1)$ enqueue / dequeue · $O(n^2)$ draft            | $O(n)$ |
| 9  | Payment, Receipts & Email | Part 3 | Circular queue (mail) · linear search for used references | $O(1)$ per email · $O(n)$ reference check            | $O(n)$ |
| 10 | Production & Tracking     | Part 4 | Min-heap · circular queue · binary search                 | $O(\log n)$ heap · $O(1)$ queue · $O(\log n)$ find   | $O(n)$ |
| 11 | Order Records             | Part 4 | Stack (undo) · insertion sort by date                     | $O(1)$ push / pop · $O(n)$ sort when nearly in order | $O(n)$ |
| 12 | Order Desk                | Part 4 | FNV-1a hash compare · one-pass bucketing by date          | $O(n)$ sign-in · $O(n^2)$ report                     | $O(n)$ |

## How we count

- Only the four notations from class:  $O(1)$ ,  $O(\log n)$ ,  $O(n)$  and  $O(n^2)$  — with "average", "amortised", "best" or "worst" where it matters, and sums such as  $O(\log n) + O(n)$ .
- $n$  is the size of whatever the function goes through: records in a table (orders, lines, products, emails, receipts), items in a queue, or the characters of a text when the text itself is the job (validators, hashing, string search).
- A loop inside a loop is  $O(n^2)$ . Naive string matching over a table is  $O(n^2)$ .
- Work on one record's own fields — escaping or formatting its text, copying it — and look-ups in the shop's short fixed lists (20 colours, 2 arrangements, 4 add-ons, 4 time slots, 2 payment methods) count as constant work inside a loop over records: those lists are part of the program, not data that grows.
- An  $O(\log n)$  step done once per item (a binary search per queued order) is written " $O(n^2)$  at most".

## Reading Big-O in one minute

### What it means

Big-O describes how the work grows when the data grows, ignoring fixed costs.  $O(\log n)$  grows very slowly (double the data  $\rightarrow$  one more step);  $O(n)$  grows in step with the data;  $O(n^2)$  grows with the square (double the data  $\rightarrow$  four times the work).

### Time vs. space

Time counts the steps (comparisons, moves). Space counts the extra memory a function needs besides its input — a sort that works on a copy is  $O(n)$  space; a binary search is  $O(1)$ .

# General questions — any member may be asked

## Why is the whole website one HTML page?

Our data lives in JavaScript arrays, and arrays exist only while the page is open — they cannot be carried to another HTML file. So the shop and the order desk are sections of one `index.html`, shown and hidden; the part after `#` in the address picks the view. The arrays stay alive while you move between screens.

## Where is the data saved? What happens when you refresh?

Only in memory (the arrays in `js/store.js`). There is no `localStorage`, cookie or server. Refreshing the page replays the demo history again — expected under the project rule, and it means every demonstration starts from the same state.

## Why didn't you use push, sort, indexOf, filter and the other built-ins?

It was a project rule, and it shows we understand what they do. Each has our own version: `listAdd` (push), `copyRange` (slice), `positionIn` (indexOf), `keepWhere` (filter), `firstWhere` (find), `renderEach` (map + join), `insertionSort` (sort), `cutText` (split), `strip` (trim), `toLower` (toLowerCase) — and no regular expressions.

## What does "procedural, not OOP" mean in your code?

There are no classes, no `this` and no methods. Data is plain records (a queue is `{ items, head, tail, count, capacity }`) and every operation is a separate function that receives the record: `cqEnqueue(queue, value)`, `stackPush(stack, value)`, `hashPut(table, key, value)`.

## Why insertion and selection sort, and not merge sort or quick sort?

The course allows only  $O(1)$ ,  $O(\log n)$ ,  $O(n)$  and  $O(n^2)$ ; merge sort is  $O(n \log n)$ . Quick sort is not stable and its worst case is  $O(n^2)$  anyway. Our lists are small and usually almost in order — orders, emails and receipts are appended as they happen — which is insertion sort's best case,  $O(n)$ . Selection sort ranks the ten best sellers with a fixed, easy-to-show cost.

## Aren't $O(n^2)$ sorts too slow?

Not for our data: the lists hold tens to hundreds of records, and we read them in the order that leaves them nearly sorted. The Order Records trace shows it: 15 completed orders sorted newest first take 15 comparisons, not the worst case of 105.

## Why hash tables, when you already have binary search?

Binary search needs an array sorted by the key you search for. The photographs are looked up by "flower|arrangement|colour" and the delivery areas by any spelling of a city name — keys that are not stored in order. A hash table answers those in  $O(1)$  on average.

## How does undo work?

It is a stack. Each "Mark ready" pushes the order id and its place in the production line. Undo pops the newest entry — last in, first out — and puts the order back exactly there.

## How are emails sent without a server?

Every email is written into the outbox array and its position queued. Every 1.5 seconds the sender takes the front of the queue and posts it to EmailJS (a web service that sends email for a browser page). Demo customers (example.com) are never emailed. If EmailJS is not set up, the email is kept in the Outbox marked "not sent", and the sign-in code is shown on screen instead.

### **How secure is it?**

It is a class project that runs entirely in the browser, so we are honest about the limits. The password and the sign-in code are kept only as FNV-1a hashes, codes expire after five minutes and allow three tries, five wrong passwords lock the desk for 30 seconds, the order desk is linked from nowhere on the shop, and everything a customer types is escaped before it is shown. A real deployment needs a server, a proper password hash (bcrypt or Argon2) and HTTPS, because anything in a browser can be read with developer tools.

### **How do you prove the complexities?**

Each function states its cost above its code, and tests/rubric.js fails if one is missing or uses another notation. The unit tests count insertion sort's comparisons on sorted and reversed input ( $n - 1$  and  $n(n - 1)/2$ ). The traces in this reviewer were produced by running our actual functions on the demo data.

### **Did you use the Date object?**

Only to read the clock: readClock() reads the time and the time zone, and Date.now() gives timestamps. All date maths — adding days, the weekday, month lengths, leap years, the day a timestamp falls on — is our own day-number arithmetic in js/core.js.

### **What would you improve with more time?**

A real server and database so data survives a refresh; online payment through the payment provider instead of typed references; SMS updates; and a balanced tree or database index once the order list grows to many thousands.

# Glossary

| Term                  | Meaning                                                                                                                                           |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Array                 | Items in numbered slots (0, 1, 2 ...); reading slot $i$ is $O(1)$ . Our "tables".                                                                 |
| Key                   | The field a structure is ordered or looked up by (order number, product id, city name ...).                                                       |
| Linear search         | Check items one by one until a match. $O(n)$ ; works on unsorted data.                                                                            |
| Binary search         | On a sorted array, compare with the middle and keep one half. $O(\log n)$ .                                                                       |
| Naïve string matching | Try the search word at every position of a text, letter by letter. $O(n^2)$ over a table.                                                         |
| Insertion sort        | Take each item and slide the larger ones before it one place right until its spot is found. $O(n)$ when already in order, $O(n^2)$ worst. Stable. |
| Selection sort        | For each position, find the item that belongs there among the rest. Always $n(n-1)/2$ comparisons, $O(n^2)$ .                                     |
| Stable sort           | Equal items keep their original order.                                                                                                            |
| Linked list           | Nodes that each point to the next. Adding at a known end is $O(1)$ ; removing re-points one link.                                                 |
| Stack (LIFO)          | Last in, first out: push and pop at the top. Used for undo and the depth-first tree walks.                                                        |
| Queue (FIFO)          | First in, first out: enqueue at the rear, dequeue at the front. Used for quote requests, standard production and email.                           |
| Circular buffer       | A queue in a fixed array whose rear wraps round to the start, so nothing is shifted: $O(1)$ enqueue and dequeue.                                  |
| Min-heap              | A tree kept in an array where every parent comes before its children; the smallest key is always at index 0. Insert and extract are $O(\log n)$ . |
| N-ary tree            | Each node can have any number of children. Our category tree: a root, 3 branches, 10 lines.                                                       |
| Depth-first search    | Visit a node, then go as deep as possible before backtracking — here with an explicit stack.                                                      |
| Hash table            | A hash function turns a key into a bucket number, so a look-up checks one short list. $O(1)$ on average.                                          |
| FNV-1a                | Our string hash: start at 2166136261; for each character XOR it in, then multiply by 16777619 (kept to 32 bits).                                  |
| Collision / chaining  | Two keys in the same bucket; they are kept in a chain (a short list) inside that bucket.                                                          |
| Load factor           | Keys $\div$ buckets ( $160 \div 199 \approx 0.8$ for the photographs).                                                                            |
| Recursion             | A function that calls itself on a smaller problem, ending at a base case.                                                                         |
| Greedy algorithm      | Take the best choice at each step (the largest note that fits) without looking back.                                                              |
| Amortised             | The average cost per operation over many operations — enqueue is $O(1)$ amortised although a rare grow is $O(n)$ .                                |
| Big-O                 | How the work grows with the size of the data (worst case unless stated).                                                                          |

## File map — who explains what

| Part                     | Rules and structures (js/)                                                                                                                                                                       | Screens (js/)                                                         |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| 1 · The shop window      | store.js (salesTally, topSellers) · algorithms.js (selectionSortDesc, linearSearch, insertionSort) · structures.js (tree, stack) · core.js (textHas)                                             | shop-catalog.js                                                       |
| 2 · Choosing a gift      | structures.js (hash table, linked list) · store.js (buildReferenceIndex, makeFlowerItem, makeQuoteItem, basket) · algorithms.js (sumRecursive, breakIntoBills)                                   | shop-product.js · shop-basket.js                                      |
| 3 · Checkout and payment | store.js (lookupArea, deliveryQuote) · orders.js (validateRequest, quoteHistory, sendQuote, placeOrder, acceptQuote, issueReceipt) · notify.js · mail.js                                         | shop-basket.js · shop-track.js · admin-orders.js · receipt.js         |
| 4 · Behind the counter   | structures.js (heap, circular queue, stack) · orders.js (productionLine, completeNext, undoCompletion, completedByDate, voidOrder) · algorithms.js (binarySearch) · reports.js · admin-signin.js | admin-desk.js · admin-overview.js · admin-products.js · shop-track.js |